$ mortgage-calculator --help

Introduction to Software Engineering in the Age of AI

A hands-on introduction,
built one system at a time

<mortgage_calculator />

Introduction to Software Engineering in the Age of AI

A hands-on introduction, built one working system at a time

Robert Ioffe

2026

Contents

Introduction to Software Engineering in the Age of AI

To my wife, Ellen, for her patience and support.

Who This Book Is For

This book is written for first-year students and casual learners — people who are curious about software engineering and willing to sit down and build something real, but who don't yet have a professional engineering background to draw on.

Here's what we assume you *do* have: comfort using a computer (you can find a file, open a terminal, install something), and enough curiosity about numbers to follow a formula when it's explained rather than just handed to you. Here's what we assume you *don't* have: prior experience with Python, git, or any of the tools this book introduces. If you've never opened a terminal before, Chapter 0 starts there, on purpose.

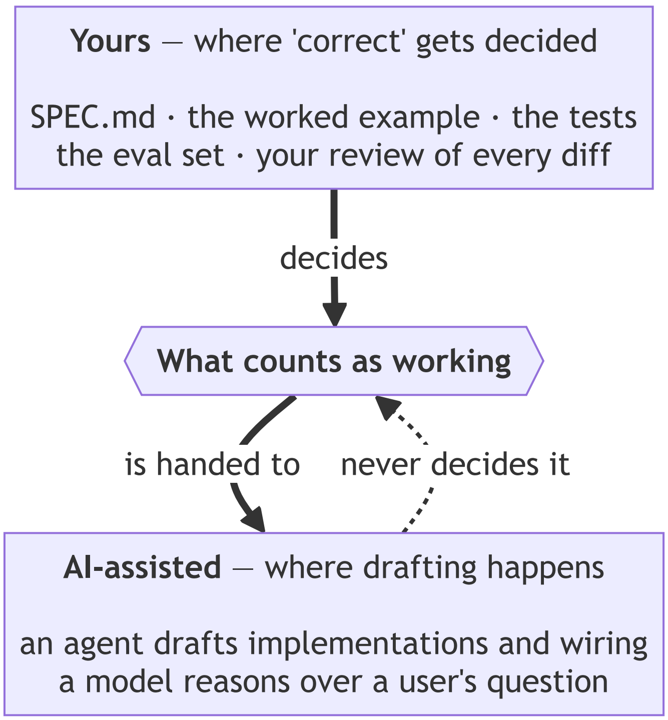
We should also say plainly who this book is *not* for. If you're already a working software engineer looking for advanced patterns in agentic development — multi-agent orchestration, production-grade evaluation infrastructure, enterprise deployment — you'll outgrow this book quickly, and that's fine. This is a first book, not a deep one. We'd rather tell you that now than have you discover it forty pages in.

What "Hybrid" Means in This Book

You'll see the word *hybrid* a lot in the chapters ahead, so it's worth defining before we use it for the first time.

A hybrid system, as this book means it, has two parts: a core that you design, write, and verify yourself — and an AI-assisted layer built around that core, where an agent drafts and a model reasons, but neither one gets to decide what "correct" means. That decision stays with you, and it gets written down, in three different forms, at three different points in the book: first as a plain-language spec, then as executable tests, then as a formal schema a language model can call. Same contract, three altitudes. You'll notice this pattern recur — we've flagged it with a callout box each time it shows up, because it's the actual idea this book is trying to teach, more than any individual tool.

The boundary that definition draws is the one thing worth carrying through every chapter that follows, so here it is as a picture before it's a practice:



The heavy arrows run downward on purpose. Everything in the top box is something you write, and together they settle what "working" means; the assisted layer at the bottom works *from* that settlement, constantly and usefully. The dotted arrow going back up is the one this book spends fourteen chapters teaching you not to draw.

This split — human-verified core, AI-assisted layer — isn't a compromise or a hedge. It's the whole point. A book that taught you to let an agent write everything would be teaching you to trust something you can't yet evaluate. A book that banned AI assistance entirely would be pretending the tools in Chapter 1 don't exist, or don't matter. This book tries to do something more useful than either: show you exactly where the boundary between "yours" and "assisted" belongs, and give you enough practice drawing that line yourself that you can keep drawing it correctly long after you've finished the last chapter.

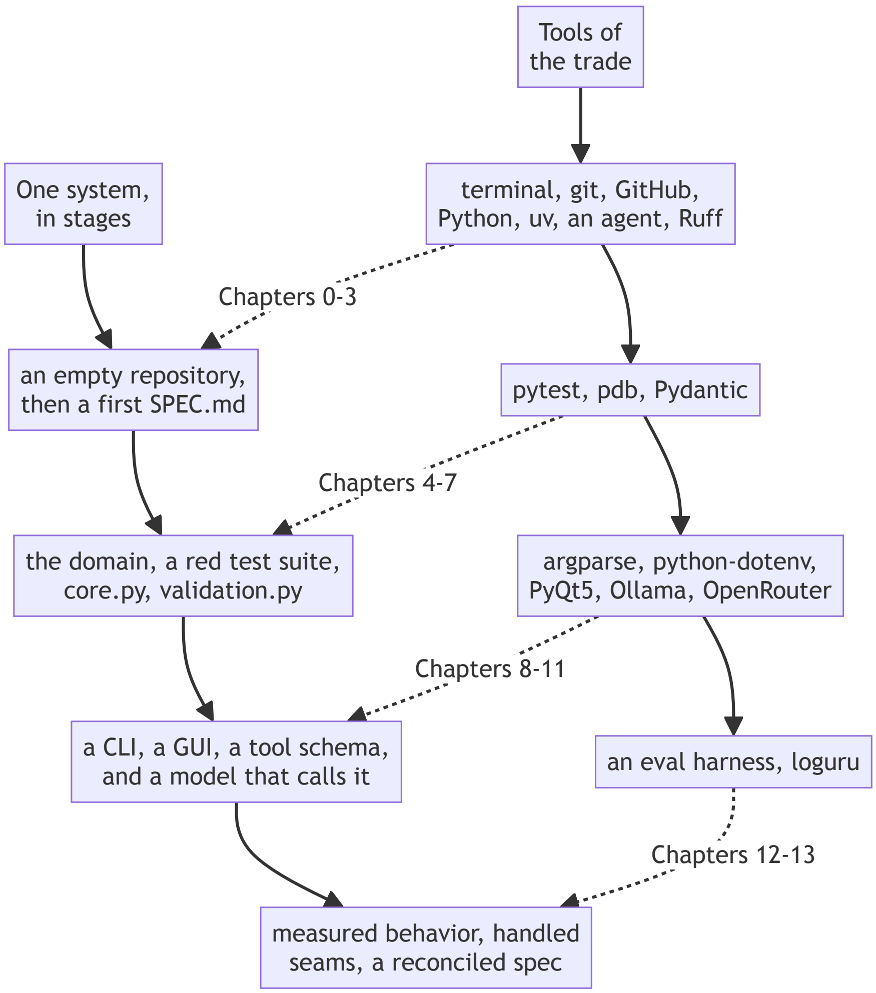
How This Book Is Structured

Two threads run through this book side by side, braided rather than stacked.

The first is **tools of the trade** — the terminal, git, Python, a coding agent, local and hosted language models, and everything in between. We introduce each tool right before the chapter that needs it, not all at once up front. You won't get a 300-page tools reference before you're allowed to write a line of code.

The second is **one system, built in stages**: a hybrid mortgage calculator, extended chapter by chapter from an empty repository to a working application with three different front ends — a command line, a graphical interface, and a language model that can operate it on your behalf. Every chapter leaves that system a little more complete, and a little more trustworthy, than it found it.

Braided, the two threads look like this — each group of tools arriving just before the stage of the system that needs it:



The dotted arrows are the braid: no tool appears in the top row until the bottom row has a use for it. That's why Pydantic shows up in the same group as validation rather than in a setup chapter, and why loguru waits until there are seams worth logging.

Most build chapters follow the same shape: a goal, the design decisions behind it, a test-driven development cycle, an implementation (often agent-assisted), an honest look at what the agent's role actually was, and a checkpoint — a clear statement of what should be true about your system right now. We're naming that shape once, here, so that by Chapter 3 it feels familiar instead of repetitive. Alongside it, you'll see two recurring devices: a **definition of done** checklist that grows a line or two with each chapter, and **process concept** callout boxes marking ideas — like TDD, or "the spec was wrong, not the code" — that matter well beyond the chapter they first appear in.

How to Use This Book

This book is meant to be built along with, not just read. You'll get more out of every chapter by actually typing the commands, running the tests, and reading the diffs an agent proposes than by reading a description of someone else doing it. That said, if you want to read it straight through first to get the shape of the thing before building anything, that works too — just know that the "definition of done" checklists and worked examples are written for someone with a terminal open.

Before Chapter 0, you need one thing: a computer capable of running a local language model. We'll help you figure out whether yours qualifies in Chapter 1, with a plain hardware gut-check rather than a spec sheet you have to interpret yourself. If your setup turns out not to be enough, or gives you trouble, Chapter 1 also includes an escape hatch — a path to a hosted model that keeps you moving without derailing the rest of the book. You can come back to running things locally later, once the pressure's off.

A note on pacing: every chapter ends at a checkpoint, and every checkpoint describes exactly what should be true about your project at that point. If something's not working, that checkpoint is there so you can compare against a known-good state instead of debugging in isolation, wondering how far back the problem goes. Use it.

Finally, permission to skip things: the Appendix covers tools and practices — Docker, CI/CD, type checking, and a few others — that are genuinely useful but not required to finish this book. Skip it entirely on your first pass. It'll still be there when a future project actually needs it.

A Note on What This Book Doesn't Cover

In the interest of finishing a book you'll actually complete, we left some real, valuable things out on purpose: Docker, continuous integration, static type checking, pre-commit automation, and a few others. None of them are missing by accident, and none of them are things we think you shouldn't eventually learn — they're just overhead this particular project, at this particular level, doesn't need yet. Each one gets a short, honest description in the Appendix: what it does, what it would have added here, and where to go looking when you're ready for it.

That's the scope discipline this whole book tries to model, starting now, before

Chapter 0 even begins: know what you're building, know what you're deliberately leaving out, and write both of those things down.

— *Robert Ioffe*

Portland, Oregon

September 1, 2026

Chapter 0 — Orientation

Contents

0.1 Why This Chapter Exists

Every chapter after this one assumes you have a few things already in place: a terminal you're not afraid of, a git repository that's tracking your work, and a Python project set up the way the rest of this book expects. This chapter builds all of that. Nothing here is specific to mortgages, AI, or agents — it's the ground floor everything else stands on.

By the end of this chapter, you'll have an empty-but-real project: initialized, version-controlled, pushed to GitHub, and ready to hand to a coding agent in Chapter 1.

0.2 The Terminal

0.2.1 Opening a Terminal

- **macOS**: open Spotlight (Cmd+Space), type `Terminal`, press Enter.
- **Linux**: almost every distribution ships a terminal app in its applications menu; on many, Ctrl+Alt+T opens one directly.
- **Windows**: install Windows Terminal from the Microsoft Store, and use it to run WSL (Windows Subsystem for Linux) rather than the classic Command Prompt. This book assumes a Unix-like shell throughout; WSL gives you one without leaving Windows.

Whichever you use, you should end up looking at a mostly-empty window with a blinking cursor next to a **prompt** — something like `robert@machine ~ %`. That prompt is waiting for you to type a command.

0.2.2 Navigation

Three commands get you almost everywhere:

```
pwd            # print working directory - "where am I?"
ls             # list what's in the current directory
cd Documents   # change directory - "go here"
```

A few navigation shortcuts worth knowing immediately:

```
cd ..       # go up one level
cd ~        # go to your home directory
cd -        # go back to wherever you just were
```

Paths can be **relative** (`cd Documents`, starting from where you are) or **absolute** (`cd /Users/robert/Documents`, starting from the very top). When in doubt, `pwd` tells you where "where you are" actually is.

0.2.3 File Basics

```
mkdir mortgage-calculator-book  # make a new directory
```

```
touch notes.txt                  # create an empty file
cat notes.txt                    # print a file's contents
rm notes.txt                     # delete a file
```

That last one deserves its own line: **`rm` does not ask twice, and it does not go to a recycle bin.** Get in the habit of double-checking what you're about to delete, especially once wildcards enter the picture (`rm *.txt` deletes *every* `.txt` file in the current directory, no confirmation).

0.2.4 Why the Terminal Matters More Now, Not Less

It might seem like an AI-assisted book would need the terminal less than an older one — surely the agent handles the typing? In practice, the opposite is true. Pi, the coding agent you'll meet in Chapter 1, does its work *in* a terminal: reading files, running commands, executing tests. Understanding the terminal isn't a prerequisite you'll outgrow once the agent shows up — it's how you'll read what the agent is actually doing, and how you'll independently verify it.

0.3 Git Basics

0.3.1 What Version Control Is Actually For

Git is often introduced as "backup," which undersells it. What git actually gives you is a series of **checkpoints** — snapshots of your project at moments you chose, each one recoverable, each one comparable to any other. That matters for any project, but it matters especially once an agent is also editing your files: git is how you know exactly what changed, when, and whether you want to keep it.

0.3.2 Installing and Configuring Git

Check whether you already have it:

```
git --version
```

If that fails, install it via your system's package manager (`brew install git` on macOS, `sudo apt install git` on Debian/Ubuntu-based Linux, or the WSL equivalent). Then tell git who you are — it stamps this on every commit you make:

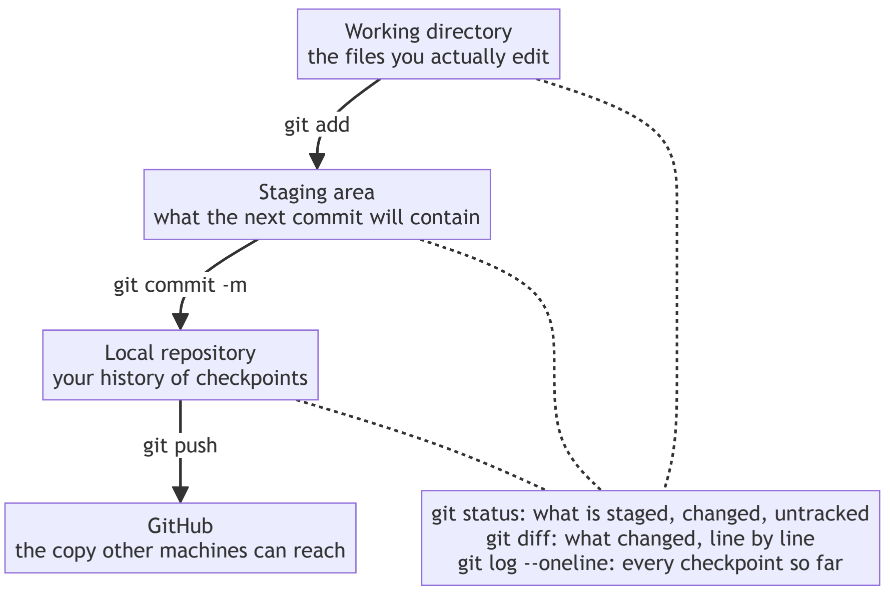
```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Remember these two values — they resurface sooner than you'd expect. Section 0.7.2 will show you exactly where.

0.3.3 The Core Loop

```
cd mortgage-calculator-book
git init                          # start tracking this directory
git status                        # what's changed since the last
commit?
git add SPEC.md                   # stage a specific file
git add .                         # stage everything that's changed
git commit -m "Initial commit"    # save a checkpoint, with a
message
```

Those commands move your work between three distinct places, and keeping the three straight is most of what makes git click:



The solid arrows move things forward; the dotted ones only look. Nothing skips a step: a file you edited isn't in a commit until it has been through staging, which is exactly why `git add` and `git commit` are two commands rather than one.

`git status` is the command you'll run constantly — more than any other in this book. It tells you what's staged, what's changed but unstaged, and what git doesn't know about yet. Run it often; there's no penalty for checking.

0.3.4 Reading a Commit Log

```
git log --oneline
```

This gives you a compact history of every checkpoint you've made — one line per commit, most recent first. As this project grows across the rest of the book, this is how you'll orient yourself: "what did I actually do in Chapter 4?"

> **Process concept: commits as checkpoints.** A commit is cheap, fast, and reversible in a way that makes it worth doing far more often than feels necessary at first. Get in the habit now, before Chapter 1 hands your repository to an agent: commit before you let anything — agent or otherwise — make a change you haven't reviewed yet. If something goes wrong, you want a recent checkpoint to fall back to, not a blank slate.

0.4 GitHub.com

0.4.1 Creating an Account

Sign up at github.com if you haven't already. You don't need to create a repository through the website yet — section 0.4.4 below sets one up directly from the terminal, which turns out to be less work once you have one more tool installed.

0.4.2 Installing and Authenticating gh

`gh` is GitHub's own command-line tool — it lets you create repositories, open pull requests, and authenticate, all without leaving the terminal. Install it following the instructions at cli.github.com (on macOS: `brew install gh`), then confirm:

```
gh --version
```

Now authenticate:

```
gh auth login
```

This starts an interactive flow: choose `GitHub.com`, then choose your preferred protocol — **SSH** is this book's recommendation, for the reason section 0.4.3 explains — then let it open a browser to confirm the login. If you chose SSH, `gh` will offer to generate a new SSH key and upload it to your account for you automatically. Say yes; that's the easiest path, and it means section 0.4.3 below is mostly just explaining what just happened rather than more work for you to do.

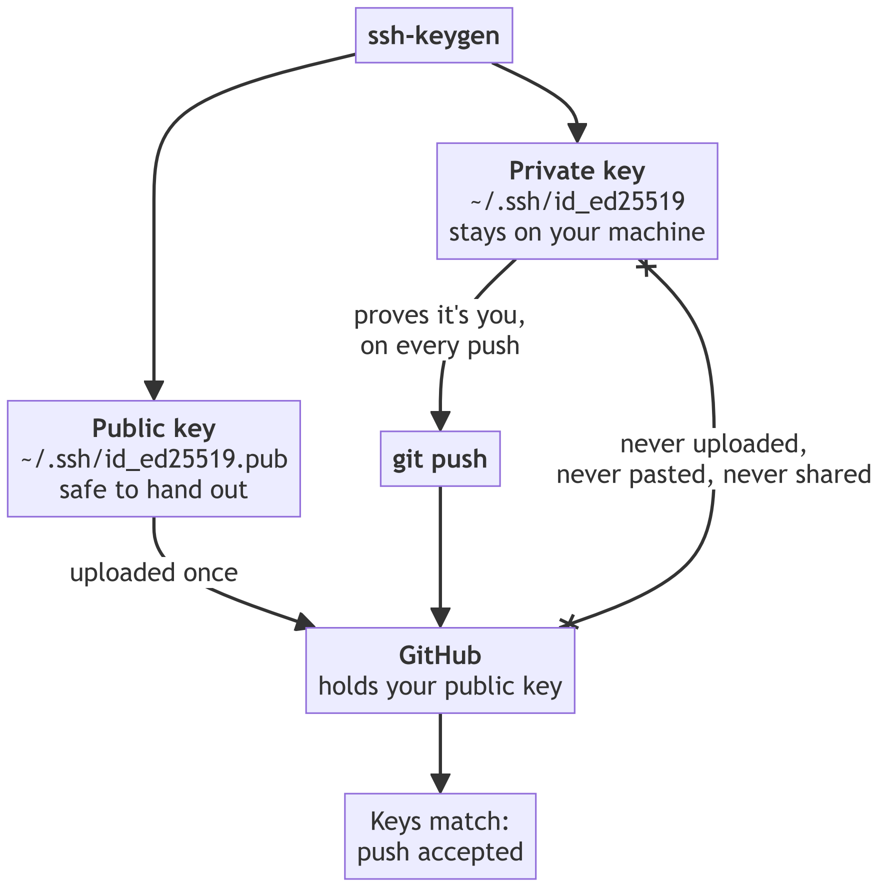
0.4.3 SSH Keys, Briefly

Every push to GitHub needs to prove it's really you. SSH keys are how: a matched pair of cryptographic keys, one **private** (stays on your machine, never shared), one **public** (uploaded to GitHub). GitHub can use the public key to confirm that whoever's pushing holds the matching private key, without you typing a password or token on every push.

If you let `gh auth login` generate a key for you in 0.4.2, this is already done — skip to the verification command below. If you'd rather set it up by hand, or want to see what `gh` did automatically, here's the manual version:

```
ssh-keygen -t ed25519 -C "you@example.com"  # accept the defaults
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
cat ~/.ssh/id_ed25519.pub                   # copy this output
```

Then, in a browser: GitHub → Settings → SSH and GPG keys → New SSH key, and paste what you copied.



One key of the pair is designed to be published and the other is designed never to leave — that asymmetry is the entire trick. GitHub never learns your private key; it only ever checks that whoever is pushing holds the one matching the public key you uploaded. The crossed-out arrow is the mistake to never make, and it's the reason `cat`-ing the wrong one of those two files into a browser is worth slowing down for.

Either way — automatic or manual — verify it worked:

```
ssh -T git@github.com
```

You're looking for `Hi <your-username>! You've successfully authenticated, but GitHub does not provide shell access.` That message, not an error, means you're set up correctly.

0.4.4 Creating and Connecting the Repository

With gh authenticated, creating a repository and connecting your local project to it is one command, run from inside mortgage-calculator-book:

```
gh repo create mortgage-calculator-book --public --source .
--remote origin
```

This creates the repository on GitHub, and wires up your local repository's origin remote to point at it — both steps 0.4.4 and part of what used to be a separate "connect local to remote" step, done together.

If you'd rather create the repository through the browser instead: go to github.com/new, name it mortgage-calculator-book, and leave it empty — no README, no .gitignore (0.7.3 creates that one properly, before it's actually needed). Then connect it by hand:

```
git remote add origin \
    git@github.com:YOUR_USERNAME/mortgage-calculator-book.git
```

Either path, finish by making sure your branch is named main and pushing:

```
git branch -M main            # only needed if not already on
main
git push -u origin main
```

After this, git push alone (no flags) will work for future commits.

0.4.5 Reading a Diff

Before going further, one concept worth naming properly, because you'll rely on it constantly starting in Chapter 1: a **diff** — short for "difference" — is a line-by-line comparison between two versions of a file. Run git diff after changing a tracked file, and git shows you exactly what changed: lines removed, lines added, and a little unchanged context around them so you can see where the change sits.

For example, editing a line in README.md might produce:

```
 # Mortgage Calculator
-A calculator.
+Calculates the fixed periodic payment for a fixed-rate mortgage.
```

The unprefixed line is unchanged context. The - line is what the file used to say. The + line is what replaced it. That's the entire vocabulary: minus means removed, plus means added, no prefix means unchanged.

This is arguably the single most load-bearing skill in this whole book. Starting in Chapter 1, every time Pi proposes a change, you'll be reading a diff like this one to see exactly what it's proposing before you accept it.

0.5 vi, Just Enough

0.5.1 Why Bother

You will, at some point in this book — or in real engineering work afterward — end up needing to edit a file with no graphical editor available. The most common version of this: connecting to a remote machine over SSH (the same secure-connection technology from 0.4.3, just used here to log into a server rather than to authenticate with GitHub) and finding yourself with only a terminal — no windows, no mouse, no familiar text editor installed, just whatever's already on that machine. vi (or its more common modern form, vim) is installed almost everywhere by default, which is exactly why twenty minutes now saves you from being stuck later.

0.5.2 Modes

vi's defining quirk is that it has **modes** — the same keys do different things depending on which mode you're in.

- **Normal mode** (where you start): keys are commands, not text.
- **Insert mode**: keys are text, typed normally.
- **Command mode**: for saving, quitting, and search-and-replace.

0.5.3 The Commands You Actually Need

```
i        enter insert mode (start typing)
Esc      leave insert mode, back to normal
:w       write (save)
:q       quit
:wq      write and quit
dd       delete the current line
/word    search for "word"
```

That's genuinely enough to survive. Open a scratch file and practice: `vi scratch.txt`, press `i`, type a sentence, press `Esc`, then `:wq`. Confirm it actually saved:

```
cat scratch.txt
```

You should see the sentence you typed printed back to you.

0.5.4 When to Reach for It

vi is not this book's daily driver — most of your editing will happen in whatever full editor you're comfortable with, or through the agent itself. Treat vi as a tool for the specific situation where nothing else is available, not as something you need to master.

0.6 Python 3.12

0.6.1 Installing

Download Python 3.12 from python.org or install it via your package manager. This book uses 3.12 specifically — later chapters rely on language features and standard-library behavior introduced by that version, so an older Python may produce confusing, version-specific errors that have nothing to do with your code.

0.6.2 Checking Your Install

```
python3 --version
```

You're looking for `Python 3.12.x`. If you have multiple versions installed and the wrong one shows up, `uv` (0.7 below) will help you pin the right one per-project, so don't spend too long fighting your system's default.

0.6.3 What's Relevant Here

If you've seen Python tutorials from a few years ago, one thing is worth explaining properly before it shows up throughout this book: **type hints**. A type hint is an optional annotation on a function's parameters and return value, stating what kind of data is expected:

```python
def greet(name: str) -> str:
    return f"Hello, {name}"
```

Here, `name: str` says the `name` parameter should be a string, and `-> str` says the function returns a string. Python doesn't strictly enforce these at runtime by default — they're documentation, for you and for Pi starting in Chapter 1, more than an ironclad guarantee — but they make code substantially easier to both read and generate correctly, which is why this book uses them in every function from Chapter 6 onward. Error messages are also noticeably more specific about *where* and *why* something went wrong than in older Python versions. Both matter for this book — you'll be reading type hints throughout, and reading error messages closely is a skill this book leans on more than most.

0.7 uv & Good Project Structure

0.7.1 What uv Is

`uv` is a fast, modern tool for managing Python projects — dependencies, virtual environments, and the Python version itself, all through one tool instead of the traditional pip-plus-venv-plus-something-else combination. This book uses `uv` throughout because it removes a category of setup friction that has nothing to do with learning software engineering.

Install it following the instructions at docs.astral.sh/uv, or via your system's package manager — on macOS, `brew install uv` works just as well. Either path works fine for this book; use whichever you're more comfortable keeping updated. Then confirm:

```
uv --version
```

0.7.2 Initializing the Project

```
uv init --package
```

The `--package` flag matters: bare `uv init` sets up a single-file `main.py` sitting at the project root, not the installable `src/` layout this book uses throughout. `--package` is what actually generates that layout, along with a `[project.scripts]` entry point — both covered in 0.7.3 just below.

This generates a `pyproject.toml` — the file that describes your project: its name, its dependencies, and how it's built. Here's what it actually contains right after running `uv init --package` (this example is from the author's own machine — yours will show your own name and email, not this one):

```toml
[project]
name = "mortgage-calculator-book"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
authors = [
    { name = "Robert Ioffe", email = "robert.ioffe@vinipuh.com" }
]
requires-python = ">=3.12"
dependencies = []

[project.scripts]
mortgage-calculator-book = "mortgage_calculator_book:main"

[build-system]
requires = ["uv_build>=0.12.5,<0.13.0"]
build-backend = "uv_build"
```

A few things worth noticing. The `authors` field wasn't typed by hand — `uv init --package` pulled it straight from the same `git config --global user.name` and `user.email` you set back in 0.3.2, which is why it was worth getting those right early. `dependencies = []` is empty for now; it starts filling up in Chapter 5. And `[project.scripts]` already contains an entry pointing at a `main` function that doesn't exist yet anywhere in your project — `uv init --package` generates this automatically as a placeholder for a command-line entry point; Chapter 8 replaces it with the CLI's real one, so don't worry about it being unfulfilled until then.

You'll also see a `.python-version` file appear alongside `pyproject.toml` — that's `uv` pinning the exact Python version for this project, the same reproducibility idea as `uv.lock` in 0.7.4 below, just for the interpreter rather than the dependencies. It's meant to be committed, not ignored.

You'll come back to `pyproject.toml` constantly; every `uv add` command in later chapters edits it for you.

0.7.3 Project Layout

This book uses a **src layout**: your actual package lives inside a `src/` directory, rather than sitting directly in the project root.

```
mortgage-calculator-book/
|-- .gitignore
|-- pyproject.toml
|-- uv.lock
|-- README.md
|-- SPEC.md
|-- src/
|   `-- mortgage_calculator_book/
|       `-- __init__.py
`-- tests/
```

The reason: a src layout forces your code to be *installed* to be imported, the same way it would be for anyone else using it — which catches a whole class of "works on my machine because I happened to be in the right directory" bugs before they happen. It's a small amount of extra structure up front that pays for itself the moment you write your first test in Chapter 5.

One more file belongs in this skeleton before the first commit: `.gitignore`. 0.4.4 deliberately left it out when the GitHub repository was created — this is where it actually gets built, and it needs to exist *before* anything below generates the files it's meant to exclude, not after:

```
vi .gitignore
```

```
# Byte-compiled / cached Python files
__pycache__/
*.pyc
*.pyo

# Test and linter caches
.pytest_cache/
.ruff_cache/

# Virtual environments
.venv/
```

Every one of these gets created automatically as this project grows — `.venv/` by `uv` itself, starting with `uv add` in 0.7.4 just below; `__pycache__/` the first time any Python file actually runs; `.pytest_cache/` and `.ruff_cache/` once pytest and Ruff exist, in Chapters 5 and 3. None of them belong in git: they're regenerable, often large, and specific to your machine, not your project. Skip this file now, and a plain `git add .` in any later chapter will happily stage all of it.

To check your own layout matches this at a glance, rather than reading through nested `ls` output by hand, either of these works:

```
tree -I '.git'          # brew install tree on macOS if you don't
have it
ls -R                   # no install needed, less readable for deep
trees
```

Once it matches, commit and push it — this is a real checkpoint, worth its own commit message:

```
git add .
git commit -m "Set up project skeleton"
git push
```

Then open your repository on github.com in a browser and confirm the files are actually there. This is worth doing deliberately, not just trusting the terminal: a push can succeed locally against the wrong remote, or against a branch you didn't mean to push, and the only way to be sure is to look.

0.7.4 Adding a Dependency

```
uv add requests
```

This updates `pyproject.toml` and creates (or updates) `uv.lock` — a file that pins the *exact* version of every dependency, direct and indirect, so that "it works on my machine" means the same thing on any machine that runs `uv sync`. You won't need any dependencies yet in this chapter; you'll see this command for real starting in Chapter 5.

0.7.5 The Skeleton You're Building Toward

By the end of this chapter, your project should match the layout above, minus anything you haven't built yet — just `.gitignore`, `pyproject.toml`, `uv.lock`, an empty `README.md` and `SPEC.md`, and an empty `mortgage_calculator_book` package. Every later chapter adds to this same skeleton; nothing gets rebuilt from scratch.

Confirm it works:

```
uv run python -c "import mortgage_calculator_book; print('ok')"
```

If that prints `ok`, your project is wired together correctly.

0.8 Checkpoint

Before moving to Chapter 1, this should all be true:

- ☐ You can open a terminal and navigate with `pwd`, `ls`, `cd`
- ☐ Git is installed and configured with your name and email
- ☐ `mortgage-calculator-book` is a git repository with at least one commit
- ☐ `gh` is installed and authenticated (`gh auth login` completed successfully)
- ☐ `ssh -T git@github.com` prints the "successfully authenticated" message
- ☐ You can explain, in your own words, what a diff shows and why the `+`/`-` prefixes matter
- ☐ You can open, edit, and save a file in vi without help, and verify the save with `cat`
- ☐ `uv run python --version` reports 3.12.x
- ☐ `uv run python -c "import mortgage_calculator_book; print('ok')"` prints `ok`
- ☐ `.gitignore` exists and excludes `__pycache__/`, `.pytest_cache/`, `.ruff_cache/`, and `.venv/`
- ☐ Your project has been committed and pushed, and you've confirmed the files are visible on github.com in a browser — not just assumed from a successful-looking terminal command

What's next: Chapter 1 hands this repository to a coding agent for the first time — installing Pi, connecting it to a local model, and making your first small, reviewed, agent-assisted change.

Chapter 1 — Meet Your Coding Agent

Contents

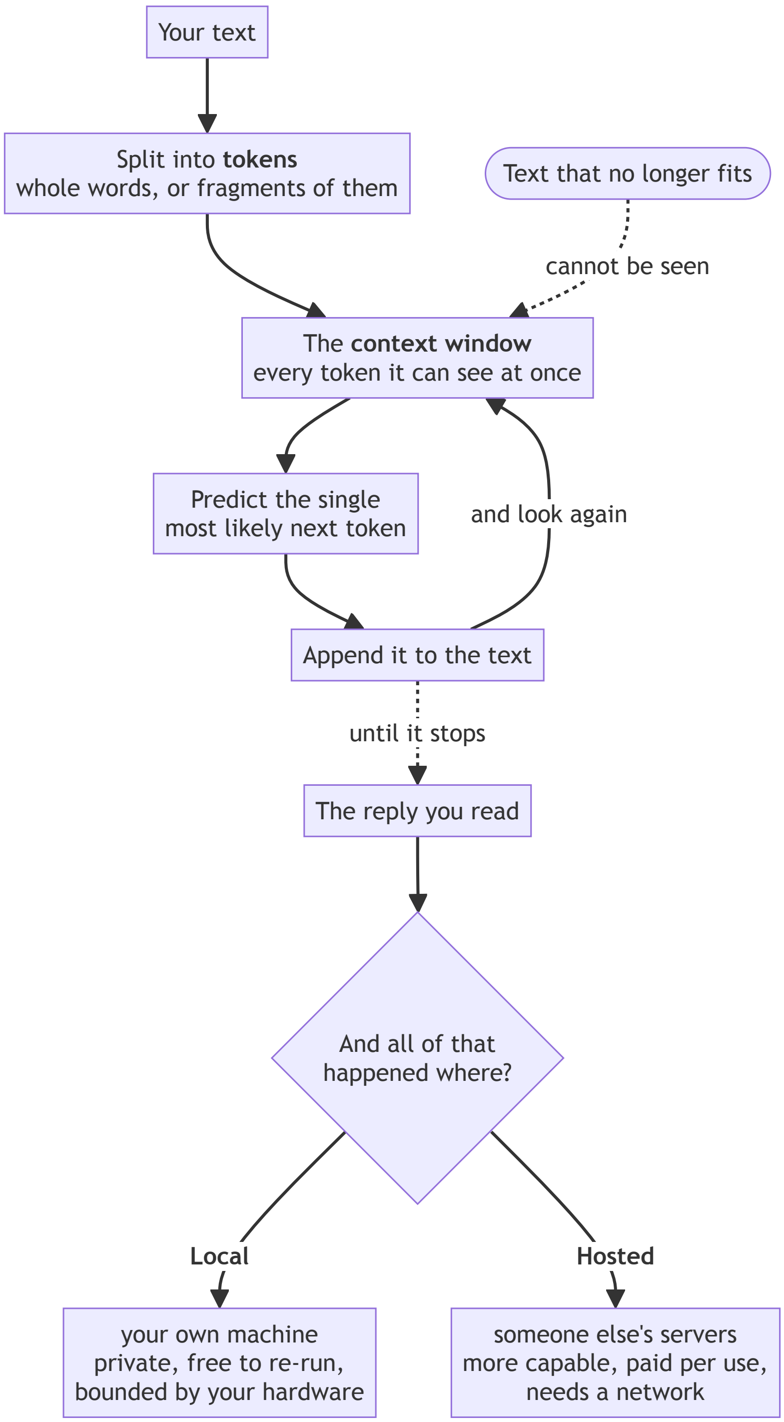
1.1 Why This Chapter Exists

Chapter 0 gave you an empty, properly structured project. This chapter gives that project a collaborator. By the end, you'll have a local language model running on your own machine, a coding agent called Pi configured to use it, and — most importantly — a first small, deliberately reviewed interaction with that agent, on the record in your git history.

Nothing gets built for the mortgage calculator in this chapter. That starts in Chapter 4. This chapter is entirely about setting up the working relationship you'll rely on for the rest of the book.

1.2 What a Model Actually Is

A language model, at a basic level, is a program trained on enormous amounts of text that has learned to predict what text comes next, given what came before. That's the whole definition this book needs. Everything below just unpacks what that prediction process implies in practice — and the diagram gives you the shape of it before the prose does:



1.2.1 Tokens

A language model doesn't read text the way you do. It breaks text into **tokens** — small chunks, sometimes whole words, sometimes fragments of words — and predicts, one token at a time, what's likely to come next. That's the entire mechanism underneath everything Pi does: read some tokens, predict the next ones, repeat. It's worth holding onto this plain description, because it's easy to start attributing more to a model than that description supports, and this book would rather you keep a clear, ungenerous picture of what's actually happening.

1.2.2 Context Windows

A model can only "see" a limited number of tokens at once — its **context window**. Everything outside that window, the model simply doesn't have access to. This matters more than raw model size for almost everything you'll do in this book: a smaller model that can see your whole file is often more useful than a larger one that can only see half of it. You'll feel the practical effects of this directly once Pi starts working across multiple files in later chapters.

1.2.3 Local vs. Hosted

A **local** model runs entirely on your own machine — private, free to run repeatedly, but limited by your hardware. A **hosted** model runs on someone else's infrastructure, reachable over the internet — typically more capable, but costing money per use and requiring a network connection. This book uses both: a local model for Pi in this chapter, and later, in Chapter 11, a choice between a second local model and a hosted one for the calculator's own intelligence engine — two different jobs, and the right answer isn't the same for both.

> **Process concept: demystifying the agent, early.** It's worth being blunt about this now, before Pi has done anything impressive: the agent you're about to install is a tool with real limits, not an oracle. Everything in this book's approach — reviewing diffs, writing tests first, keeping a human-authored spec — assumes you'll treat it that way throughout. Chapter 1 is a strange place to plant that idea, before you've even seen the tool work, but it's easier to keep a level head about a tool before it impresses you than after.

1.3 Ollama

1.3.1 Installing

Ollama is a tool for running language models locally with minimal setup. Download and install it for your platform, then confirm:

```
ollama --version
```

1.3.2 Pulling a Model

```
ollama pull qwen3:8b
```

This downloads the model's weights to your machine — a multi-gigabyte file, so this step takes a few minutes depending on your connection. (We'll pick the specific model for Pi's job in section 1.4; `qwen3:8b` here is just to confirm Ollama itself works.)

1.3.3 A First Raw Conversation

Before wiring anything to Pi, talk to the model directly:

```
ollama run qwen3:8b
```

This drops you into an interactive prompt. Ask it something simple — "what is a checksum?" — and read the response. This isn't testing Pi yet; it's building a baseline sense of what the underlying model sounds like, on its own, with no agent framework around it. Type `/bye` to exit.

1.3.4 Where the Model Lives

Model weights are stored under Ollama's own data directory (`~/.ollama` on macOS and Linux by default). If you're working with limited disk space, `ollama list` shows what you've pulled, and `ollama rm <model>` removes one. Worth knowing before you've pulled three 8-gigabyte models you don't need anymore.

1.4 Choosing a Local Model for the Coding Agent

1.4.1 Why the Choice Matters Here Specifically

Not every model is good at the same things. A model tuned for open-ended conversation isn't necessarily good at reading a diff, following an instruction precisely, or staying inside the bounds of a file it was asked to edit. The job in this chapter is narrow — coding agent support — and it's worth picking a model suited to that job rather than whatever's most popular in general.

1.4.2 This Book's Working Example: Qwen3

This book uses a Qwen3 variant as its running example for Pi's model, sized to run well on typical development hardware while still handling code-focused tasks competently:

```
ollama pull qwen3.8:27b
```

On macOS specifically, pull the MLX-optimized variant instead — built for Apple Silicon rather than a generic cross-platform target:

```
ollama pull qwen3.8:27b-mlx
```

At roughly 18GB, it's a smaller download than the generic build, and runs noticeably better on Apple Silicon's unified memory architecture. Either variant gives Pi meaningfully better judgment on multi-file changes than the smaller 8B model from 1.3.2, if your hardware supports it. If not, 1.4.3 below will help you decide where to land.

1.4.3 A Hardware Gut-Check

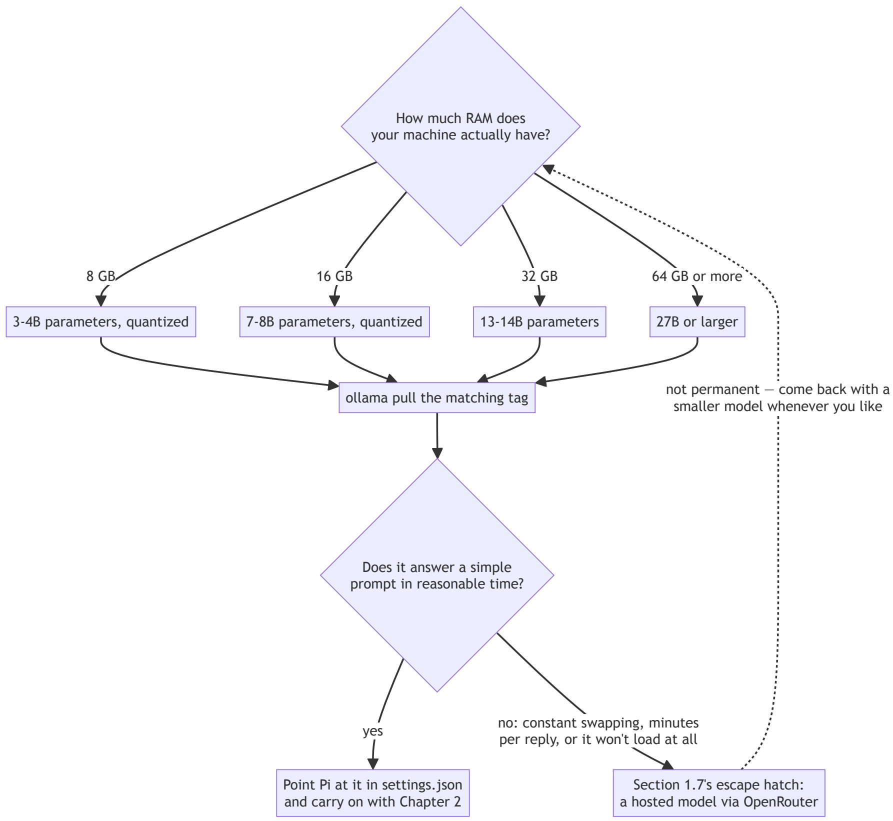
As a rough guide — not a precise spec, just enough to self-select before installing something too large:

Available RAM	Reasonable model size
8 GB	3–4B parameters, quantized
16 GB	7–8B parameters, quantized
32 GB	13–14B parameters
64 GB+	27B+ parameters

Quantization — reducing the precision of a model's internal numbers — trades a small amount of accuracy for a large reduction in memory and disk use, which is why quantized model sizes appear throughout this table rather than the full-precision numbers you might see quoted elsewhere. Ollama pulls a reasonable default quantization automatically; you don't need to choose one by hand for this book.

If your hardware doesn't comfortably fit even the smallest row here, section 1.7's escape hatch is exactly for you — don't force a model that's too large for your machine just to stay "local" on principle.

The whole decision, including the escape hatch, as one path:



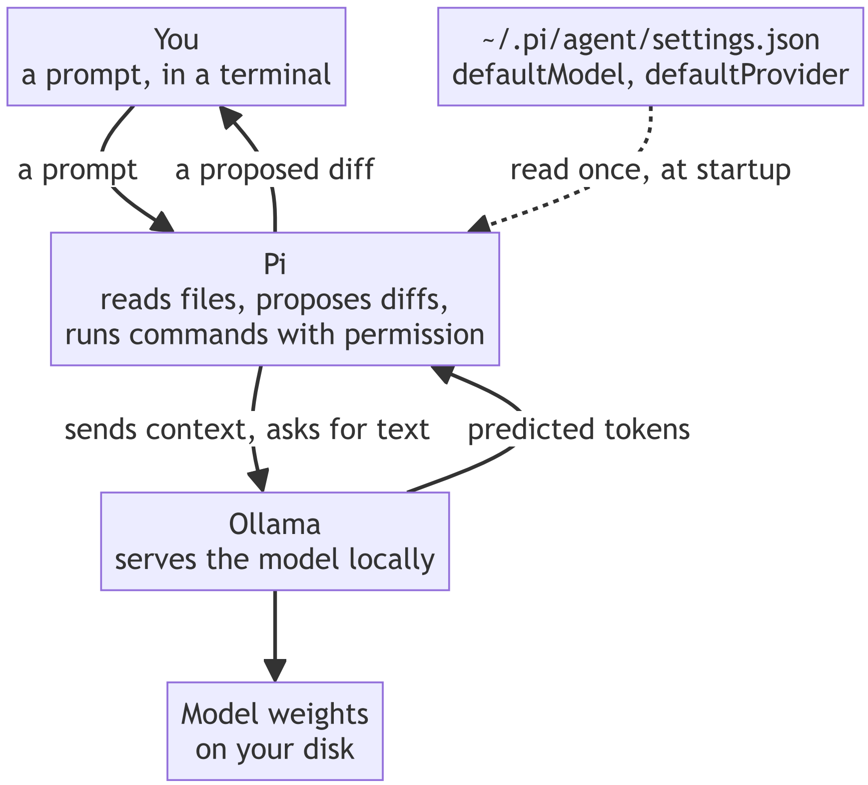
Notice that the escape hatch loops back rather than terminating. Taking it isn't a failure state or a one-way door; it's a way to keep moving today, with the local path still open tomorrow.

1.5 Installing and Configuring Pi

1.5.1 What Pi Actually Does

Pi is a coding agent: a program that can read your project's files, propose changes, and — with your permission at each step — run commands and write to disk. The model from section 1.4 is Pi's reasoning engine; Pi itself is the framework that lets that reasoning act on a real codebase instead of just producing text in a chat window.

Four separate pieces are involved, and it's worth seeing which one does what before you install any of them:



Only the rightmost box does any predicting. Everything Pi adds — finding the right files, assembling context, applying an edit, running a command — sits between you and that prediction, which is why section 1.4's choice of model and this section's choice of agent are genuinely separate decisions.

1.5.2 Installation

Follow the install instructions for your platform from Pi's documentation. Confirm it's available:

```
pi --version
```

1.5.3 Pointing Pi at Your Local Model

Pi doesn't have a `pi config set` command for this — model selection lives directly in Pi's settings file, `~/.pi/agent/settings.json`. This is a good first real use for the vi skills from 0.5: open the file and edit the values by hand.

```
vi ~/.pi/agent/settings.json
```

The fields that matter here are `defaultModel`, `defaultProvider`, and `defaultThinkingLevel`:

```
{
  "defaultModel": "qwen3.8:27b-mlx",
  "defaultProvider": "ollama",
  "defaultThinkingLevel": "medium"
}
```

Set `defaultModel` to whichever tag you pulled in 1.4.2, and `defaultProvider` to `ollama`. `defaultThinkingLevel` controls how much the model reasons before answering — `medium` is a reasonable default to start with. (Pi's settings file has a few more fields too — including a `packages` list for installed extensions — that you can safely ignore for now; this book doesn't need any of them.)

1.5.4 Skills

Pi supports **skills** — packaged instructions that give it additional domain-specific guidance beyond its defaults. There's no `pi skills` command; skills are viewable and toggled through the same interactive screen `pi config` opens (Tab switches between scopes). Adding one is a matter of copying files rather than installing a package: a **global** skill goes in `~/.pi/agent/skills/`, available to every project; a **local** skill goes in `.pi/agent/skills/` inside a specific project, available only there. You don't need any skills for this book's core path — we'll flag it explicitly in later chapters if a particular skill would help with that chapter's task.

1.5.5 Starting Pi in the Repository

There's no separate `pi init` step. Pi's available commands, for reference:

```
pi install <source> [-l]     Install extension source and add to
settings
pi remove <source> [-l]      Remove extension source from
settings
pi uninstall <source> [-l]   Alias for remove
pi update [source|self|pi]   Update pi, extensions, or model
catalogs
pi list                      List installed extensions from
settings
pi config [-l]               Open TUI to enable/disable package
resources
pi auth <command>            Print credentials or check provider
readiness
```

Pointing Pi at your project is just a matter of being in it when you start it:

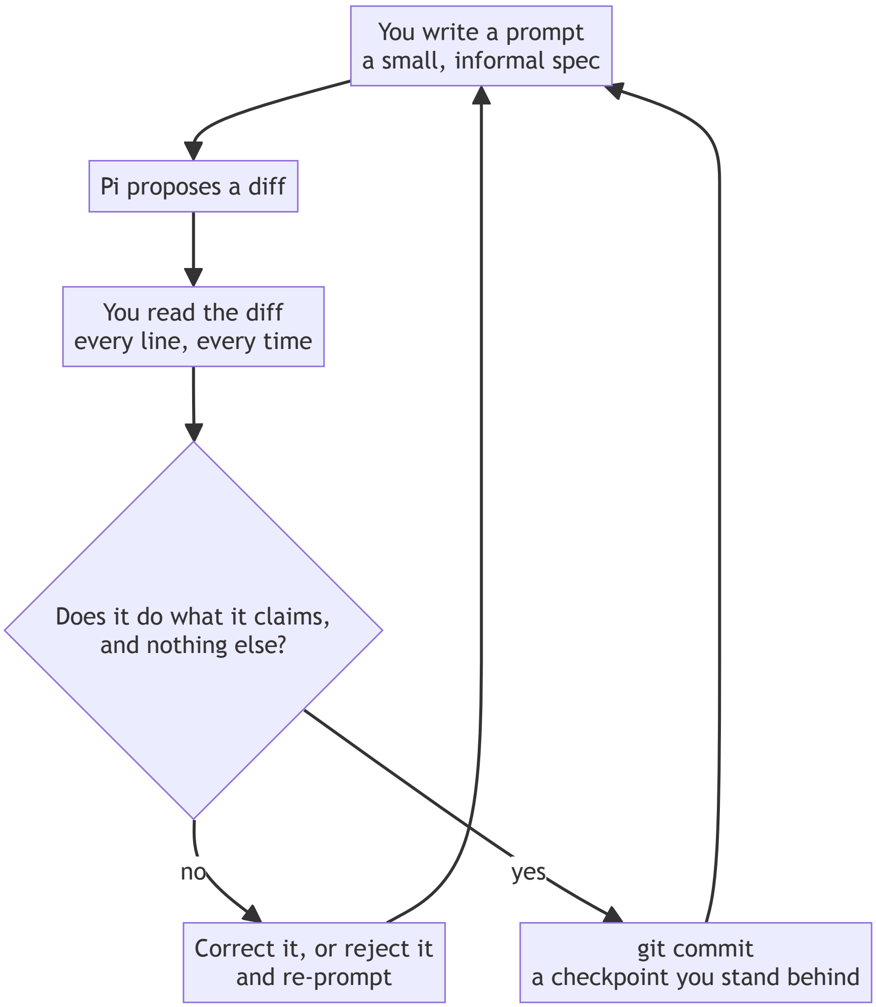
```
cd mortgage-calculator-book
pi
```

Running `pi` with no arguments starts an interactive session using the current directory as its project context — nothing more to configure.

1.6 Your First Agent-Assisted Interaction

1.6.1 A Deliberately Small Task

Every agent-assisted change in this book — from this docstring through Chapter 13's error handling — runs the same short loop:



The loop never gets a step removed as the book goes on — the diffs just get bigger. When a later chapter says "review this the same way you reviewed Chapter 1.6's," this is the shape it means.

Before trusting Pi with anything that matters, give it something safe and easy to check:

```
pi "Add a one-line docstring to the top of \
    src/mortgage_calculator_book/__init__.py describing \
    what this package is for."
```

This launches Pi in interactive mode with that instruction as your first message — Pi will think, propose a change, and then wait for you inside the session rather than exiting on its own. Once you've reviewed what it did (1.6.2 below), exit with **Ctrl+D**.

If you'd rather Pi run one instruction and exit immediately, without an interactive session, add `-p`:

```
pi -p "Add a one-line docstring to the top of \
       src/mortgage_calculator_book/__init__.py describing \
       what this package is for."
```

`-p` is short for "print": Pi runs the instruction, prints its response, and exits on its own — often more convenient for the small, self-contained tasks this book asks of it. The rest of this book shows Pi invocations in the plain form above; assume interactive mode and a Ctrl+D exit when you're done, unless a chapter says otherwise, and reach for `-p` yourself any time you'd rather skip the session entirely.

1.6.2 Reading the Diff

Pi will show you a proposed change before applying it. **Read it before accepting.** For a change this small, that takes seconds — but the habit is the point, not the specific diff. This book asks you to read every diff Pi proposes, all the way through Chapter 13, precisely because the changes get larger and the stakes get higher, and the habit needs to already be automatic by then.

```
git diff   # if the change was already applied, review it the
normal git way
```

1.6.3 What "Trust but Verify" Looks Like Here

For this task, verifying is simple: does the docstring exist, does it say something true and reasonably clear about the package? For a one-line docstring, that's a five-second check. It won't stay that simple — Chapter 6's core library and Chapter 11's model wiring will ask considerably more of your review skills — but the underlying question is always the same one you're practicing right now: *does this change do what it claims to do, and nothing else?*

> **Process concept: prompting as spec-writing.** Notice that the instruction you gave Pi in 1.6.1 was really a tiny, informal spec — a statement of what should exist and roughly what it should say. Chapter 2 takes that same idea and gives it a proper name and a proper document: SPEC.md. Everything from here to Chapter 2 is really just this one interaction, done slightly larger.

Once you've reviewed the change and you're satisfied with it, commit it:

```
git add .
git commit -m "Add package docstring (Pi-assisted)"
```

1.7 Escape Hatch: If Local Setup Isn't Working

1.7.1 Recognizing the Signs

If Ollama is taking minutes to respond to a simple prompt, if your machine is swapping memory constantly, or if the model simply refuses to load — these are signs your hardware and your chosen model size aren't a good match, not signs you're doing something wrong.

1.7.2 A Minimal Fallback

You can point Pi at a hosted model instead, so you're not stuck before this book even really starts. The same settings file from 1.5.3 handles this too — there's no `pi config set` command here either:

```
vi ~/.pi/agent/settings.json
```

```
{
  "defaultModel": "qwen/qwen3.8-27b",
  "defaultProvider": "openrouter"
}
```

This requires an OpenRouter account, an API key, and — since hosted calls cost real money per request — some actual credit sitting in the account; a fresh account with a zero balance will authenticate just fine and then fail the moment it tries to run anything. Chapter 11 covers this setup properly, including cost and usage tracking. For now, if you need this escape hatch: create an OpenRouter account, add a small amount of credit, generate a key, and set it as an environment variable:

```
export OPENROUTER_API_KEY="your-key-here"
```

1.7.3 This Isn't Permanent

Using a hosted model here doesn't lock you out of local models for the rest of the book — it just gets you moving today. Once you've read Chapter 1.4's hardware guidance more carefully, or found a smaller model that fits your machine, you can switch back at any point by editing `defaultModel` and `defaultProvider` in `~/.pi/agent/settings.json` again.

1.8 Checkpoint

Before moving to Chapter 2, this should all be true:

- ☐ Ollama is installed and a model has been pulled successfully
- ☐ You've had at least one raw conversation with the model via `ollama run`

- ☐ Pi is installed, and `~/.pi/agent/settings.json` points `defaultModel` and `defaultProvider` at either your local model or the Chapter 1.7 fallback
- ☐ Running `pi` from inside `mortgage-calculator-book` starts a session with that project as its context
- ☐ You've given Pi one small task, read its diff, exited cleanly (Ctrl+D or `-p`), and committed the result

What's next: Chapter 2 turns the informal instruction-giving you just practiced into something more deliberate — a written SPEC.md that describes what the calculator is actually supposed to do, before any real code exists.

Chapter 2 — Writing SPEC.md

Contents

2.1 Why This Chapter Exists

At the end of Chapter 1, you gave Pi a one-line instruction and reviewed a one-line change. That worked because the task was tiny enough to hold in your head. The mortgage calculator won't be tiny for long. This chapter gives you — and Pi — something more durable to work from than a chat message: a written spec, committed to the repository alongside the code it describes.

By the end of this chapter, you'll have a first draft of `SPEC.md`, committed to your repository. It won't be complete, and it won't be entirely correct — Chapter 4 exists specifically to find out where.

2.2 What a Spec Is (and Isn't)

2.2.1 Intent, Written Before Code

A spec, as this book uses the term, is a description of what a system should do, written before the system exists to do it. It's not a design document — it doesn't describe *how* the calculator will compute a payment, only *what* it means for that computation to be right. It's also not a requirements document in the heavyweight, sign-off-required sense you might associate with large organizations. Think of it as closer to a clear, written promise than a contract: a statement specific enough to hold yourself to, informal enough to write in an afternoon.

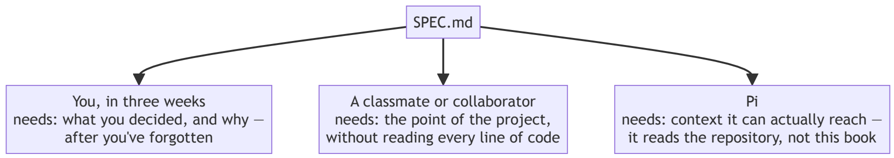
2.2.2 Living, Not Fixed

The most important thing to understand about SPEC.md before you write a word of it: it will be wrong in places, and that's expected. You'll revise it in Chapter 4 once real mortgage math enters the picture, again in Chapters 9 and 11 as new interfaces get added, and a final time in Chapter 13 once the whole system exists and you can check the spec against what actually got built. A spec that never changes usually means nobody's been checking it against reality.

2.2.3 Three Readers, One Document

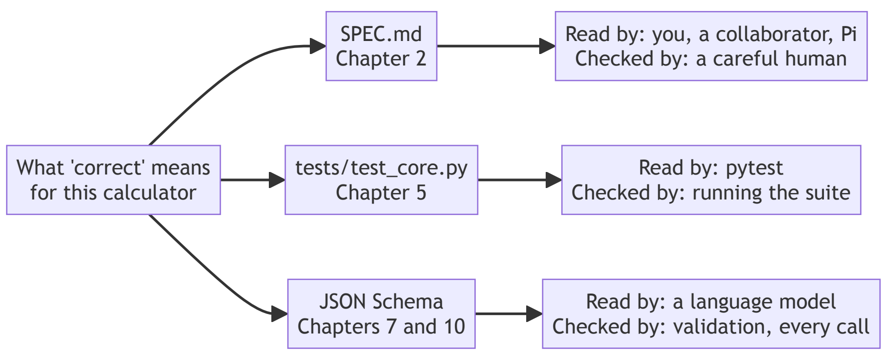
SPEC.md serves three different readers, and it's worth writing with all three in mind:

- **You, in three weeks** — when you've forgotten exactly what "valid" was supposed to mean for a loan term.
- **A classmate or collaborator** — who needs to understand the project without reading every line of code first.
- **Pi** — which will read this file as context for later tasks, the same way it read your one-line instruction in Chapter 1, just at greater length and with more precision.



The third reader is the one that changes how you write. Pi has no access to the conversation you had while deciding something, or to the chapter you're reading right now — only to files in the repository. That constraint comes back twice more, in 4.7.3 when the derivation has to be written into `docs/derivation.md`, and in 9.6.2 when the UI sketch has to become `docs/ui.md`.

> **Process concept: same contract, three altitudes.** SPEC.md is the first of three places this book will write down what "correct" means for this system. Chapter 5's tests are the second — the same intent, made executable. Chapter 7 and Chapter 10's schemas are the third — the same intent again, made machine-readable enough for a language model to call. You're not writing three different things across the book; you're writing one idea, at three different levels of formality, for three different audiences.



Three arrows out of one box, not a chain of three — that's the part worth getting right. The tests in Chapter 5 aren't derived from SPEC.md by some mechanical translation, and Chapter 10's schema isn't derived from the tests. All three are you, writing down the same decision, for a reader that needs it in a different form.

2.3 Anatomy of a Lightweight SPEC.md

A spec at this scale needs four things, no more:

2.3.1 What the Calculator Does

One paragraph, plain language. Not a feature list — a description someone with no context could read and understand the point of the project.

2.3.2 Inputs and Outputs

Named, with types. Not formulas yet — that's Chapter 4's job, once the domain math has been properly explained. For now, just: what goes in, what comes out.

2.3.3 What "Correct" Means

The criteria something must meet to count as working — even before you know the exact math. For a mortgage calculator, this might be as simple as "the computed payment, multiplied by the number of payments, should recover the total amount paid" — a property you can state and check without yet having derived the formula that produces it.

2.3.4 What's Out of Scope

Often more useful than what's in scope. Explicitly naming what this project *won't* do — variable-rate mortgages, refinancing calculations, multiple currencies — prevents scope from quietly expanding one "just add this one thing" at a time.

2.4 Writing the First Draft, With Pi

2.4.1 Prompting Pi to Help Structure the Draft

Applying the same "prompting as spec-writing" idea from Chapter 1.6, but at document scale rather than one-line scale:

```
pi "Draft a SPEC.md for a fixed-rate mortgage \
    calculator. It should describe what the tool does, \
    its inputs and outputs, what 'correct' means for it, \
    and what's explicitly out of scope. Keep it under \
    one page."
```

2.4.2 Reading Pi's Suggestions Critically

Pi will likely produce something reasonable-looking and, in at least one place, quietly wrong or presumptuous — filling a gap you didn't actually specify with a plausible-sounding assumption. This is exactly the risk this chapter exists to teach you to catch. A common example: Pi may assume monthly payments without your having said so, or describe "periodic interest rate" as if it were the same thing as the annual rate. Read every sentence and ask, for each one: *did I actually mean this, or did the agent fill in a gap I left open?*

2.4.3 Editing Down to What You Actually Mean

Take Pi's draft and cut, correct, and rewrite until every sentence is something you'd defend if asked. Below is roughly what a reasonable first draft looks like at this point in the book — deliberately imperfect, in ways Chapter 4 will catch:

```
# SPEC.md - Fixed-Rate Mortgage Calculator

## What it does
Calculates the fixed periodic payment for a fixed-rate mortgage,
given a principal amount, an interest rate, and a loan term.

## Inputs
- principal: the loan amount, in dollars
- rate: the interest rate
- term: the length of the loan, in years

## Outputs
- payment: the fixed amount paid each period

## What "correct" means
The computed payment, multiplied by the number of payments,
should equal the total amount paid over the life of the loan.

## Out of scope
- Variable-rate mortgages
- Refinancing calculations
- Multiple currencies
```

If you're reading closely, you may already see what Chapter 4 is going to have to fix: "rate" doesn't say whether it's annual or periodic, "term" doesn't say what payment frequency it implies, and "the number of payments" is used before anything defines how it's calculated. Leave it as it is for now — the gaps are the point, and finding them with real domain knowledge in hand is more useful than getting them right by luck on a first pass.

2.4.4 What You Might Actually See

The illustrative draft above is intentionally simple, so the specific gaps Chapter 4 needs to catch stay easy to spot. Real output varies — sometimes a lot — depending on which model actually wrote it, and it's worth seeing that difference once, concretely, before you run this exercise yourself.

Running the 2.4.1 prompt against **qwen3:8b** produced a noticeably different kind of draft. It added constraints nobody asked for — a $10,000 minimum loan amount, a rate capped at 0–100%, a term capped at 1–30 years — none of which trace back to anything in the prompt. That's exactly the risk 2.4.2 warned about: a plausible-sounding assumption filling a gap you never actually specified. It also introduced a "Rate Frequency" field ("Annual" or "Monthly," described as being for "effective annual rate calculations") that quietly conflates two genuinely different things — how often a rate is *quoted* versus how often payments are *made* — in a way that reads reasonably on a first pass and falls apart under the kind of scrutiny 4.3.2 asks you to apply.

Running the same prompt against **qwen3.8:27b-mlx** produced something considerably more ambitious: not just a payment figure, but a full month-by-month amortization schedule, with explicit rules for where rounding error is allowed to land (the final period only) and a battery of correctness properties — `payment == principal_paid + interest_paid` on every row, the closing balance reaching exactly zero, and so on. This is careful, genuinely well-specified engineering. It's also scope the 2.4.1 prompt never asked for and this book's project never builds: every chapter from here forward computes a single fixed payment, not a schedule. A more capable model didn't just fill gaps more sensibly here — it expanded the project's scope without being asked to, which needs exactly the same scrutiny as the smaller model's unrequested minimum-loan-amount constraint, just dressed up better.

Neither output is wrong to have produced — this is genuinely what real coding agents do, and it's part of why 2.4.2's instruction to read every sentence critically matters regardless of which model you're using. Your own first draft will very likely look like neither the illustrative version above nor either of these — which is fine. The skill this chapter is teaching isn't "get the spec Pi is supposed to produce"; it's "notice what you didn't actually ask for."

2.5 Committing the Spec

2.5.1 Where It Lives

`SPEC.md` sits at the root of your project, alongside `README.md` and `pyproject.toml` — visible immediately to anyone (or anything) opening the repository, not buried in a subdirectory.

2.5.2 Markdown, Reused

The Markdown syntax you're using here — headers, lists — is the same syntax Chapter 8 will use for the project's README. Learning it once, here, means Chapter 8 is mostly application rather than new material.

2.5.3 Committing It as Its Own Checkpoint

```
git add SPEC.md
git commit -m "Add first-draft SPEC.md"
git push
```

A spec is worth its own commit message, separate from any code — it's a real artifact of the project, not a scratch file.

2.6 What Happens When the Spec Is Wrong

This draft has gaps, and that's fine for now. Chapter 4 introduces the actual mathematics of fixed mortgages — periodic rates, payment frequency, the derivation of the payment formula itself — and one of that chapter's jobs is to come back to this exact document and fix what turns out to be ambiguous or simply missing. That revision is normal process, not a sign the first draft failed. Expecting it now, rather than being caught off guard by it in Chapter 4, is the whole reason this section exists.

2.7 Checkpoint

Before moving to Chapter 3, this should all be true:

- ☐ `SPEC.md` exists at the project root
- ☐ It describes what the calculator does, its inputs and outputs, what "correct" means, and what's out of scope
- ☐ You can point to at least one sentence in it and explain why you edited or kept what Pi originally proposed
- ☐ It's committed and pushed to GitHub

What's next: Chapter 3 makes sure whatever code gets written from here forward — yours or Pi's — is held to a consistent quality standard before it's ever committed. Chapter 4 will come back to this exact SPEC.md once real mortgage math is on the table.

Chapter 3 — Holding the Agent to a Standard

Contents

3.1 Why This Chapter Exists

You have a spec now, and a working relationship with an agent that can write code against it. Before any real code gets written, this short chapter sets up one more thing: a floor for quality that applies automatically, to every change, whether you wrote it or Pi did.

By the end of this chapter, Ruff will be installed, configured, and run against the one piece of code that already exists in your project — the docstring Pi added back in Chapter 1.6.

3.2 "Passes Tests" Isn't the Same as "Good Code"

3.2.1 Why This Matters More Once an Agent Is Involved

Code can be functionally correct and still be inconsistent, oddly formatted, or written in a style that doesn't match the rest of your project — none of which a test suite will ever catch, because tests check behavior, not style. That's true of code you write yourself, too, but it becomes a sharper problem once an agent is generating a meaningful share of your codebase: Pi has no inherent reason to match *your* conventions unless something enforces them, and "close enough" drift across dozens of agent-assisted changes adds up into a codebase that's technically correct and genuinely unpleasant to read.

3.2.2 A Plausible Example

Imagine asking Pi to add a function, and it comes back with correct logic, but inconsistent quote styles, an unused import left over from an earlier attempt, and inconsistent spacing around operators. Every test still passes. Nothing here is a bug. It's still worth catching, and it's not worth catching *by eye*, every time, forever.

> **Process concept: quality bars you set once, and enforce automatically.** The alternative to a linter isn't "no standard" — it's "a standard you have to remember to apply yourself, every single time, including on the days you're tired or rushing." This chapter trades that for a tool that never forgets to check.

3.3 What Ruff Is

3.3.1 Linting and Formatting, One Tool

Linting catches likely mistakes and style problems — unused imports, inconsistent naming, patterns known to cause bugs. **Formatting** enforces a consistent visual style — spacing, quote style, line length — regardless of who wrote the code. Ruff does both, in one fast tool.

3.3.2 Why Ruff, Specifically

Python has historically split this work across three separate tools — Black for formatting, Flake8 for linting, isort for import ordering. Ruff reimplements the useful parts of all three in a single, much faster tool, which is why this book uses it instead of the older combination: fewer things to install, fewer things to configure, fewer places for configuration to drift out of sync with each other.

3.4 Installing and Configuring Ruff

3.4.1 Installing

```
uv add --dev ruff
```

The `--dev` flag matters here: Ruff is a tool *you* use while developing, not something the calculator itself needs at runtime, so it belongs in your project's development dependencies, not its regular ones.

3.4.2 A Minimal Configuration

Add a small section to your existing `pyproject.toml`:

```toml
[tool.ruff]
line-length = 100

[tool.ruff.lint]
select = ["E", "F", "I"]
```

This is deliberately narrow: `E` and `F` cover common style and correctness issues, `I` handles import ordering. Ruff supports many more rule categories than this, but a first-year project doesn't need all of them enabled on day one — start narrow, and widen later if you find yourself wanting more.

3.4.3 Running It

`uv add --dev ruff` installs Ruff into your project's virtual environment, `.venv/` — not globally on your system. If you try `ruff check .` right after installing it and get a `command not found` error, that's why: your shell doesn't yet know to look inside `.venv/`. Activate the environment:

```
source .venv/bin/activate
```

(On native Windows outside WSL, the equivalent is `.venv\Scripts\activate`.)

Confirm it worked:

```
which ruff
```

You're looking for a path inside your project's `.venv/` — something like `.../mortgage-calculator-book/.venv/bin/ruff` — not a Ruff installed somewhere else on your system. This matters: if you happen to have a different Ruff on your machine already, `which ruff` is how you'd catch yourself accidentally running the wrong one.

Once activated, this same terminal session can run `ruff`, `pytest`, and `python` bare, without prefixing every command with `uv run` — which is why later chapters show commands like `pytest -v` directly rather than `uv run pytest -v`. `uv run ruff check .` is the alternative that works either way, activated or not, since it always runs inside the project's environment regardless of your shell's current state — reach for it instead if you'd rather not activate manually, or you're running a one-off command in a fresh terminal that isn't activated.

With Ruff actually runnable:

```
ruff check .      # lint
ruff format .     # format
```

`ruff check` reports problems; `ruff format` rewrites files to match the configured style. Run both — they check different things. If `ruff check` reports anything, try `ruff check --fix .` first — it automatically fixes whatever it's confident about (unused imports, several formatting-adjacent issues) and leaves only what genuinely needs a human decision. (One exception worth knowing about: Ruff treats `__init__.py` specially and won't auto-remove an unused import there, even with `--fix`, since an unused-looking import in that particular file is often an intentional re-export rather than a mistake — that one you'd remove by hand, if you ever actually want it gone.) Not everything is auto-fixable; read and fix whatever's left by hand, the same as any other error message.

Installing Ruff changed `pyproject.toml` and `uv.lock` — a real change, worth its own commit:

```
git add .
git commit -m "Add Ruff as a dev dependency"
git push
```

3.5 Running Ruff Against Agent Output

3.5.1 Pointing It at Chapter 1's Change

```
ruff check src/mortgage_calculator_book/__init__.py
```

For a one-line docstring, this will likely come back clean — which is itself a useful first result: it tells you Ruff is wired up correctly, even though there was nothing to fix yet.

3.5.2 Reading and Fixing What It Flags

When Ruff does flag something (and it will, once real code exists from Chapter 6 onward), its output names the specific rule and line: treat each one as a small checklist item, not an annoyance to silence. If a rule genuinely doesn't fit your project, that's a configuration decision to make deliberately in `pyproject.toml` — not something to work around case by case.

3.5.3 Asking Pi to Fix Ruff's Complaints

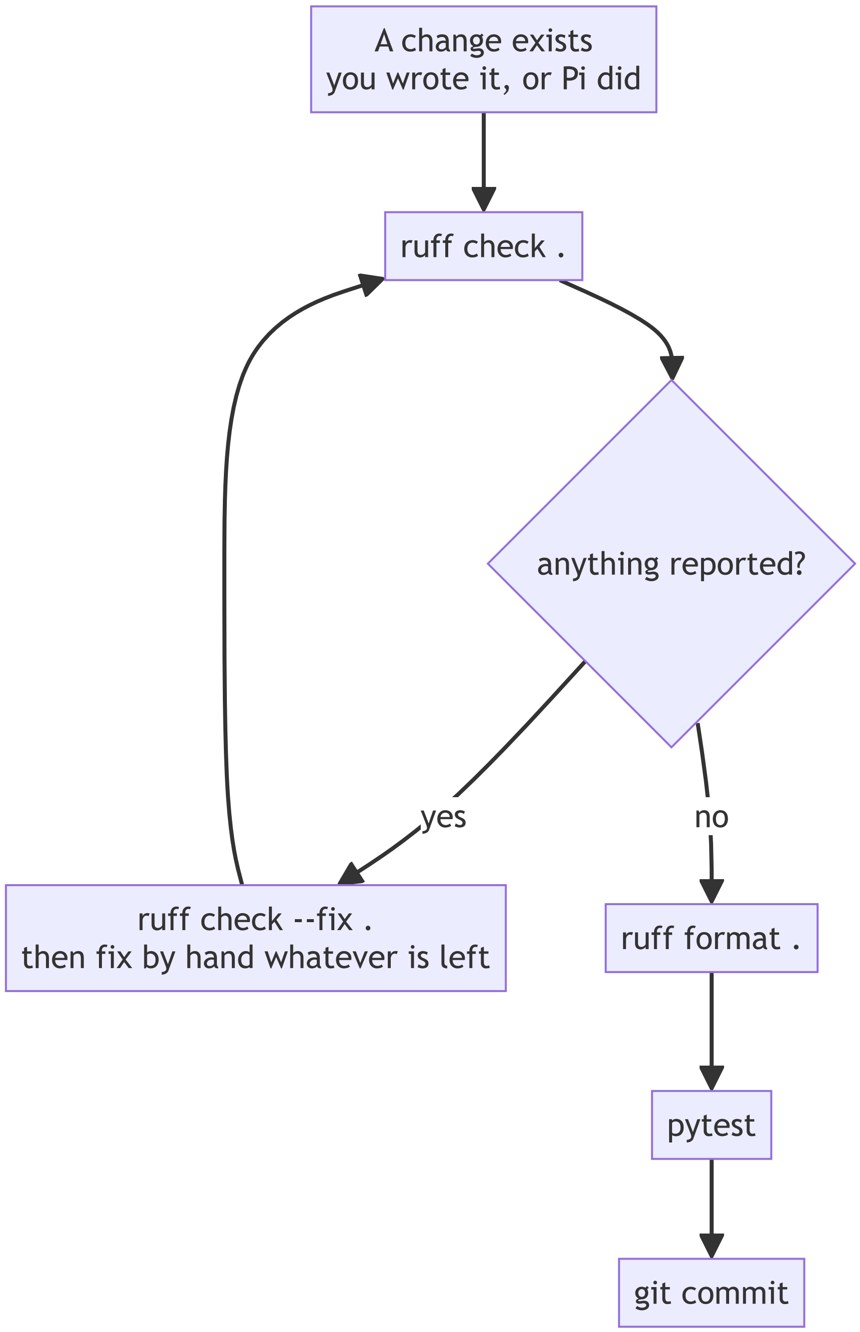
```
ruff check . || pi "Fix the issues reported by ruff check in this project."
```

Review that diff exactly the way you reviewed Chapter 1.6's — Pi fixing a lint error is still a proposed change, and still worth reading before accepting.

3.6 Making It a Habit, Not a One-Off

3.6.1 Before Every Commit

From this point forward, run `ruff check .` and `ruff format .` before every commit, for the rest of the book. This isn't automated yet — Appendix A.5 covers pre-commit hooks, which would do this for you — but doing it by hand for now is deliberate: understanding *why* you run it matters more, this early, than automating it away before you've felt the reason for it.



Every chapter from 6 onward ends by running this loop and says so in a single line. This is the loop those lines mean. The `--fix` arrow going back rather than forward is the part worth noticing: fixing lint findings can introduce new ones, so the check runs again rather than being assumed clean.

3.6.2 Looking Ahead

Chapter 13's Hardening chapter revisits standards again, but at the level of the whole system — error handling, logging, a final consistency check. This chapter is the per-commit version of that same instinct: catch problems small and early, rather than all at once at the end.

3.7 Checkpoint

Before moving to Chapter 4, this should all be true:

- ☐ Ruff is installed as a dev dependency
- ☐ `pyproject.toml` has a minimal `[tool.ruff]` configuration
- ☐ `ruff check .` and `ruff format .` both run without errors against the current project
- ☐ You've run Ruff at least once against Pi-generated content, even if there was nothing to fix

What's next: Chapter 4 leaves tooling aside for a chapter and gets into the actual mathematics of fixed mortgages — the domain content SPEC.md will need once Chapter 4 comes back to revise it.

Chapter 4 — The Domain: Fixed Mortgages

Contents

4.1 Why This Chapter Exists

No code gets written in this chapter. Instead, this chapter gives you the thing every later chapter's code will be checked against: a correct, hand-verified understanding of how a fixed-rate mortgage payment is actually calculated, and one specific worked example with exact numbers.

By the end, you'll have a revised `SPEC.md` — fixing the gaps Chapter 2 left open on purpose — and a worked example that Chapter 5's first test, Chapter 6's core library, and Chapter 12's evaluation set will all check themselves against.

4.2 Fixed Mortgage Primer

4.2.1 What a Mortgage Is

A mortgage is a loan used to buy property, where the property itself secures the loan — if payments stop, the lender can take the property back. Setting aside everything else about how mortgages work in practice, the part this book cares about is narrower and purely mathematical: given a loan amount, an interest rate, and a length of time to repay it, what should each periodic payment be?

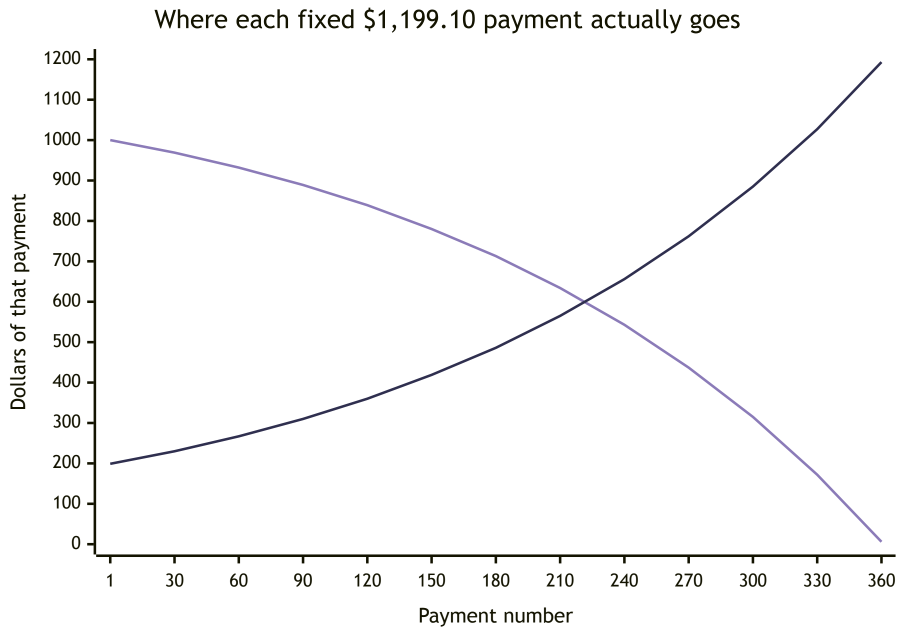
4.2.2 Why "Fixed"

A **fixed-rate** mortgage has an interest rate that stays the same for the entire loan term, which means the payment amount stays the same too, from the first payment to the last. A **variable-rate** mortgage's rate can change over time, which means the payment can change too — a meaningfully different, more complex calculation this book deliberately leaves out of scope (as SPEC.md already says).

4.2.3 The Shape of a Payment

Here's the part that surprises a lot of first-time borrowers: even though the payment amount is fixed, what that payment is *made of* changes every period. Early payments are mostly interest, with a small amount going toward the actual principal. Late payments are the reverse — mostly principal, a small amount of interest. Why: interest is charged on whatever principal is still outstanding, and outstanding principal is highest at the start and lowest at the end. We won't build a full amortization schedule until it's needed, but understanding this shape now will make the formula in section 4.4 feel like it's describing something real, not just producing a number.

That shape, for the exact loan section 4.5 works out by hand:

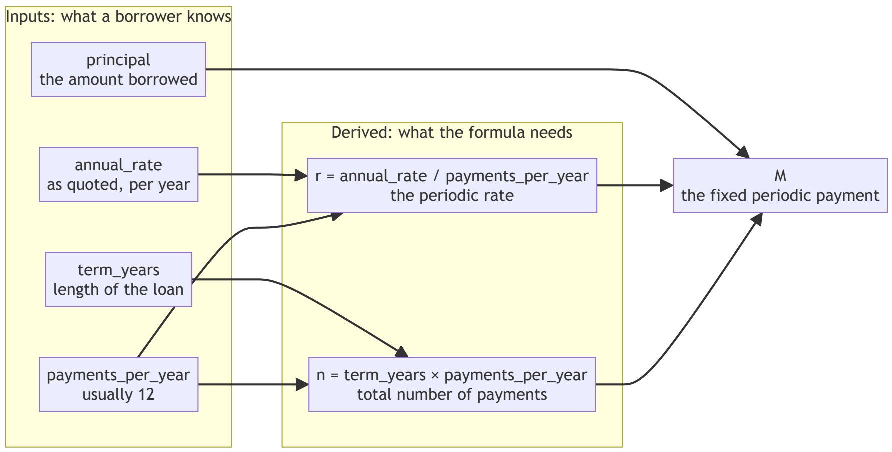


The pale line falling from left to right is interest; the dark line rising to meet it is principal. At every point along the x-axis the two add to the same $1,199.10 — the payment never changes, only its composition does. The first payment is $1,000 interest and $199 principal; the last is $6 and $1,193. They cross at payment 223, a little past year 18 of 30, which is the concrete version of "early payments are mostly interest": on a 30-year loan, more than half of what you pay goes to interest for the first eighteen years.

4.3 The Four Core Quantities

Every payment calculation in this book comes down to four numbers.

Two of them get used exactly as given; the other two get combined into a pair of derived quantities first, and it's that second pair the formula in 4.4 actually consumes:



Notice `payments_per_year` feeding both derived quantities. That single fact is why Chapter 2's spec was wrong in two places at once rather than one: leaving payment frequency unstated didn't just make the rate ambiguous, it made the payment count ambiguous too. This same shape shows up three more times in this book — as SPEC.md's "Derived quantities" section in 4.7.4, as three functions in `core.py` in Chapter 6, and as `MortgageInput`'s four fields in Chapter 7.

4.3.1 Principal

The amount actually borrowed. Everything else in the calculation exists to answer one question about this number: how do you pay it back, with interest, in equal installments?

4.3.2 Periodic Interest Rate

This is the number Chapter 2's draft spec quietly got wrong: "rate" is not the same as "periodic rate." Mortgage rates are quoted **annually** — a "6% mortgage" means 6% per year — but payments happen more often than once a year, almost always monthly. The rate that actually applies to each payment period is the annual rate divided by the number of periods per year:

$$r = \frac{\text{annual rate}}{\text{payments per year}}$$

For a 6% annual rate paid monthly: $r = 0.06/12 = 0.005$, or 0.5% per month. This conversion — annual to periodic — is the single most common place a first attempt at this calculation goes wrong, and it's exactly the gap Chapter 2's spec left open.

4.3.3 Total Number of Payments

Term length and payment frequency combine to give the total number of payments:

$$n = \text{term in years} \times \text{payments per year}$$

A 30-year loan paid monthly has $n = 30 \times 12 = 360$ payments. The same 30-year loan paid **biweekly** (every two weeks, 26 times a year) would have $n = 30 \times 26 = 780$ payments instead — a different total, and, as it turns out, a different total amount of interest paid over the life of the loan, purely from paying more frequently in smaller amounts. We'll come back to that as an aside once the main formula is in hand.

4.3.4 Fixed Periodic Payment

The quantity everything above exists to compute: the single, unchanging amount paid every period. This is what section 4.4 derives.

4.4 The Gory Details: Deriving the Formula

4.4.1 The Formula

$$M = P \cdot \frac{r(1+r)^n}{(1+r)^n - 1}$$

where M is the fixed periodic payment, P is the principal, r is the periodic rate from 4.3.2, and n is the total number of payments from 4.3.3. It looks dense on first read. The derivation below builds it up piece by piece, so it stops looking like something handed down from nowhere.

4.4.2 Where It Comes From

Think about the loan from two directions at once, both measured at the *end* of the loan — payment n.

Direction one: what the lender is owed. If nothing were ever paid, the principal would simply grow with interest every period, compounding: after n periods, the lender would be owed $P(1 + r)^n$.

Direction two: what's been paid. Each payment M, made at the end of its period, also "grows" from the moment it's paid until the end of the loan — because in effect, that payment could have been earning interest at rate r for however many periods remain after it. The very last payment doesn't grow at all (it's already at the end). The second-to-last payment grows for one period. And so on, back to the first payment, which grows for $n-1$ periods. Summed up, the total grown value of every payment is:

$$M(1+r)^{n-1} + M(1+r)^{n-2} + \cdots + M(1+r)^1 + M(1+r)^0$$

This is a **geometric series** — the same kind of sum you may have seen in an algebra course, just with an interest rate standing in for the usual ratio. It has a well-known closed form:

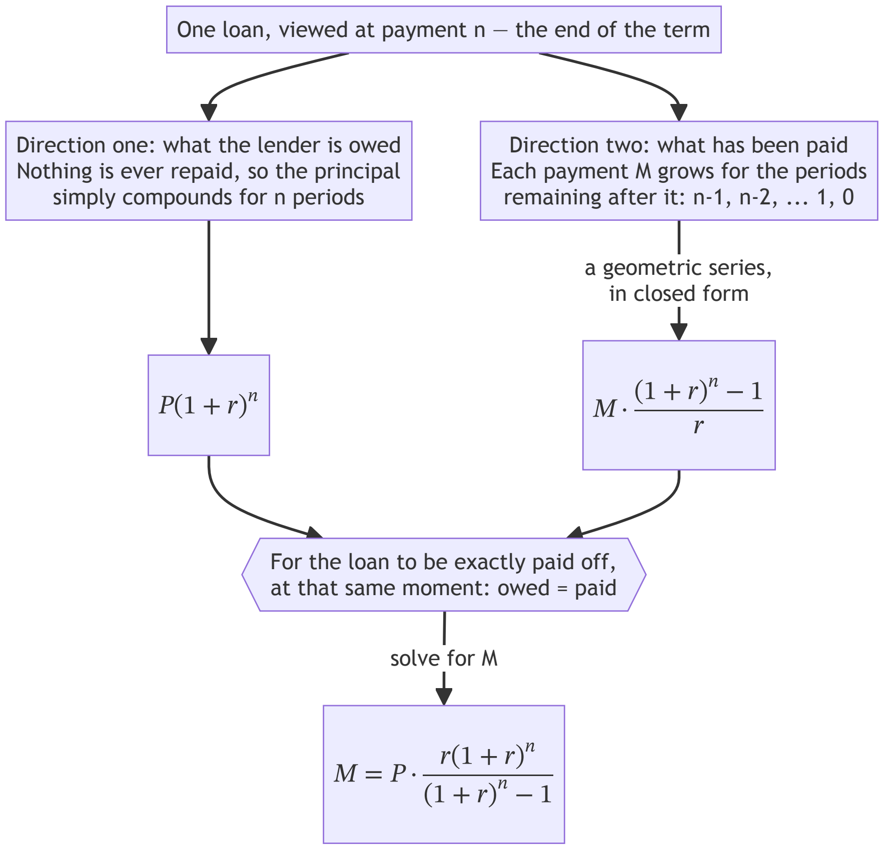
$$M \cdot \frac{(1+r)^n - 1}{r}$$

Setting the two directions equal. For the loan to be exactly paid off at payment n — no more, no less — what the lender is owed has to exactly equal what's been paid, both measured at that same end point:

$$P(1+r)^n = M \cdot \frac{(1+r)^n - 1}{r}$$

Solving for M gives the formula from 4.4.1. Nothing about it is arbitrary — it's just "what's owed equals what's paid," stated at a single point in time to make the comparison fair, then rearranged.

The whole derivation, on one page:



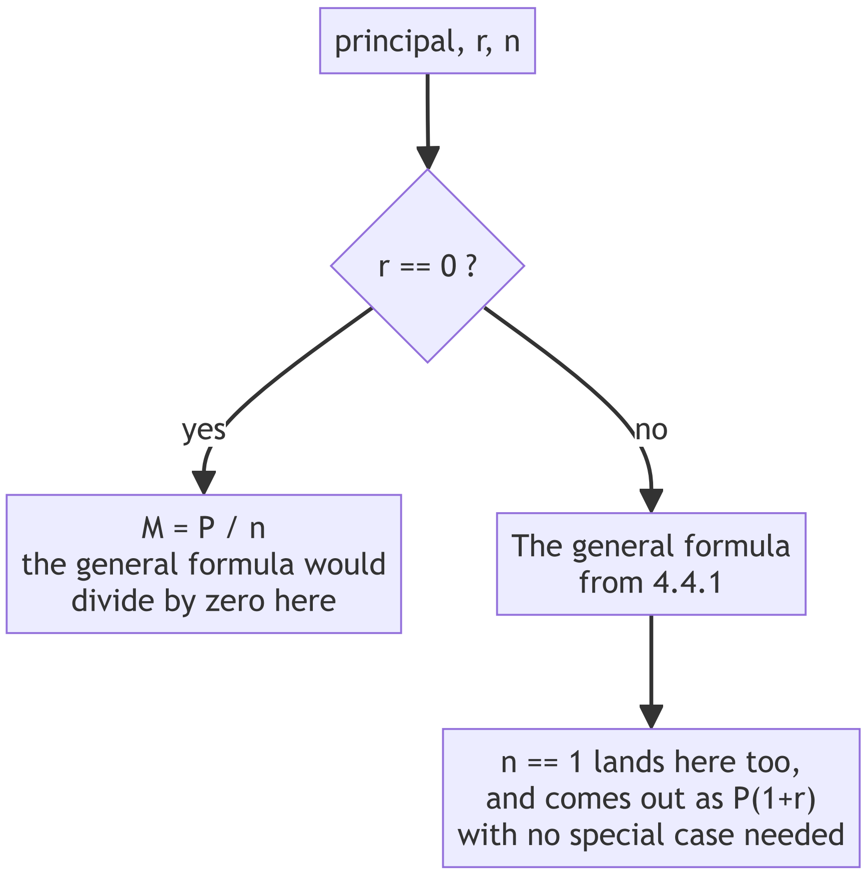
The reason both branches have to be measured at the same moment is the part most worth holding onto. Money at payment 1 and money at payment 360 aren't comparable amounts; carrying both sides forward to the same end point is what makes setting them equal a fair comparison rather than a sleight of hand.

4.4.3 Two Edge Cases Worth Naming Now

Zero interest. The formula above divides by r — which breaks down completely when $r = 0$. But the underlying question still has an obvious answer: with no interest at all, each payment should simply be the principal divided evenly across all payments, $M = P/n$. Chapter 6's implementation will need an explicit branch for this case; the general formula won't handle it on its own.

A single payment. When $n = 1$, there's no series to sum — just one payment that has to cover the principal plus one period's interest: $M = P(1 + r)$. This is worth checking against the general formula directly (it works out to the same thing when you substitute $n = 1$), which makes it a useful sanity check on the formula itself, not just a case to handle in code.

Only one of the two actually needs a branch, which is the distinction worth carrying into Chapter 6:



Zero interest needs its own branch because the formula genuinely breaks there — division by `r`. A single payment does not, because the general formula already gives the right answer at `n = 1`. Adding a branch for the second case would be a real mistake, not a harmless precaution: if `n = 1` ever needs special-casing, something upstream is wrong and the branch would hide it. Chapter 6.6.2 makes exactly this point again, standing over the code. Both of these become explicit tests in Chapter 5, and explicit branches in Chapter 6's implementation.

4.5 A Worked Example, By Hand

4.5.1 The Numbers

This is the example the rest of the book returns to repeatedly — commit these numbers to memory or bookmark this page, because Chapters 5, 6, 8, 9, 10, 11, and 12 all check themselves against it.

Quantity	Value
Principal (P)	\$200,000
Annual interest rate	6%
Payment frequency	Monthly (12/year)
Periodic rate (r)	0.005
Term	30 years
Total payments (n)	360

4.5.2 Computing the Payment

$$M = 200{,}000 \cdot \frac{0.005 \times (1.005)^{360}}{(1.005)^{360} - 1}$$

$(1.005)^{360} \approx 6.022575$. Substituting:

$$M \approx 200{,}000 \cdot \frac{0.005 \times 6.022575}{6.022575 - 1} \approx 200{,}000 \times 0.0059955 \approx \$1{,}199.10$$

Fixed periodic payment: \$1,199.10 (more precisely, \$1,199.1010503... before rounding to the cent).

From here, two more figures worth having on hand: total paid over the life of the loan is the *unrounded* payment times 360 — $1{,}199.1010503\ldots \times 360 \approx$ \$431,676.38 (the rounded \$1,199.10 times 360 instead gives \$431,676.00 — a few cents off, once 360 payments each a fraction of a cent smaller add up) — and total interest paid is that total minus the original principal: \$231,676.38 — more than the principal itself, which is a genuinely useful thing for a first-year reader to sit with for a moment before moving on.

4.5.3 This Is the Answer Key

Flag this explicitly: when Chapter 5 writes its first test, and Chapter 6 writes its first real implementation, "does it reproduce \$1,199.10 for this exact scenario" is the bar. If your code disagrees with this number by more than a rounding error, the code is wrong, not the example — these numbers were computed independently, ahead of time, specifically so you have something outside your own implementation to check it against.

4.6 Optional: Using Pi as a Study Aid

If you'd like a second check on the arithmetic above:

```
pi "Verify this mortgage payment calculation by hand: \
    principal \$200,000, annual rate 6%, 30-year term, \
    monthly payments. Show your work."
```

Read what comes back carefully rather than accepting it at face value. Models are not immune to arithmetic slips, and a subtly wrong intermediate step — the periodic rate conversion, in particular, is a common place for this to happen — can produce a final answer that looks plausible without being right. If Pi's answer disagrees with 4.5.2's, don't assume either one is automatically correct; redo the arithmetic yourself and find out which one made the mistake. That's not a wasted exercise — it's the same verification skill this book has been building since Chapter 1.6, applied to math instead of code.

4.7 Revising SPEC.md

4.7.1 Going Back to Chapter 2's Draft

Open the `SPEC.md` you committed in Chapter 2.5.3. With sections 4.3 and 4.4 now in hand, its gaps should be obvious in a way they weren't before.

4.7.2 What Was Actually Wrong

Specifically:

- **"rate"** never said whether it meant the annual rate or the periodic rate — and 4.3.2 showed those are different numbers that are easy to conflate.
- **"term"** never said what payment frequency it implied, even though 4.3.3 showed frequency changes the total number of payments (and, as the bi-weekly aside noted, the total interest paid).
- **"the number of payments"** was used in the "what correct means" section without ever being defined.

4.7.3 Writing the Derivation Down

There's a second problem here, subtler than the three gaps above: SPEC.md's revision is about to say "see [somewhere] for the full derivation" — but "somewhere" can't just mean this chapter. This book's prose isn't part of the git repository, and it never will be. Pi reads your project's files when it works; it doesn't read the book you're reading right now. "See Chapter 4.4" is perfectly good advice for you — it's a dead end for Pi, and not much better for a future maintainer who opens this repository years from now without this book on the shelf next to them.

The fix is to write the derivation down *in the project*, not just in your head or in a book external to it. Create `docs/derivation.md`:

```
mkdir docs
vi docs/derivation.md
```

```
# Fixed-Rate Mortgage Payment: Derivation

## The formula

    M = P * [r(1+r)^n] / [(1+r)^n - 1]

M is the fixed periodic payment, P is the principal, r is the
periodic rate (annual_rate / payments_per_year), and n is the
total number of payments (term_years * payments_per_year).

## Where it comes from

At the end of the loan, what the lender is owed if nothing were
ever paid -- P(1+r)^n -- must equal what's actually been paid,
each payment M grown at rate r for however many periods remain
after it: P(1+r)^n = M * [(1+r)^n - 1] / r. Solving for M gives
the formula above.

## Edge cases

- Zero interest (r == 0): the formula divides by zero; payment is
  simply P / n.
- A single payment (n == 1): reduces to M = P(1+r), which the
  general formula also produces correctly at n=1.

## Answer key

principal=200000, annual_rate=0.06, term_years=30,
payments_per_year=12 -> payment = $1,199.10. Used throughout
tests/test_core.py as the reference case every implementation is
checked against.
```

Commit it on its own:

```
git add docs/derivation.md
git commit -m "Add docs/derivation.md: the amortization formula"
git push
```

Now SPEC.md's "see … for the full derivation" can point somewhere Pi — and anyone else working in this repository — can actually read.

4.7.4 The Revision

```
# SPEC.md - Fixed-Rate Mortgage Calculator

## What it does
Calculates the fixed periodic payment for a fixed-rate mortgage,
given a principal amount, an annual interest rate, a loan term in
years, and a payment frequency.

## Inputs
- principal: the loan amount, in dollars
- annual_rate: the annual interest rate, as a decimal (e.g. 0.06
for 6%)
- term_years: the length of the loan, in years
- payments_per_year: number of payments per year (default: 12,
monthly)

## Derived quantities
- periodic_rate = annual_rate / payments_per_year
- n_payments = term_years * payments_per_year

## Outputs
- payment: the fixed amount paid each period

## What "correct" means
The computed payment, multiplied by n_payments, should equal the
total amount paid over the life of the loan. For principal =
$200,000, annual_rate = 0.06, term_years = 30, payments_per_year
=
12: payment should equal $1,199.10 (see docs/derivation.md for
the
full derivation).

## Out of scope
- Variable-rate mortgages
- Refinancing calculations
- Multiple currencies
```

Commit the revision on its own:

```
git add SPEC.md
git commit -m "Revise SPEC.md: define periodic rate, \
    payment frequency, and n_payments"
git push
```

> **Process concept: the spec was wrong, not the code.** There's no code yet — nothing here was a bug in the usual sense. But the instinct this section is building is the one that matters most

for the rest of this book: when something doesn't match reality, check whether the spec described reality correctly before assuming the implementation is at fault. Chapter 13 closes the book with exactly this same check, run one more time against the finished system.

4.8 Checkpoint

Before moving to Chapter 5, this should all be true:

- ☐ You can explain, in your own words, why the annual rate and the periodic rate are different numbers
- ☐ You can derive the payment formula's shape (even loosely) from "what's owed equals what's paid"
- ☐ You have the primary worked example's numbers ($200,000 / 6% / 30yr monthly → $1,199.10) recorded somewhere you'll return to
- ☐ `docs/derivation.md` exists, committed, and actually contains the formula — not just a reference to this chapter
- ☐ `SPEC.md` has been revised to define periodic rate, payment frequency, and n_payments, and points to `docs/derivation.md` rather than this book for the full derivation, and the revision is committed

What's next: Chapter 5 turns this worked example into an executable test — the first one this project has had, and the first thing Chapter 6's implementation will need to satisfy.

Chapter 5 — Tests & Test-Driven Development

Contents

5.1 Why This Chapter Exists

Chapter 4 gave you an answer key on paper: a formula, a derivation, and one specific scenario with an exact expected answer. This chapter turns that paper answer key into something a computer can check automatically, every time, without you re-doing the arithmetic by hand.

By the end of this chapter, pytest will be installed, your test suite will have a real structure, and the Chapter 4.5 worked example will exist as an actual test function — one that currently **fails**, because there's no core library yet to run it against. That failure is the goal, not a mistake. Chapter 6 exists to turn it green.

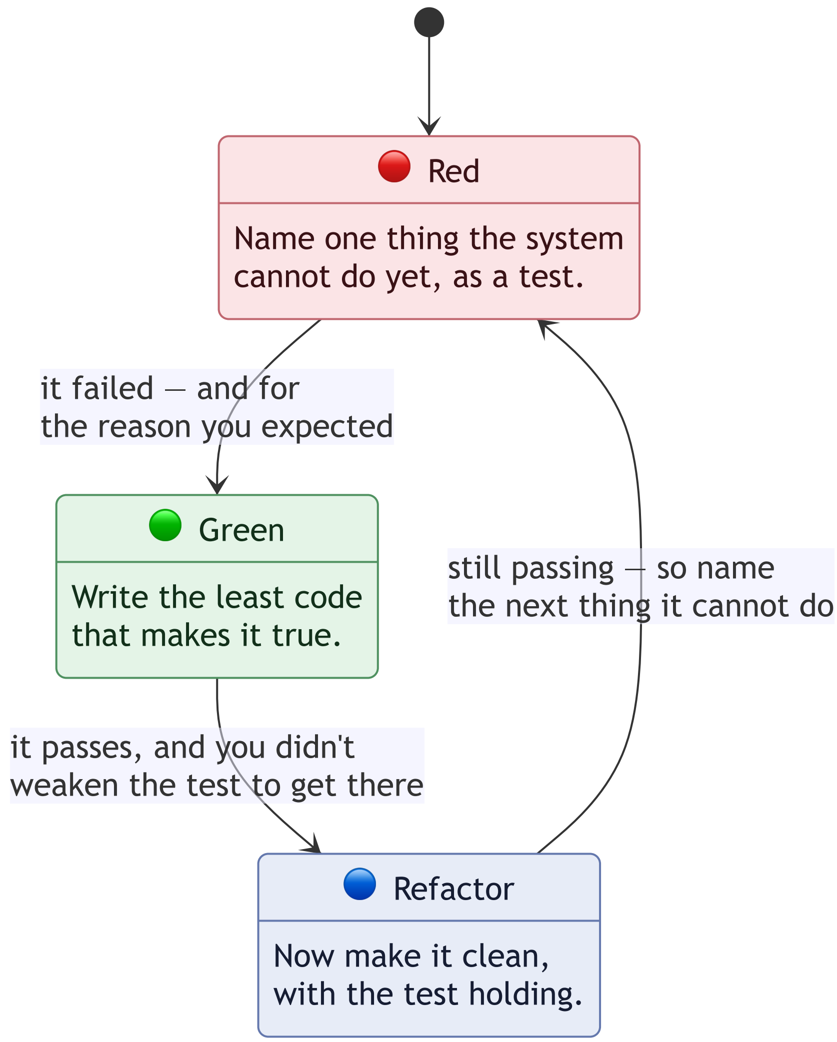
5.2 What TDD Actually Is

5.2.1 Red, Green, Refactor

Test-driven development follows a three-step cycle, repeated over and over:

1. **Red** — write a test for something that doesn't exist yet. Run it. Watch it fail.
2. **Green** — write the smallest amount of code that makes the test pass.
3. **Refactor** — clean up what you just wrote, now that you have a passing test protecting you from breaking it by accident.

Then repeat, for the next small piece of behavior.



Read the transitions, not just the states: each one names what has to be *true* before you're allowed to move on. "It failed — and for the reason you expected" is the one people skip, and it's the one that catches a test which never actually ran. Both kinds of red look similar in the terminal and mean very different things — 5.5.2's `ModuleNotFoundError` is the correct failure, saying "the thing you're describing doesn't exist yet," while 5.6.4's collection error is a broken *test*, failing before it checks anything at all. Only reading the failure tells them apart.

5.2.2 Why Write the Test First

This feels backwards the first time you do it — how can you test something that doesn't exist? But writing the test first forces you to answer a question you'd otherwise skip past: *what, exactly, does "working" mean here, before I've built anything and started rationalizing whatever I happened to build?* A test written after the code tends to describe what the code does. A test written before the code describes what the code is *supposed* to do — a small but real difference, and one that matters more, not less, once an agent might be the one writing the implementation.

5.2.3 A Tiny Example First

Before touching mortgage math, try the cycle on something small enough to see the whole shape at once. This needs one tool you haven't installed yet — pytest, the test runner this book uses from here on. Section 5.3 covers it properly; for now, just get it installed so this example can actually run:

```
uv add --dev pytest
```

This adds `pytest` to the `dev` dependency group in `pyproject.toml` — alongside `ruff`, already there since Chapter 3.4.1:

```
[dependency-groups]
dev = [
    "ruff>=0.14.0",
    "pytest>=8.4.2",
]
```

(Your exact version numbers may differ.) This is the same mechanism `uv add requests` used back in 0.7.4, just landing in a group reserved for tools you use while developing rather than ones the calculator needs at runtime.

Create a `tests` directory and open a new file in it — this is a good first real use for the vi skills from 0.5:

```
mkdir tests
vi tests/test_scratch.py
```

Type in the following (`i` to start typing, `Esc` then `:wq` to save and exit):

```
def test_add_returns_sum():
    from mortgage_calculator_book.scratch import add
    assert add(2, 3) == 5
```

Run it:

```
pytest tests/test_scratch.py -v
```

It fails — an import error, since `mortgage_calculator_book.scratch` doesn't exist yet, rather than a failed assertion, but a failure either way. That's **red**. Now create the missing piece the same way:

```
vi src/mortgage_calculator_book/scratch.py
```

```
def add(a, b):
    return a + b
```

Run the test again:

```
pytest tests/test_scratch.py -v
```

Green. There's nothing to refactor yet, but the cycle is complete. Delete both scratch files once you've felt the shape of it — they're not part of the real project:

```
rm tests/test_scratch.py src/mortgage_calculator_book/scratch.py
```

5.3 pytest Basics

5.3.1 Installing

5.2.3 already had you install this, to make the tiny example runnable. If you skipped ahead and don't have it yet:

```
uv add --dev pytest
```

5.3.2 Test Discovery

pytest finds tests automatically, following a convention rather than an explicit list: files named `test_*.py` (or `*_test.py`), inside functions named `test_*`. This is why the scratch example above worked without registering it anywhere — pytest went looking for exactly that pattern.

5.3.3 Assertions

pytest uses Python's plain `assert` statement — no special assertion methods to memorize. When an assertion fails, pytest rewrites the failure output to show you both sides of the comparison, which is more useful than it sounds the first time you see a failing test explain itself clearly.

5.3.4 Running the Suite

```
pytest              # run everything
pytest -v           # verbose - show each test by name
pytest tests/test_core.py       # run one file
pytest tests/test_core.py::test_matches_worked_example    # run
one test
```

5.4 Fixtures, Just Enough

5.4.1 The Problem They Solve

Several tests in this chapter need the same data — the Chapter 4.5 worked example's principal, rate, term, and expected payment. Repeating those numbers in every test function invites the numbers to quietly drift apart from each other over time. A **fixture** is pytest's answer: shared setup, defined once, requested by any test that needs it.

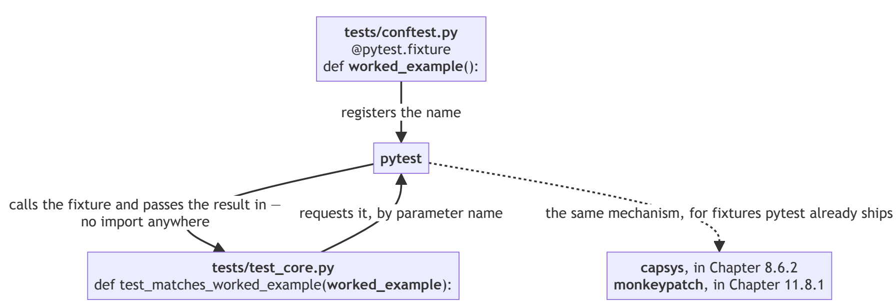
5.4.2 The Worked Example as a Fixture

Create `tests/conftest.py`:

```
import pytest

@pytest.fixture
def worked_example():
    """Answer key from docs/derivation.md: $200k/6%/30yr
    monthly."""
    return {
        "principal": 200_000,
        "annual_rate": 0.06,
        "term_years": 30,
        "payments_per_year": 12,
        "expected_payment": 1199.10,
    }
```

Any test function that takes `worked_example` as a parameter automatically receives this dictionary — pytest matches it by name, no import needed.

That last clause is worth a picture, because nothing else in Python works this way:



The parameter name *is* the wiring. Rename the fixture and every test that asked for it stops working; rename the parameter and pytest will tell you it doesn't recognise the name. That's also why `capsys` and `monkeypatch` later need no import — they're the same mechanism with pytest supplying the fixture instead of your `conftest.py`.

5.4.3 Scope-Setting

pytest's fixture system goes considerably further than this — different scopes, fixtures that depend on other fixtures, fixtures shared across an entire test session. This book uses exactly the pattern above and nothing more; if you want the fuller picture later, pytest's own documentation is thorough and well-written.

5.5 Encoding the Answer Key as a Test

5.5.1 Translating the Worked Example

Create `tests/test_core.py`:

```
from mortgage_calculator_book.core import calculate_payment

def test_matches_worked_example(worked_example):
    payment = calculate_payment(
        principal=worked_example["principal"],
        annual_rate=worked_example["annual_rate"],
        term_years=worked_example["term_years"],
        payments_per_year=worked_example["payments_per_year"],
    )
    assert payment == pytest.approx(
        worked_example["expected_payment"], abs=0.01
    )
```

Note `pytest.approx(..., abs=0.01)` rather than a plain `==`. Chapter 4.5 computed the exact payment as `1199.1010503...` before rounding — comparing floating-point results with plain equality is a reliable way to get bitten by a rounding difference in the fifteenth decimal place that has nothing to do with whether your code is actually correct. `abs=0.01` means "correct to the cent," which is the precision that actually matters for currency here. You don't need `import pytest` twice — add it to the top of the file alongside the `calculate_payment` import.

5.5.2 Running It — and Watching It Fail

```
pytest tests/test_core.py -v
```

This fails immediately, with an import error: `mortgage_calculator_book.core` doesn't exist yet. **This is the "red" in red/green/refactor, made concrete.** You haven't done anything wrong.

5.5.3 Why a Failing Test Is a Milestone

It's worth pausing on this rather than rushing past it: you now have a precise, automatic, unambiguous definition of what Chapter 6 needs to build. Not "make a mortgage calculator" in the abstract — make this exact test pass. That's a meaningfully smaller, clearer target than the one you started the book with.

5.6 Agent-Assisted TDD: Letting Pi Draft Test Skeletons

5.6.1 Prompting from the Spec

```
pi "Read SPEC.md and propose additional pytest test \
    cases for calculate_payment, beyond the worked \
    example already in tests/test_core.py. Don't \
    implement calculate_payment itself - just propose \
    the test functions."
```

5.6.2 Reviewing What It Proposes

Read every proposed test with two questions in mind: does it test *behavior* described in SPEC.md, or does it quietly test some *implementation detail* that hasn't been decided yet? And does its expected value actually follow from the formula in Chapter 4.4, or is it a plausible-looking number Pi generated without truly computing it? A test with a wrong expected value is worse than no test at all — it will pass or fail for the wrong reasons, and it'll sit there looking trustworthy until something forces you to check it by hand.

5.6.3 Accepting, Correcting, or Rejecting Each One

Treat this the same way you've treated every other agent proposal so far: some of what Pi suggests will be worth keeping as-is, some will need a corrected expected value, and some won't belong in this test file at all. All three outcomes are normal.

> **Process concept: tests as the executable proof of the spec, not a restatement of it.** A test that just re-describes SPEC.md in code form ("test that calculate_payment exists and returns a number") isn't pulling its weight. A good test asserts something specific enough that it could actually fail in a way that teaches you something — which is exactly what section 5.5's worked-example test does, and what you should be checking Pi's proposals against.

5.6.4 What a Larger Model Actually Produced

The 5.6.1 prompt against a larger model — qwen3.8:27b-mlx, from Chapter 1.4.2 — can produce something considerably more ambitious than a handful of test skeletons: nine cases plus three explicit "I won't guess at this, decide first" questions about behavior SPEC.md doesn't define (what should happen at `term_years = 0`? should negative principal raise or just compute?). That last part is worth calling out on its own — flagging an undecided contract instead of quietly assuming one is exactly the discipline this book has been asking *you* to practice, showing up unprompted in the agent's own output.

Some of the proposed cases are genuinely strong, and worth knowing about even though this book's test suite stays simpler: a **present-value round-trip** check — that discounting every payment back at the periodic rate recovers the original principal, which is SPEC.md's own definition of "correct" turned into a test that doesn't depend on the implementer's formula at all — plus **linearity** (doubling the principal should exactly double the payment) and **monotonicity** (a longer term should always lower the payment; a higher rate should always raise it). These are called *property tests*: instead of asserting one hand-computed number, they assert a relationship that has to hold regardless of how the formula is written. That's a real technique, worth filing away even if this book doesn't build a full property-based suite around it.

One proposed case uses a pytest feature this book hasn't shown yet — **parametrize** — and it's worth pausing on the syntax before getting to what went wrong with it. Rather than write four nearly-identical test functions (one each for annual, quarterly, biweekly, and weekly payments), `@pytest.mark.parametrize` lets you write the test body *once* and run it several times, once per row of a table you supply. The decorator's first argument is a string naming the values that change from run to run; the second is a list of tuples supplying those values, one tuple per run. Pytest calls the test function once per tuple, passing each value in by name — which means the function's own parameter list has to actually include those names, or pytest has nowhere to put them. Here, `ppy` is short for "payments per year" — matching the `payments_per_year` argument `calculate_payment` itself expects — and `expected` is the payment each row claims is correct:

```
@pytest.mark.parametrize("ppy,expected", [
    (1,  212_000.0),
    (4,  3603.70),
    (26, 553.17),
    (52, 594.74),
])
def test_non_monthly_frequencies(
    principal=200_000, annual_rate=0.06, term_years=30
):
    payment = calculate_payment(
        principal=principal, annual_rate=annual_rate,
```

```
        term_years=term_years, payments_per_year=ppy,
    )
    assert payment == pytest.approx(expected, abs=0.01)
```

Try running this exactly as given, and it doesn't get anywhere near checking any numbers — pytest refuses to even collect it:

```
ERROR test_core.py::test_non_monthly_frequencies:
function uses no argument 'ppy'
```

Look at the function signature and the reason is right there: `test_non_monthly_frequencies` takes `principal`, `annual_rate`, and `term_years` — but never `ppy` or `expected`, the two names the parametrize table is trying to hand it. Pytest has values to inject and nowhere to put them. That's the first, easiest thing to catch here — a beginner doesn't need to verify a single number by hand to notice the whole test won't even run. The fix is mechanical: add the missing names to the signature.

```
def test_non_monthly_frequencies(
    ppy, expected, principal=200_000, annual_rate=0.06,
term_years=30
):
```

Fix that, and the test is at least *runnable* — though not yet, in this project specifically: `calculate_payment` doesn't exist until Chapter 6, so right now this would fail with the same import error every other test in this chapter fails with, not a clean pass/fail split. The two-pass-two-fail result below is what you'd see *after* Chapter 6's implementation exists — worth keeping straight so you're not confused when trying this today gets you an import error instead. Once that implementation exists, though, it's genuinely informative: two rows pass, two fail. `ppy=4` and `ppy=26` check out. `ppy=1` and `ppy=52` don't, and it's worth working out why by hand — the same way 4.6 asked you to check Pi's arithmetic — rather than assuming the numbers are simply wrong. At `ppy=1`, `calculate_payment` actually returns about **$14,529.78**, not the $212,000 the table claims. But $212,000 isn't a made-up number — it's exactly what a *single* payment after one year on this loan would be (the $200,000 growing by 6% interest, paid off in one shot), a completely different scenario from the one this row's actual parameters describe (30 *annual* payments, not 1). Same story at `ppy=52`: the real result is about **$276.53**, while $594.74 is the correct weekly payment for a $30,000 loan over one year — again a genuine number, just answering a different question than the one this test's fixed `principal=200_000, term_years=30` actually poses.

Both wrong rows are individually correct arithmetic, attached to the wrong problem. That's a harder mistake to catch than a plain miscalculation, because nothing about $212,000 or $594.74 looks unreasonable in isolation — it only falls apart once you check each row against the *specific* inputs that row is actually parametrized to use, which is exactly why 5.6.2 asked you to verify expected values rather than skim them for plausibility. Two layers of review, in other words: does it even run, and only then, does it compute the right thing.

Pointing this out and asking Pi to take another pass — the normal next step, not a special one — produced a version worth looking at, because the fix isn't just "corrected numbers":

```
@pytest.mark.parametrize(
    "principal, annual_rate, term_years, payments_per_year,
    expected",
    [
        (200_000, 0.06, 30,  1, 14_529.78),  # annual
        (200_000, 0.06, 30,  4,  3_603.70),  # quarterly
        (200_000, 0.06, 30, 12,  1_199.10),  # monthly (== worked
example)
        (200_000, 0.06, 30, 26,    553.17),  # biweekly
        (200_000, 0.06, 30, 52,    276.53),  # weekly
    ],
)
def test_non_monthly_frequencies(
    principal, annual_rate, term_years, payments_per_year,
expected
):
    payment = calculate_payment(
        principal=principal, annual_rate=annual_rate,
        term_years=term_years,
payments_per_year=payments_per_year,
    )
    assert payment == pytest.approx(expected, abs=0.01)
```

This runs and all five rows pass — verify that independently, the same way as everything else in this section, rather than taking a second round of Pi's output any more on faith than the first. But notice *what* changed, not just that the numbers are now right: every row now carries its own `principal`, `annual_rate`, and `term_years` explicitly, instead of leaving three of the four inputs as fixed defaults in the function signature and only varying `payments_per_year` per row. That's not a cosmetic difference — it's what actually made the original bug possible in the first place. When only `ppy` varied per row, a row's expected value could quietly describe a *different* principal or term than the one the test would actually run with, and nothing about the test's structure would catch it. With every input explicit in the same row as its answer, that entire class of mistake has nowhere left to hide — a row is either internally consistent or it visibly isn't, without needing to cross-reference a default hiding three lines up. It also picked up a nice detail worth noticing on your own next time: the `payments_per_year=12` row's expected value is `1_199.10` — the exact Chapter 4.5 worked example, now sitting inside this table as a built-in cross-check against a number you already trust.

None of this means the larger model did badly — quite the opposite; the property tests and the "flag it, don't guess" section are better instincts than a lot of hand-written test suites manage. It means review scales with ambition: the more a model gives you, the more of it actually needs checking, not less.

5.7 Debugger Basics: pdb

5.7.1 When to Reach for It

Once Chapter 6 exists and something eventually fails in a way that isn't obvious from the error message alone, the instinct to immediately ask Pi to fix it is worth resisting, at least briefly. Finding your own bugs first — even occasionally, even just to practice — builds exactly the judgment you'll need to properly review Pi's fixes later. If you can't tell whether a fix actually addresses the bug, you're not really reviewing it.

5.7.2 Trying It, On Something That Actually Exists

Everything else in this chapter is either already correct (the worked-example fixture) or deliberately red because `calculate_payment` doesn't exist yet — that's Chapter 6's job, not a bug to step through. Neither gives you a real failure to investigate *right now*. So build one, small and throwaway, the same way 5.2.3 did.

```
vi tests/test_scratch_debug.py
```

```python
from mortgage_calculator_book.scratch_debug import total_paid

def test_total_paid_sums_all_payments():
    assert total_paid([100, 200, 300]) == 600
```

```
vi src/mortgage_calculator_book/scratch_debug.py
```

```python
def total_paid(payments):
    """Sum a list of periodic payments."""
    total = 0
    for amount in payments:
        total = amount
    return total
```

Run it:

```
pytest tests/test_scratch_debug.py -v
```

It fails: `assert 300 == 600`. That tells you *that* something's wrong, not *why*. Rather than stare at four lines until it jumps out — it will eventually, but that's not the point of this exercise — drop a breakpoint into the loop and watch it happen:

```python
def total_paid(payments):
    total = 0
```

```
    for amount in payments:
        breakpoint()
        total = amount
    return total
```

Run the test again. Execution pauses the first time the loop body runs, before that line executes. Print both variables (`p total` → `0`, `p amount` → `100`), then step past the assignment (`n`) and print `total` again — it's `100`, matching `amount`, which is expected after the first payment. Continue to the next iteration (`c`); you're back at the same breakpoint, `amount` now `200`, `total` still `100` from before. Step past the assignment again (`n`) and print `total` once more: it's `200` — not `300`, which is what `100 + 200` would give you if the loop were accumulating. That's the bug, watched directly rather than guessed at: `total = amount` should be `total += amount`.

Fix it, delete the `breakpoint()` line, and confirm:

```
pytest tests/test_scratch_debug.py -v
```

Green. Delete both scratch files — not part of the real project, same as 5.2.3's:

```
rm tests/test_scratch_debug.py \
   src/mortgage_calculator_book/scratch_debug.py
```

For reference, the commands you just used:

```
n         step to the next line
s         step into a function call
c         continue running until the next breakpoint (or the end)
p some_variable   print a variable's current value
```

5.7.3 Why This Is Worth a Page

This isn't meant to make you a debugging expert — a page of practice won't do that. It's meant to give you one more tool between "the test failed" and "ask the agent to fix it," so that when you do read Pi's proposed fix later, you're reading it as someone who at least attempted to understand the failure themselves, not as someone encountering the bug for the first time via someone else's patch.

5.8 Edge Cases Worth Testing Now

5.8.1 The Two Cases from Chapter 4.4.3

Add these to `tests/test_core.py`, even though `calculate_payment` doesn't exist yet:

```
def test_zero_interest_loan():
    payment = calculate_payment(
        principal=12_000,
        annual_rate=0.0,
```

```
        term_years=1,
        payments_per_year=12,
    )
    assert payment == pytest.approx(1000.00, abs=0.01)


def test_single_payment():
    payment = calculate_payment(
        principal=10_000,
        annual_rate=0.06,
        term_years=1,
        payments_per_year=1,
    )
    assert payment == pytest.approx(10600.00, abs=0.01)
```

Both use the exact numbers from Chapter 4.5's edge-case discussion — chosen there specifically because they're clean enough to verify by hand (\$12,000 ÷ 12 = \$1,000; \$10,000 × 1.06 = \$10,600).

5.8.2 Planting Red Tests on Purpose

Run the full suite now:

```
pytest -v
```

Same collection error as 5.5.2, not three separate failures — `mortgage_calculator_book.core` still doesn't exist, so pytest can't even import the file to reach any of the three tests inside it, worked example and both edge cases alike:

```
ERROR tests/test_core.py
ModuleNotFoundError: No module named
'mortgage_calculator_book.core'
```

That's still exactly the state Chapter 6 needs to find this project in — one missing module standing between here and green, not three separate bugs to chase. You'll see three individually reported passes once Chapter 6 exists; right now, before any implementation exists at all, one honest collection error covering all three is the correct red, not a sign something's already wrong.

This red suite is worth committing — it's a real, intentional artifact of this project, not a mistake to hide:

```
git add .
git commit -m "Add pytest and a red test suite for
calculate_payment"
git push
```

5.9 Checkpoint

Before moving to Chapter 6, this should all be true:

- ☐ pytest is installed as a dev dependency, and `pyproject.toml` shows it under `[dependency-groups]`
- ☐ `tests/conftest.py` defines the `worked_example` fixture
- ☐ `tests/test_core.py` contains three tests: the worked example, zero-interest, and single-payment — all currently red via one collection error, since `core.py` doesn't exist yet
- ☐ You can explain why each test's expected value is correct, independent of any code
- ☐ `ruff check .` and `ruff format .` both pass on the test files
- ☐ You've used `breakpoint()` at least once, even in the scratch example
- ☐ The red suite is committed and pushed — not just sitting locally

What's next: Chapter 6 makes these three tests go green — the first real implementation this project has had.

Chapter 6 — Mathematical Core Library

Contents

6.1 Why This Chapter Exists

Chapter 5 left you with three failing tests and a precise target: make them pass, using the formula derived in Chapter 4.4. This chapter does exactly that — the first real implementation this project has had, and the first real payoff of the red/green/refactor cycle set up in Chapter 5.

By the end, all three tests from Chapter 5 will be green, and you'll have a small, pure module that every later front end — CLI, UI, and eventually a language model — calls into without ever duplicating this logic.

6.2 What "Pure" Means, and Why It's the Rule Here

6.2.1 Pure Functions

A function is **pure** if it does one thing: given the same inputs, it always returns the same output, with no side effects — no printing, no reading files, no reaching out to a network, no depending on anything outside its own arguments. `calculate_payment(200_000, 0.06, 30, 12)` should return the same number every single time it's called, forever, with nothing else going on.

6.2.2 The Rule for This Module

Everything in `mortgage_calculator_book.core` follows this rule, with no exceptions. No `print()` statements for debugging left behind, no reading configuration, no formatting output for a human to read — that's the CLI's job in Chapter 8 and the UI's job in Chapter 9, not this module's.

> **Process concept: why purity matters more, not less, with an agent in the loop.** A pure function is a narrow, easily verified contract: given these inputs, this exact output. That's precisely the kind of task an agent can be checked against cheaply and completely — run the function, compare the result, done. The moment a function reaches out to the filesystem, the network, or global state, verifying an agent's implementation of it becomes considerably harder, because "correct" now depends on things outside the function's own signature. Keeping the core pure isn't just good architecture in the abstract — it's what makes Chapter 6.5's agent-assisted implementation actually checkable.

6.3 Module Layout

6.3.1 Where This Lives

Inside the `src/mortgage_calculator_book/` package from Chapter 0.7.3, create `core.py`. This is the only file this chapter touches.

6.3.2 Naming Things to Match the Spec

Function names in this module should read as a direct translation of `SPEC.md`'s "Derived quantities" and "Outputs" sections from Chapter 4.7.4 — not because it's required, but because it means anyone (or anything) reading the spec and the code side by side can match them up on sight.

6.4 Implementing the Periodic Rate and Payment-Count Helpers

6.4.1 Annual Rate to Periodic Rate

The conversion flagged repeatedly since Chapter 4.3.2, as its own small, independently testable function:

```
def annual_rate_to_periodic(
    annual_rate: float, payments_per_year: int
) -> float:
    """Convert an annual interest rate to the rate for a single
    period."""
    return annual_rate / payments_per_year
```

6.4.2 Term and Frequency to Total Payments

```
def total_payments(term_years: int, payments_per_year: int) ->
int:
    """Total number of payments over the life of the loan."""
    return term_years * payments_per_year
```

Both are one-liners, and that's fine — the point of separating them out isn't complexity, it's giving each conversion a name and a place where a mistake in either one is easy to isolate and test on its own, rather than buried inside a larger formula.

6.4.3 Watching the Suite

These two helpers don't make any of Chapter 5's three tests pass on their own — `calculate_payment` still doesn't exist. Run `pytest -v` anyway, out of habit; still red, as expected.

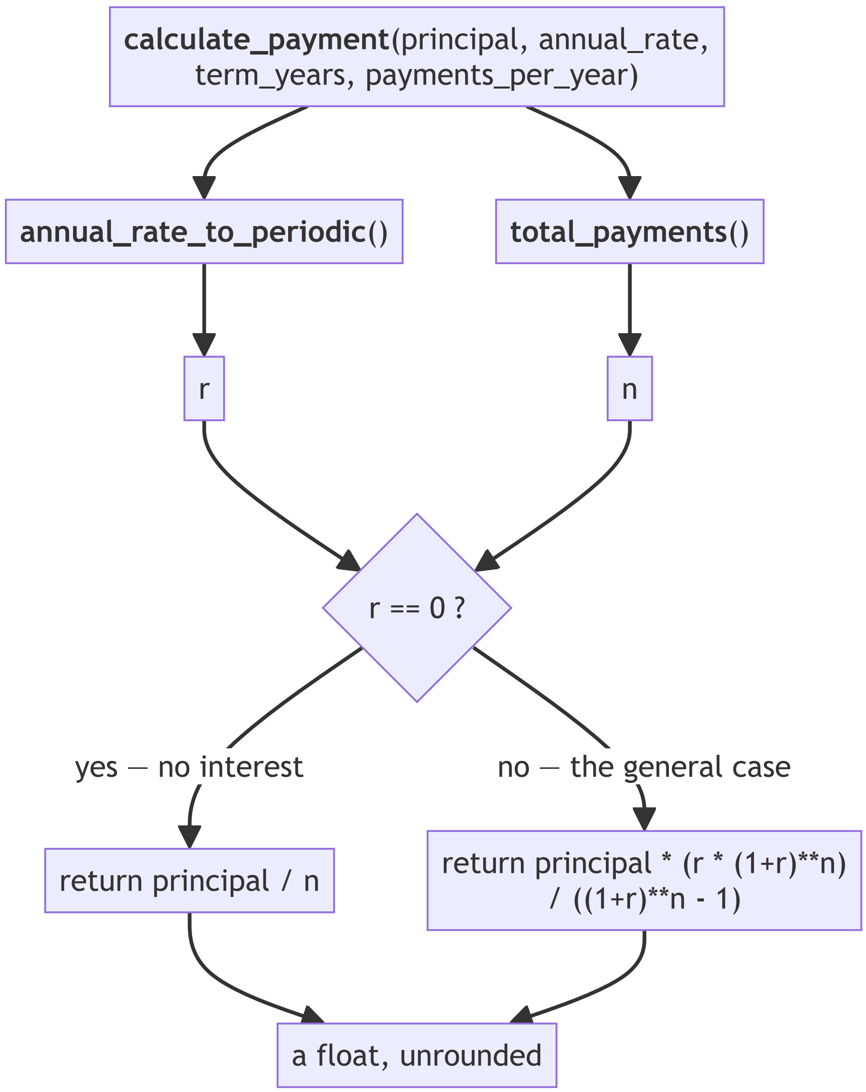
6.5 Implementing the Fixed Payment Formula

6.5.1 Code That Mirrors the Math

Translated directly from Chapter 4.4.1, term for term:

```python
def calculate_payment(
    principal: float,
    annual_rate: float,
    term_years: int,
    payments_per_year: int = 12,
) -> float:
    """Fixed periodic payment for a fixed-rate loan (see
    SPEC.md)."""
    r = annual_rate_to_periodic(annual_rate, payments_per_year)
    n = total_payments(term_years, payments_per_year)

    if r == 0:
        return principal / n

    return principal * (r * (1 + r) ** n) / ((1 + r) ** n - 1)
```

Drawn out, the module is the same picture as 4.3's inputs-and-derived-quantities diagram, with the 4.4.3 edge case added as the one branch:



Two helpers, one branch, one return type, and nothing else — no printing, no files, no network. That flatness is what "pure" bought you, and it's why 6.5.3's review checklist is only three items long: there are only three places this function can go wrong.

Notice how closely this reads against the formula: `r * (1 + r) ** n` in the numerator, `(1 + r) ** n - 1` in the denominator — the same shape as

$\dfrac{r(1+r)^n}{(1+r)^n-1}$ from 4.4.1, not an optimized or rearranged version of it. Clarity against the derivation matters more here than performance; this function will never be a bottleneck in this project.

6.5.2 Agent-Assisted Implementation

If you'd rather have Pi draft this rather than typing it yourself:

```
pi "Implement calculate_payment in \
    src/mortgage_calculator_book/core.py to make the \
    tests in tests/test_core.py pass. The formula is in \
    docs/derivation.md; SPEC.md defines what counts as \
    correct output, not how to compute it. Keep the \
    function pure - no I/O, no printing."
```

6.5.3 Reviewing Before Accepting

Whether you wrote it or Pi did, check specifically for the mistakes this formula invites:

- **Rate conversion**: does it use the *periodic* rate inside the exponentiation, not the raw annual rate?
- **Off-by-one in payment count**: is `n` computed as `term_years * payments_per_year`, not something shifted by one?
- **Silent float issues**: is there an unguarded division that could produce `inf` or a `ZeroDivisionError` for some input this module hasn't been asked to validate yet? (Chapter 7 handles rejecting bad input; this module just needs to not silently misbehave on the inputs it's given.)

6.6 Handling the Named Edge Cases

6.6.1 Zero-Interest Loans

The `if r == 0` branch above exists for exactly the reason 4.4.3 described: the general formula divides by `r`, which is undefined at zero, even though the real-world answer (`principal / n`) is completely well-defined. Run the zero-interest test:

```
pytest tests/test_core.py::test_zero_interest_loan -v
```

6.6.2 Single-Payment Terms

Unlike the zero-interest case, this one needs **no special branch** — as 4.4.3 noted, substituting $n = 1$ into the general formula produces the same answer as the direct "principal plus one period's interest" reasoning. Run it to confirm:

```
pytest tests/test_core.py::test_single_payment -v
```

If this test fails, it's worth checking your formula's algebra against 4.4.1 line by line before adding a special case you don't actually need — a passing test achieved by adding an unnecessary branch is a sign something upstream is subtly wrong, not a problem solved.

6.6.3 Confirming Both Are Genuinely Fixed

Run the full suite:

```
pytest -v
```

If you see three passes here rather than three failures, double-check that you're reading real green — not a test that was quietly weakened (a loosened `abs=` tolerance, a rewritten expected value) to make a stubborn failure go away. That's the opposite of what this chapter is for.

6.7 Verifying Against the Answer Key

6.7.1 The Worked Example

```
pytest tests/test_core.py::test_matches_worked_example -v
```

This should now pass, confirming `calculate_payment(200_000, 0.06, 30, 12)` returns something within a cent of `$1,199.10`.

6.7.2 If It Doesn't Match

In order of likelihood, check: is the periodic rate actually `annual_rate / payments_per_year`, computed *before* being raised to a power (a rate accidentally raised to a power before dividing is a classic version of this bug)? Is `n` actually 360, not 30 (term years, forgetting to multiply by frequency)? Is the formula's numerator and denominator each matching 4.4.1 exactly, with no terms transposed? These three, in this order, catch the overwhelming majority of first attempts that don't match.

6.8 Refactor

6.8.1 The Refactor Step, For Real This Time

Chapter 5.2's tiny `add` example didn't have anything worth refactoring. This one might: are `annual_rate_to_periodic` and `total_payments` named clearly? Is `calculate_payment` still readable against the formula, or did fixing 6.7.2's bug leave behind anything awkward? Clean it up now, with the full test suite as a safety net — this is exactly what the "refactor" step is for.

6.8.2 Ruff Pass

```
ruff check . && ruff format .
```

Consistent with the habit established in Chapter 3.6.1 — every commit, not just some.

6.8.3 A Note on Rounding

You may notice `calculate_payment` currently returns full floating-point precision (`1199.1010503055138`), not a value already rounded to the cent — the tests handle that with `pytest.approx(..., abs=0.01)` rather than the function itself rounding. This is deliberate for now: rounding is a presentation decision, and mixing it into the core risks baking a specific display choice into a function that's supposed to stay pure and general. Chapter 7's validation layer is where this book properly addresses currency precision and bounds — flagged here so it doesn't feel like an oversight when it doesn't come up again until then.

6.9 Checkpoint

Before moving to Chapter 7, this should all be true:

- ☐ `src/mortgage_calculator_book/core.py` contains `annual_rate_to_periodic`, `total_payments`, and `calculate_payment`
- ☐ All three tests from Chapter 5 pass: `pytest -v` shows three green
- ☐ `calculate_payment` has no I/O, no printing, no dependencies outside its own arguments
- ☐ You can explain, without looking at the code, what each of the three checks in 6.7.2 is guarding against
- ☐ `ruff check .` and `ruff format .` both pass
- ☐ Everything is committed

What's next: Chapter 7 wraps this trusted, pure core in a validation layer — because so far, `calculate_payment` will happily compute a confident, wrong-looking answer for a negative principal or an absurd interest rate, and nothing has stopped it from being asked to.

Chapter 7 — Validation

Contents

7.1 Why This Chapter Exists

Chapter 6 left you with a trusted, correct core — but one that will compute a confident answer for a negative principal, a 400% interest rate, or a term of zero years, because nothing has ever told it not to. This chapter builds the layer that catches bad input before it ever reaches `calculate_payment`.

By the end, a Pydantic-based input model will sit between the outside world and Chapter 6's core, fully tested, enforcing exactly what SPEC.md says is valid.

7.2 Why Validation Is Its Own Layer, Not Bolted onto the Core

7.2.1 Revisiting Purity

Chapter 6.2 established a firm rule: the core has no I/O, no dependencies outside its own arguments — nothing but calculation. Validation is different in kind, not just in code: it exists specifically to deal with input from the outside world, which is exactly what the core was built to never touch directly. Putting a check like `if principal <= 0: raise ValueError` inside `calculate_payment` would quietly violate the boundary Chapter 6 spent a whole chapter establishing.

7.2.2 What Skipping This Layer Costs

Without it, bad input doesn't announce itself clearly — it either crashes somewhere unhelpful (a `ZeroDivisionError` three function calls deep, far from where the bad number actually entered the system) or, worse, produces a number that looks like a real answer but means nothing (a "payment" computed from a negative principal). Neither is a good experience for a user, and neither is easy to debug.

7.2.3 Where This Is Headed

Notice the shape of what you're about to build: a declarative description of what valid input looks like, expressed as a schema. Chapter 10's tool interface needs almost exactly this same shape — a schema a language model can read and be held to — which is why this chapter's work gets reused rather than redone.

7.3 What Needs Validating

7.3.1 Revisiting SPEC.md's Inputs

From Chapter 4.7.4's revised spec: `principal`, `annual_rate`, `term_years`, `payments_per_year`. Each needs a definition of "valid" beyond just "the right type":

- `principal` — must be positive; a loan of zero or negative dollars isn't a loan.
- `annual_rate` — must be a plausible rate. Zero is valid (Chapter 6.6.1's edge case); something like 1.5 (150%) almost certainly represents a typo, not a real mortgage.
- `term_years` — must be positive.
- `payments_per_year` — must be positive.

7.3.2 Turning Edge Cases into Explicit Boundaries

Chapter 6's zero-interest case showed that `annual_rate = 0` is a legitimate input, not an error — which means the validation boundary here has to be "rate cannot be negative," not "rate must be strictly positive." Get this wrong and you'll accidentally reject a test case Chapter 6 was specifically built to handle.

7.4 Introducing Pydantic

7.4.1 The Problem It Solves

A validation function built from a chain of `if` statements works, but it scales badly — the checks live separately from the data they're checking, error messages have to be written by hand for every case, and there's no single place that describes what "valid input" even means at a glance. Pydantic solves this by letting you declare validity as part of a data model's definition, rather than as separate procedural code.

7.4.2 Installing

```
uv add pydantic
```

Unlike Ruff and pytest, this isn't a `--dev` dependency — the calculator needs Pydantic at runtime, not just while you're developing it.

7.4.3 A First Minimal Model

Try this directly rather than just reading it — and unlike 5.2.3 or 5.7.2, there's no file to create or delete here. This is small enough to try straight in Python's interactive prompt (its **REPL** — "read-eval-print loop": type an expression, see its result immediately, repeat). Start one inside your project's environment:

```
uv run python
```

You're now looking at a `>>>` prompt, running inside your project the same way `uv run` always does. Type this, pressing Enter after each line (a blank line after the `class` block runs it):

```
from pydantic import BaseModel

class Example(BaseModel):
    name: str
    age: int
```

Now try it with valid data:

```
Example(name="Robert", age=40)
```

You should see `Example(name='Robert', age=40)` printed back — it worked, Pydantic accepted it. Now try something that doesn't match the declared type:

```
Example(name="Robert", age="not a number")
```

This time Pydantic raises a `ValidationError` — a real, detailed error naming exactly which field failed and why, without you writing a single `isinstance` check yourself. Read what it prints; it's more informative than a typical Python `TypeError`, and it's the same style of error you'll be reading for the rest of this chapter.

Exit the REPL when you're done — same key you used to leave Pi's interactive session back in 1.6.1: **Ctrl+D**.

Nothing here needed saving or cleaning up; it never touched a file in the first place. Section 7.5 is where a version of this actually becomes part of the project.

7.5 Building the Input Model

7.5.1 The Model

In `src/mortgage_calculator_book/validation.py`:

```
from pydantic import BaseModel, ConfigDict, field_validator


class MortgageInput(BaseModel):
    """Validated input for a fixed-rate mortgage payment
    calculation."""

    model_config = ConfigDict(extra="forbid")

    principal: float
    annual_rate: float
    term_years: int
    payments_per_year: int = 12

    @field_validator("principal")
    @classmethod
    def principal_must_be_positive(cls, v: float) -> float:
```

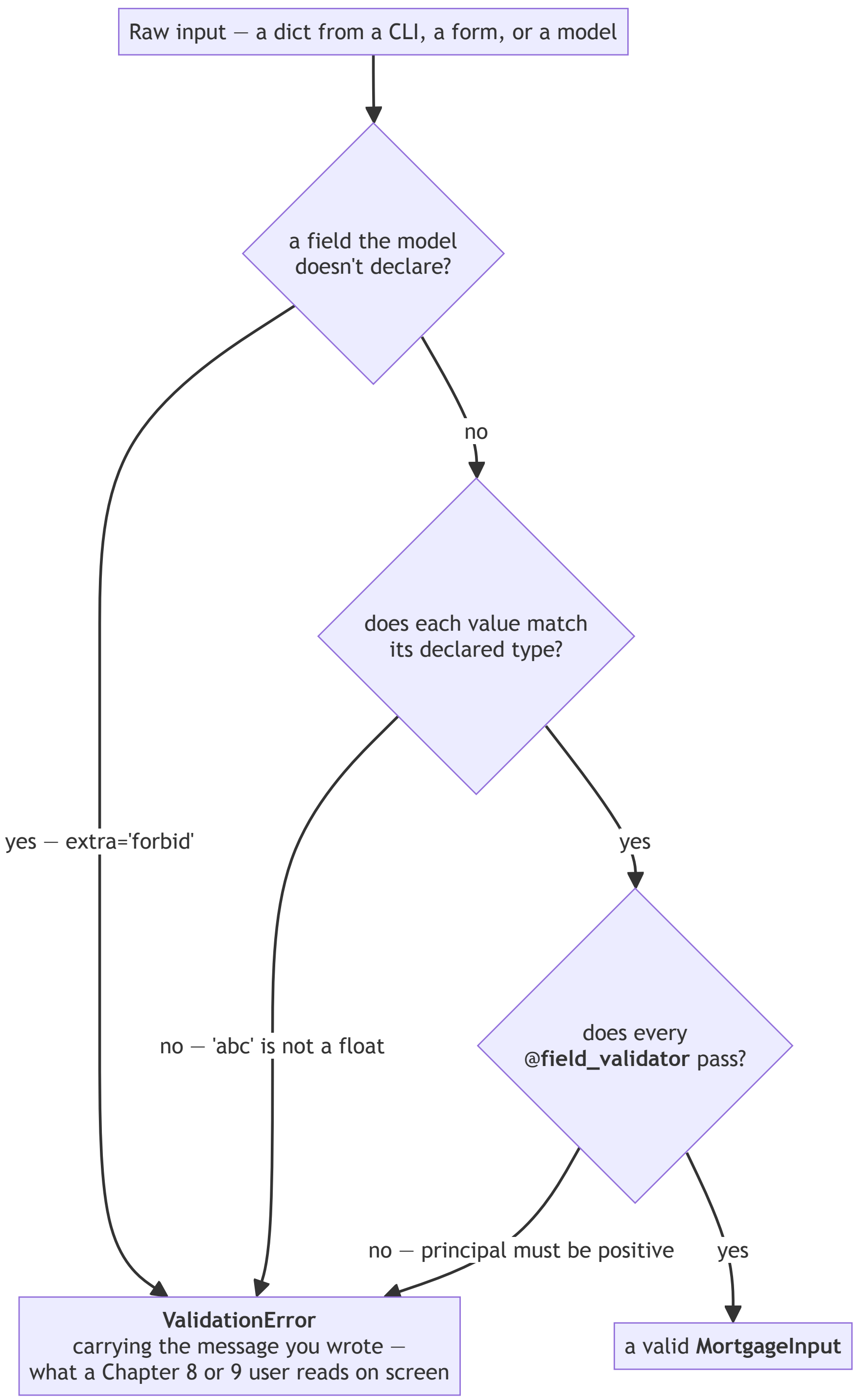
```
        if v <= 0:
            raise ValueError("principal must be positive")
        return v

    @field_validator("annual_rate")
    @classmethod
    def rate_must_be_plausible(cls, v: float) -> float:
        if v < 0:
            raise ValueError("annual_rate cannot be negative")
        if v >= 1:
            raise ValueError("annual_rate must be less than 1
            (e.g. 0.06 for 6%)")
        return v

    @field_validator("term_years")
    @classmethod
    def term_must_be_positive(cls, v: int) -> int:
        if v <= 0:
            raise ValueError("term_years must be positive")
        return v

    @field_validator("payments_per_year")
    @classmethod
    def payments_per_year_must_be_positive(cls, v: int) -> int:
        if v <= 0:
            raise ValueError("payments_per_year must be
            positive")
        return v
```

`model_config = ConfigDict(extra="forbid")` is easy to skip, since everything works without it — Pydantic's default is to silently *ignore* fields it doesn't recognize, not reject them. That's a real gap, not a hypothetical one: without this line, `MortgageInput(principal=200_000, annual_rate=0.06, term_years=30, something_unexpected="test")` succeeds silently, dropping `something_unexpected` without a word. Harmless-looking today, from a human typing a mistyped field name — considerably less harmless once Chapter 10 hands this same model a dictionary built from whatever a language model decided to send.

7.5.2 Custom Validators, Explained

Each `@field_validator` runs after Pydantic's own type checking, and raises a plain `ValueError` with a message written for a human to read — not a generic type error. Notice `rate_must_be_plausible` allows exactly zero (`v < 0` rejects negative, but zero passes through) while rejecting anything at or above 100% — matching 7.3.2's requirement precisely.

Three checks run in a fixed order, and any one of them can end the trip:



Order matters more than it looks. Type checking runs *before* your validators, which is why `principal_must_be_positive` can assume it has a float and never needs an `isinstance` check of its own. And `extra="forbid"` runs first of all, which is what makes 7.5.1's warning about a silently dropped field a real protection rather than a nicety.

7.5.3 Error Messages as a User-Facing Surface

The strings inside each `raise ValueError(...)` aren't incidental — they're what a user of Chapter 8's CLI or Chapter 9's UI will actually see when they enter something invalid. Write them the way you'd want to read them yourself: specific about what's wrong, not just "invalid input."

Commit this, the same as every other real step in this project:

```
git add src/mortgage_calculator_book/validation.py
git commit -m "Add MortgageInput validation model"
git push
```

7.6 Agent-Assisted TDD, Applied to Validation

7.5 had you type `MortgageInput` by hand, in full, before running a single test against it — useful for seeing the whole class at once, but not actually how this book has been building things since Chapter 5: tests first, then an implementation that has to earn a passing result. This section does it that way for real, which means setting aside the version you just wrote and committed.

7.6.1 Setting Aside What You Just Built

Because 7.5.3 committed that version, deleting it now doesn't lose anything — it's still sitting in git history if you want it later, including for a comparison at the end of this section:

```
git rm src/mortgage_calculator_book/validation.py
git commit -m "Remove hand-written validation.py to redo via TDD"
```

7.6.2 Writing the Failing Tests First

In `tests/test_validation.py`:

```
import pytest
from pydantic import ValidationError

from mortgage_calculator_book.validation import MortgageInput


def test_valid_input_accepted():
    result = MortgageInput(
```

```
        principal=200_000, annual_rate=0.06, term_years=30,
payments_per_year=12
    )
    assert result.principal == 200_000


def test_negative_principal_rejected():
    with pytest.raises(ValidationError):
        MortgageInput(principal=-1000, annual_rate=0.06,
term_years=30)


def test_zero_principal_rejected():
    with pytest.raises(ValidationError):
        MortgageInput(principal=0, annual_rate=0.06,
term_years=30)


def test_zero_rate_accepted():
    """Zero interest is valid (see docs/derivation.md), not an
    error."""
    result = MortgageInput(principal=12_000, annual_rate=0.0,
term_years=1)
    assert result.annual_rate == 0.0


def test_negative_rate_rejected():
    with pytest.raises(ValidationError):
        MortgageInput(principal=200_000, annual_rate=-0.01,
term_years=30)


def test_rate_above_one_rejected():
    with pytest.raises(ValidationError):
        MortgageInput(principal=200_000, annual_rate=1.5,
term_years=30)


def test_zero_term_rejected():
    with pytest.raises(ValidationError):
        MortgageInput(principal=200_000, annual_rate=0.06,
term_years=0)


def test_unexpected_field_rejected():
    with pytest.raises(ValidationError):
```

```
        MortgageInput(
            principal=200_000,
            annual_rate=0.06,
            term_years=30,
            extra_field="test",
        )
```

Run it:

```
pytest tests/test_validation.py -v
```

One collection error, the same shape as Chapter 5.8.2's — `validation.py` genuinely doesn't exist now, so pytest can't import the file to reach any of these eight tests. That's real red this time, not something to take on faith.

7.6.3 Prompting Pi

```
pi "Implement MortgageInput in \
    src/mortgage_calculator_book/validation.py as a \
    Pydantic model to make the tests in \
    tests/test_validation.py pass. Read SPEC.md first \
    for what counts as valid input."
```

Once Pi's proposal is applied, confirm it actually earns green before reviewing further:

```
pytest tests/test_validation.py -v
```

7.6.4 Reviewing the Proposal, and Comparing Notes

Check specifically that the boundaries match SPEC.md and Chapter 7.3 exactly, not just "look reasonable" — a proposal that rejects `annual_rate = 0` would pass a casual read but silently break Chapter 6.6.1's zero-interest case the moment this validation layer gets wired in front of it.

Worth doing once, now that two independent versions exist: compare Pi's proposal against the one you wrote by hand in 7.5, still sitting in git history.

```
git log --oneline -- src/mortgage_calculator_book/validation.py
```

This shows every commit that touched this file, most recent first — the hand-written version is the one from 7.5.3, right before the `git rm` commit from 7.6.1. Grab its hash and look at it:

```
git show <hash>:src/mortgage_calculator_book/validation.py
```

Reading two independently correct implementations of the same four constraints side by side is a genuinely useful exercise on its own: where do they differ in approach, and does either version's error messages actually read better than the other's?

Once you're satisfied, commit Pi's version the normal way:

```
git add src/mortgage_calculator_book/validation.py
tests/test_validation.py
git commit -m "Rebuild MortgageInput via agent-assisted TDD"
git push
```

7.7 Wiring Validation to the Core

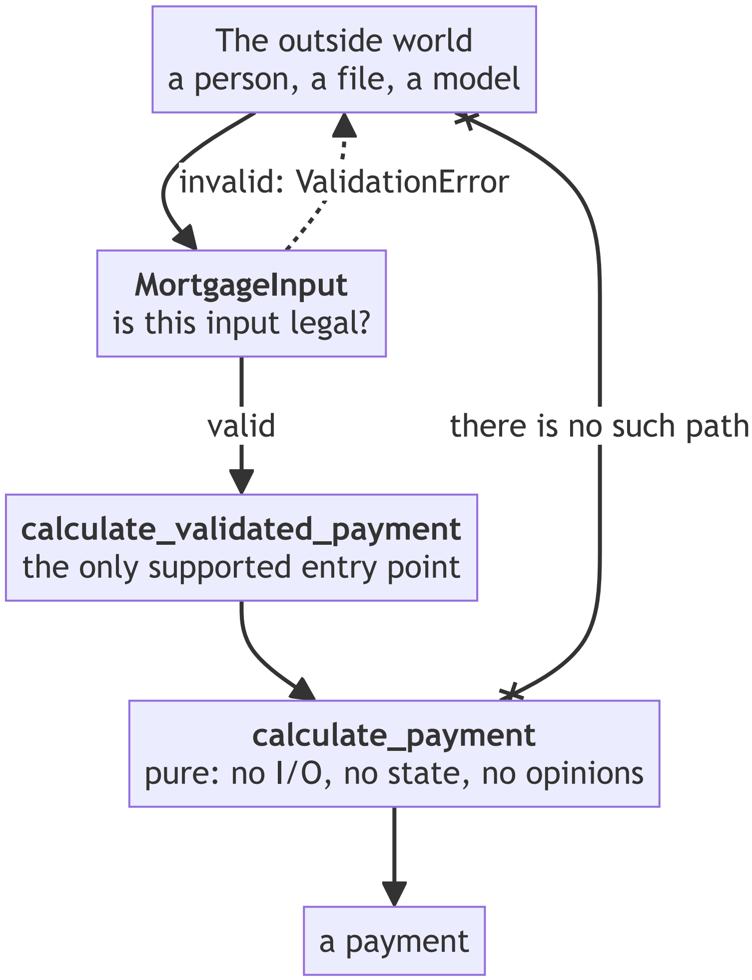
7.7.1 The Only Path In

Add one more function to `validation.py`, or to a small new module, that connects the two:

```
from mortgage_calculator_book.core import calculate_payment
from mortgage_calculator_book.validation import MortgageInput


def calculate_validated_payment(data: MortgageInput) -> float:
    """The only supported entry point: validated input in, a
    payment out."""
    return calculate_payment(
        principal=data.principal,
        annual_rate=data.annual_rate,
        term_years=data.term_years,
        payments_per_year=data.payments_per_year,
    )
```

7.7.2 Confirming the Boundary

That one function completes a shape this book keeps returning to:



The crossed-out arrow down the side is the actual rule: there is no path from the outside world to the core that skips validation. Everything Chapters 8 through 11 add gets added at the *top* — a CLI, a window, a language model, each one a new arrow into `MortgageInput` — and nothing below it changes again for the rest of the book. Keep this picture; the next four chapters are all footnotes to it.

`calculate_payment` still never sees raw, unvalidated input — it only ever receives values that have already passed through `MortgageInput`. And `MortgageInput` never performs any mortgage math itself — it only validates. Each piece does exactly one job, which is the same separation-of-concerns idea from Chapter 6.2, now applied one layer further out.

7.7.3 Running Everything

```
pytest -v
```

Chapter 6's three tests, plus this chapter's eight, all green.

7.8 Refactor and Ruff Pass

```
ruff check . && ruff format .
```

Same habit as every prior chapter — nothing new here, which is itself a small sign the habit has taken hold.

7.9 Checkpoint

Before moving to Chapter 8, this should all be true:

- ☐ Pydantic is installed as a runtime dependency
- ☐ `MortgageInput` enforces every constraint from SPEC.md: positive principal, rate in `[0, 1)`, positive term, positive payment frequency
- ☐ `test_zero_rate_accepted` passes — confirming validation didn't accidentally break Chapter 6's zero-interest case
- ☐ `calculate_validated_payment` is the only path from raw input to a payment result
- ☐ The full test suite (11 tests: 3 from Chapter 6, 8 from this chapter) is green
- ☐ Everything is committed

What's next: Chapter 8 gives this validated core its first real user-facing surface — a command-line interface a person can actually run.

Chapter 8 — Command Line Interface

Contents

8.1 Why This Chapter Exists

Everything so far has been invisible to anyone who isn't reading code — the core library, the validation layer, all of it only reachable through a test file. This chapter builds the calculator's first real, user-facing surface: a command a person can actually type and get an answer from.

By the end, you'll have a working CLI with both human-readable and JSON output, a `.env` pattern in place for a secret you don't need yet, and a README that describes all of it.

8.2 What the CLI Needs to Do

8.2.1 Revisiting the Spec

`SPEC.md`'s inputs — `principal`, `annual_rate`, `term_years`, `payments_per_year` — all need to become command-line flags. Its one output — `payment` — needs to be printed somewhere a user will see it. This is the first chapter where every one of the spec's inputs and outputs has to actually be exposed, rather than just referenced inside a test.

8.2.2 Sketching the Shape First

Before writing code:

```
mortgage-calculator-book \
    --principal 200000 --annual-rate 0.06 --term-years 30
# Fixed periodic payment: $1,199.10
```

And with an invalid input:

```
mortgage-calculator-book \
    --principal -1000 --annual-rate 0.06 --term-years 30
# Error: Value error, principal must be positive
```

Having this shape settled before writing `argparse` code turns the implementation into a matter of matching a known target, the same discipline Chapter 5 established for the core library.

8.3 argparse Basics

8.3.1 Why argparse

`argparse` ships with Python — no new dependency, no version to track, and it's more than capable of everything this project needs. Typer and similar libraries offer a nicer developer experience at the cost of an added dependency; for a first CLI, that trade isn't worth making yet.

8.3.2 Arguments, Types, Defaults, Help Text

```
import argparse

parser = argparse.ArgumentParser(
    description="Calculate a fixed mortgage payment."
)
parser.add_argument(
    "--principal", type=float, required=True,
    help="Loan amount, in dollars",
)
parser.add_argument(
    "--annual-rate", type=float, required=True,
    help="Annual rate, e.g. 0.06 for 6%%",
)
```

`type=float` means argparse converts the string a user typed before your code ever sees it — `"200000"` arrives as `200000.0`, not a string you'd have to convert yourself. `required=True` means argparse handles the "you forgot an argument" error entirely on its own, with no code from you.

8.3.3 A Minimal Working Parser

In `src/mortgage_calculator_book/cli.py`:

```
import argparse


def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        description=(
            "Calculate the fixed periodic payment for a "
            "fixed-rate mortgage."
        )
    )
    parser.add_argument(
        "--principal", type=float, required=True,
        help="Loan amount, in dollars",
    )
    parser.add_argument(
        "--annual-rate", type=float, required=True,
        help="Annual interest rate, e.g. 0.06 for 6%%",
    )
    parser.add_argument(
        "--term-years", type=int, required=True,
        help="Loan term, in years",
    )
    parser.add_argument(
```

```
        "--payments-per-year", type=int, default=12,
        help="Payments per year (default: 12)",
    )
    parser.add_argument(
        "--format", choices=["text", "json"], default="text",
        help="Output format",
    )
    return parser
```

Notice `6%%`, not `6%`, in the annual-rate help text — that's not a typo. `argparse` runs every `help=` string through `%`-style formatting when it renders `--help`, so a bare `%` looks like the start of a format specifier and crashes with `ValueError: incomplete format` the moment someone actually runs `--help`, not when you define the parser. `%%` is how you write a literal percent sign in that formatting style; `argparse` collapses it back down to a single `%` in what it actually prints. Worth testing directly rather than trusting this description — `main` doesn't exist until 8.4.1, so this needs the REPL technique from 7.4.3 rather than an actual command:

```
uv run python
```

```
from mortgage_calculator_book.cli import build_parser
build_parser().parse_args(["--help"])
```

This prints the full help text and then raises `SystemExit` — that's normal, expected `argparse` behavior for `--help`, not a bug. Confirm the annual-rate line shows a single `%`, not two. If you want to see the actual crash first, temporarily change `6%%` back to `6%`, rerun the two lines above, and watch it raise `ValueError: incomplete format` instead — then put the `%%` back.

8.3.4 Aside: Typer

Typer builds on Python type hints to generate a CLI with less boilerplate than `argparse`, and produces friendlier help text by default. It's a reasonable choice for a larger project. This book sticks with `argparse` specifically to avoid a dependency the project doesn't strictly need — worth knowing Typer exists for whenever a future project's CLI grows complex enough to justify it.

8.4 Wiring the CLI to Validation and the Core

8.4.1 The Full Pipeline, Assembled

Same file, `src/mortgage_calculator_book/cli.py`, building on the `build_parser` function from 8.3.3 rather than replacing it. First, add these imports alongside the existing `import argparse` at the top of the file:

```
import sys
```

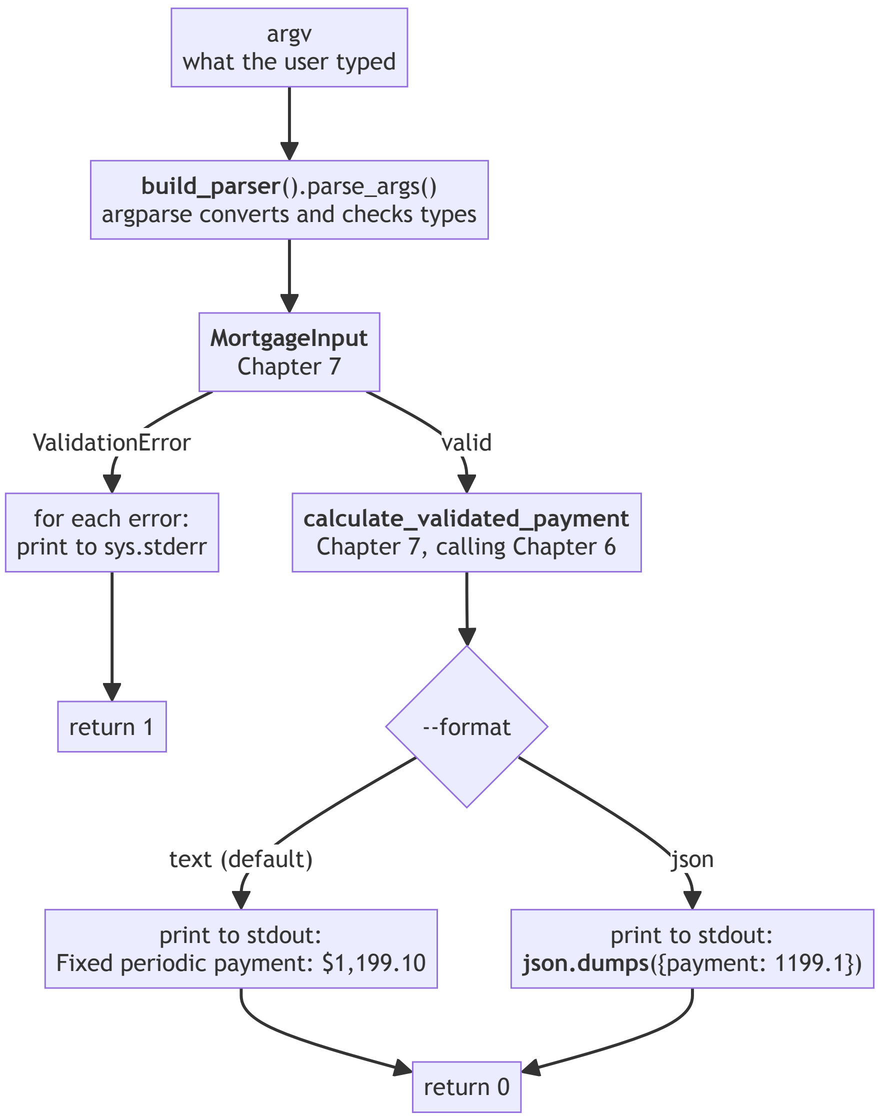
```
from pydantic import ValidationError

from mortgage_calculator_book.validation import (
    MortgageInput,
    calculate_validated_payment,
)
```

Then add `main` below `build_parser`, calling it directly:

```
def main(argv: list[str] | None = None) -> int:
    parser = build_parser()
    args = parser.parse_args(argv)

    try:
        data = MortgageInput(
            principal=args.principal,
            annual_rate=args.annual_rate,
            term_years=args.term_years,
            payments_per_year=args.payments_per_year,
        )
    except ValidationError as exc:
        for error in exc.errors():
            print(f"Error: {error['msg']}", file=sys.stderr)
        return 1

    payment = calculate_validated_payment(data)
    print(f"Fixed periodic payment: ${payment:,.2f}")
    return 0
```

At this point `cli.py` has two imports blocks worth of content (`argparse` from 8.3.3, plus the three above) and two functions (`build_parser`, then `main`) — nothing from 8.3.3 gets removed or replaced.

Parsed arguments flow into `MortgageInput` (Chapter 7), which flows into `calculate_validated_payment` (also Chapter 7, which itself calls Chapter 6's `calculate_payment`) — three chapters of work, connected for the first time.



Two details in that picture are worth more attention than they usually get, because both are things a test has to check deliberately rather than notice by accident. First, the two exits carry different codes — `1` on failure, `0` on success — which is how any script calling this command knows whether to trust the output. Second, the failure path writes to `stderr` while both success paths write to `stdout`; they are genuinely different streams, and getting them backwards produces a CLI that looks fine to a human and breaks every program that tries to read it. 8.6.2's `test_invalid_input_returns_error` checks `captured.err` specifically for exactly this reason.

8.4.2 Handling Validation Failure Gracefully

Notice what the `except ValidationError` block does *not* do: it doesn't let Pydantic's default error formatting — which is detailed, but written for developers, not end users — reach the terminal directly. It extracts just the message from each error and prints it the way a person asked to fix their input actually wants to read it.

8.4.3 Wiring Up the Command, and Trying It

Chapter 0.7.2 mentioned a `[project.scripts]` entry in `pyproject.toml`, pointing at a placeholder `main` that didn't exist yet. It does now — point the entry at it:

```
vi pyproject.toml
```

```
[project.scripts]
mortgage-calculator-book = "mortgage_calculator_book.cli:main"
```

(It was `mortgage_calculator_book:main` — the package's top-level `__init__.py`, which has no `main` function. Now it points at the one you actually built, in `cli.py`.) Pick up the change:

```
uv sync
```

Run it for real, with the Chapter 4.5 worked example:

```
uv run mortgage-calculator-book \
    --principal 200000 --annual-rate 0.06 --term-years 30
```

```
Fixed periodic payment: $1,199.10
```

Matches every other front end this project will produce — the CLI is the first to confirm this number; Chapters 9 through 11 each confirm it again, a different way. Now try the error path from 8.4.2, as a real command instead of a description of one:

```
uv run mortgage-calculator-book \
    --principal -1000 --annual-rate 0.06 --term-years 30
```

```
Error: Value error, principal must be positive
```

That printed to `stderr`, not `stdout` — worth confirming for yourself (`... 2>/dev/null` should silence it; `... 1>/dev/null` shouldn't), since it's an easy detail to lose track of once the CLI just works and you stop looking closely.

Commit the entry-point fix on its own:

```
git add pyproject.toml uv.lock
git commit -m "Wire up the mortgage-calculator-book command"
git push
```

8.5 JSON Output

8.5.1 Why JSON, Not Just Text

Printed text is fine for a person typing a command directly. It's much less fine for another program trying to read the result — parsing "Fixed periodic payment: $1,199.10" back into a number is fragile and easy to break with the smallest formatting change. JSON output is a design choice made now, deliberately, for exactly the reuse Chapter 10 needs later: a machine-readable result, not just a human-readable one.

8.5.2 A `--format` Flag

Already present in the parser from 8.3.3 (`--format`, defaulting to `"text"`). Extend `main` to branch on it:

```python
import json


def main(argv: list[str] | None = None) -> int:
    parser = build_parser()
    args = parser.parse_args(argv)

    try:
        data = MortgageInput(
            principal=args.principal,
            annual_rate=args.annual_rate,
            term_years=args.term_years,
            payments_per_year=args.payments_per_year,
        )
    except ValidationError as exc:
        for error in exc.errors():
            print(f"Error: {error['msg']}", file=sys.stderr)
        return 1

    payment = calculate_validated_payment(data)

    if args.format == "json":
        print(json.dumps({"payment": round(payment, 2)}))
    else:
        print(f"Fixed periodic payment: ${payment:,.2f}")

    return 0
```

8.5.3 Serializing with the Standard Library

`json.dumps` is all this needs — no third-party JSON library required for output this simple. `round(payment, 2)` is the first place in this project payment gets rounded for display, consistent with Chapter 6.8.3's note that the core itself stays unrounded and full-precision.

8.5.4 Forward-Note

The shape `{"payment": 1199.10}` isn't arbitrary — it's the same shape Chapter 10's tool interface will use as its output schema, so a language model calling the calculator later gets results structured the same way a script parsing this CLI's JSON output would.

`cli.py` is feature-complete for this chapter now — commit it, since nothing has captured it yet:

```
git add src/mortgage_calculator_book/cli.py
git commit -m "Add CLI: argument parsing, validation wiring, JSON
output"
git push
```

8.6 Agent-Assisted Build, With Tests First

Sections 8.3 through 8.5 had you type `build_parser` and `main` by hand, growing the file a piece at a time, before running a single test against any of it. Useful for seeing each piece land on its own, but — same issue as Chapter 7.6 — not how this book has actually been building things since Chapter 5: tests first, then an implementation that has to earn a passing result. This section does it that way for real, which means setting aside the file you just wrote and committed.

8.6.1 Setting Aside What You Just Built

Because 8.5.4 committed it, deleting it now doesn't lose anything — it's in git history if you want it later, including for a comparison at the end of this section:

```
git rm src/mortgage_calculator_book/cli.py
git commit -m "Remove hand-written cli.py to redo via TDD"
```

8.6.2 CLI-Level Tests

In `tests/test_cli.py`:

```
import json

import pytest

from mortgage_calculator_book.cli import main
```

```python
def test_text_output(capsys):
    exit_code = main(
        [
            "--principal", "200000",
            "--annual-rate", "0.06",
            "--term-years", "30",
        ]
    )
    captured = capsys.readouterr()
    assert exit_code == 0
    assert "1,199.10" in captured.out


def test_json_output(capsys):
    exit_code = main(
        [
            "--principal", "200000",
            "--annual-rate", "0.06",
            "--term-years", "30",
            "--format", "json",
        ]
    )
    captured = capsys.readouterr()
    assert exit_code == 0
    data = json.loads(captured.out)
    assert data["payment"] == pytest.approx(1199.10, abs=0.01)


def test_invalid_input_returns_error(capsys):
    exit_code = main(
        [
            "--principal", "-1000",
            "--annual-rate", "0.06",
            "--term-years", "30",
        ]
    )
    captured = capsys.readouterr()
    assert exit_code == 1
    assert "Error" in captured.err
```

`capsys` is a built-in pytest fixture — following the same fixture mechanism from Chapter 5.4, just one pytest already provides — that captures anything printed during a test, so you can assert on it without the output actually appearing in your terminal.

Run it:

```
pytest tests/test_cli.py -v
```

One collection error, same shape as Chapters 5.8.2 and 7.6.2's — `cli.py` genuinely doesn't exist right now, so pytest can't import it to reach any of these three tests.

8.6.3 Prompting Pi

```
pi "Implement build_parser and main in \
    src/mortgage_calculator_book/cli.py to make the \
    tests in tests/test_cli.py pass, wiring together \
    MortgageInput and calculate_validated_payment from \
    validation.py."
```

Once Pi's proposal is applied, confirm it actually earns green:

```
pytest tests/test_cli.py -v
```

8.6.4 Reviewing the Diff, and Comparing Notes

Same habit as every chapter since 1.6 — with one thing specific to this chapter worth checking closely: does the error-handling path actually write to `stderr`, not `stdout`? It's an easy detail for an agent (or a human) to get backwards, and `test_invalid_input_returns_error` above only catches it because it checks `captured.err` specifically rather than just checking that *some* output happened.

Worth doing once more, the same way as 7.6.4: compare Pi's version against the one you wrote by hand across 8.3 through 8.5, still in git history.

```
git log --oneline -- src/mortgage_calculator_book/cli.py
```

This shows every commit that touched this file — the hand-written version is the one from 8.5.4, right before the `git rm` commit from 8.6.1. Grab its hash and look at it:

```
git show <hash>:src/mortgage_calculator_book/cli.py
```

Once you're satisfied, commit Pi's version:

```
git add src/mortgage_calculator_book/cli.py tests/test_cli.py
git commit -m "Rebuild cli.py via agent-assisted TDD"
git push
```

8.7 Environment Variables and `.env`

8.7.1 Why Now, Not Chapter 11

Nothing in this chapter needs a secret. This section exists here anyway, deliberately early, so that the pattern is already familiar and already trusted by the time Chapter 11 actually needs it for a real API key — rather than introducing “environment variables” and “never commit a secret” for the first time in the same chapter you’re handling money-costing API calls.

8.7.2 python-dotenv

```
uv add python-dotenv
```

Create `src/mortgage_calculator_book/config.py`:

```python
import os

from dotenv import load_dotenv

load_dotenv()

OPENROUTER_API_KEY = os.getenv("OPENROUTER_API_KEY")
```

`load_dotenv()` reads a `.env` file in your project root (if one exists) and makes its contents available via `os.getenv`. Nothing in this book’s code reads `OPENROUTER_API_KEY` until Chapter 11 — this module exists now purely to establish the pattern.

8.7.3 Never Commit Your Secrets

Create `.env.example` (committed, safe — no real values):

```
vi .env.example
```

```
OPENROUTER_API_KEY=your-key-here
```

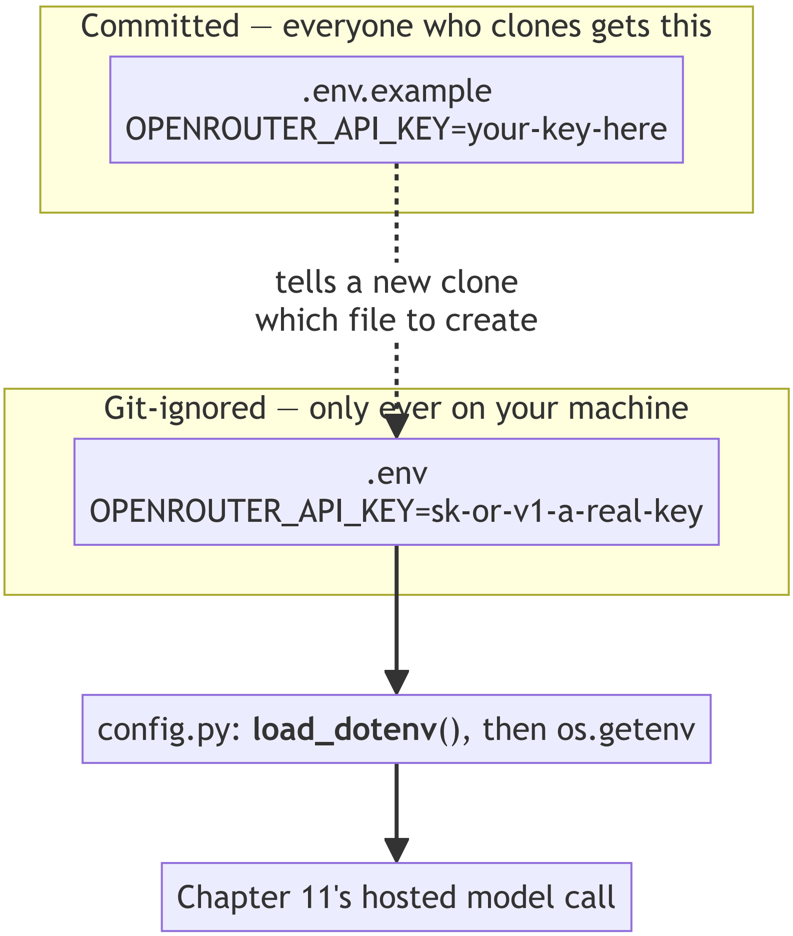
Then add one more line to `.gitignore` — the one 0.7.3 created back in Chapter 0, not a new file:

```
vi .gitignore
```

```
# Byte-compiled / cached Python files
__pycache__/
*.pyc
*.pyo

# Test and linter caches
.pytest_cache/
.ruff_cache/

# Virtual environments
.venv/
.env
```

Only that last line, `.env`, is actually new here — everything above it should already be sitting in the file from 0.7.3; this just confirms what you're adding to, rather than asking you to retype the whole thing from memory.

The rule this section wants you to leave with: `.env.example` is checked into git so anyone cloning the project knows what variables they need; `.env` itself, containing your actual key once you have one, never is.



Two files with nearly the same name and opposite rules. The committed one carries the *shape* of the secret and none of its value; the ignored one carries the value and is never seen by anyone else. Everything downstream of them reads through `config.py` and never learns which file the value came from, which is what lets Chapter 11 use a real key without a single line of code changing. Confirm `.env` really is ignored before you ever put a real key in it:

```
git check-ignore .env
```

If this prints `.env`, git will refuse to accidentally stage it — a small, worthwhile check before it matters.

8.7.4 Forward-Note

Chapter 11 is where `OPENROUTER_API_KEY` actually gets used, wiring this project to a hosted language model. Everything here is preparation, not the payoff.

8.8 Writing the README

8.8.1 A Little More Markdown

You've already used headers and lists in `SPEC.md` (Chapter 2). A README additionally benefits from **code blocks** — text wrapped in triple backticks, as you've seen throughout this book's own command examples — and **links**, written as `[text](url)`. That's genuinely enough Markdown to write a good README; this isn't a full syntax reference.

8.8.2 What Belongs in It

- What the project is (a sentence or two — SPEC.md's opening paragraph is a good starting point)
- How to install and run it
- Example usage, with real output

8.8.3 A Draft

```
# Mortgage Calculator

Calculates the fixed periodic payment for a fixed-rate mortgage.
See `SPEC.md` for the full specification.

## Setup

    uv sync

## Usage

    uv run mortgage-calculator-book \
        --principal 200000 --annual-rate 0.06 --term-years 30
    # Fixed periodic payment: $1,199.10

    uv run mortgage-calculator-book \
        --principal 200000 --annual-rate 0.06 --term-years 30
--format json
    # {"payment": 1199.1}
```

Write this now, not after the project is "finished" — there's no such moment in this book, and a README that only gets written at the end tends not to get written at all.

8.8.4 Letting Pi Draft It Instead

If you'd rather not write the README by hand, Pi can draft one from the project itself rather than from a blank page — point it at what actually exists, not just at the idea of a README:

```
pi "Draft README.md for this project. Read SPEC.md, \
    pyproject.toml, and src/mortgage_calculator_book/cli.py \
    first, and base the usage examples on the actual CLI \
    flags and command name you find there rather than \
    guessing at them."
```

Review it the same way as every other agent proposal, with one thing specific to documentation worth checking closely: does every command in it actually work, copy-pasted exactly as written? A README with a subtly wrong flag name or an invented example is worse than no README at all — it actively misleads the next person who trusts it, quite possibly you, in three weeks. Run each example yourself before accepting the diff. This is also where Pi is likeliest to invent a badge, a version number, or an installation step this project doesn't actually have; trim anything that describes a project slightly different from the one that exists.

> **Process concept: documentation as a living artifact.** This is the same spirit as `SPEC.md` from Chapter 2, aimed at a different reader: the spec is written for a collaborator (including Pi) who needs to know what the system should do; the README is written for a new user who just needs to know how to run it. Both get revised as the project changes — this README will need another pass once Chapter 9 adds a second way to run the calculator.

8.9 Refactor and Ruff Pass

```
ruff check . && ruff format .
```

Same habit as every prior chapter — nothing new here, which is itself a small sign the habit has taken hold.

Commit the README too — it's been sitting uncommitted since 8.8, the last real artifact this chapter produced:

```
git add README.md
git commit -m "Add README"
git push
```

8.10 Checkpoint

Before moving to Chapter 9, this should all be true:

- ☐ `pyproject.toml`'s `[project.scripts]` entry points at `mortgage_calculator_book.cli:main`, not the original placeholder
- ☐ `mortgage-calculator-book --principal ... --annual-rate ... --term-years ...` prints a correct, human-readable result
- ☐ `--format json` prints correctly structured JSON
- ☐ Invalid input produces a clear error message on `stderr` and a non-zero exit code
- ☐ `.env.example` is committed; `.env` is git-ignored and confirmed via `git check-ignore .env`
- ☐ `README.md` describes setup and usage with real, working examples, and is committed
- ☐ All CLI tests pass alongside every earlier chapter's tests
- ☐ `ruff check .` and `ruff format .` both pass

What's next: Chapter 9 builds a second front end — a graphical interface — calling this exact same validated core, proving the separation of concerns from Chapter 6 actually pays off.

Chapter 9 — Calculator UI

Contents

9.1 Why This Chapter Exists

Chapter 8 gave the calculator its first user-facing surface. This chapter gives it a second, entirely different one — a graphical desktop window — calling the exact same validation and core logic underneath. If Chapter 6's insistence on a pure core and Chapter 7's separate validation layer were worth the discipline, this chapter is where that discipline pays for itself directly: almost none of the code you write here will touch mortgage math at all.

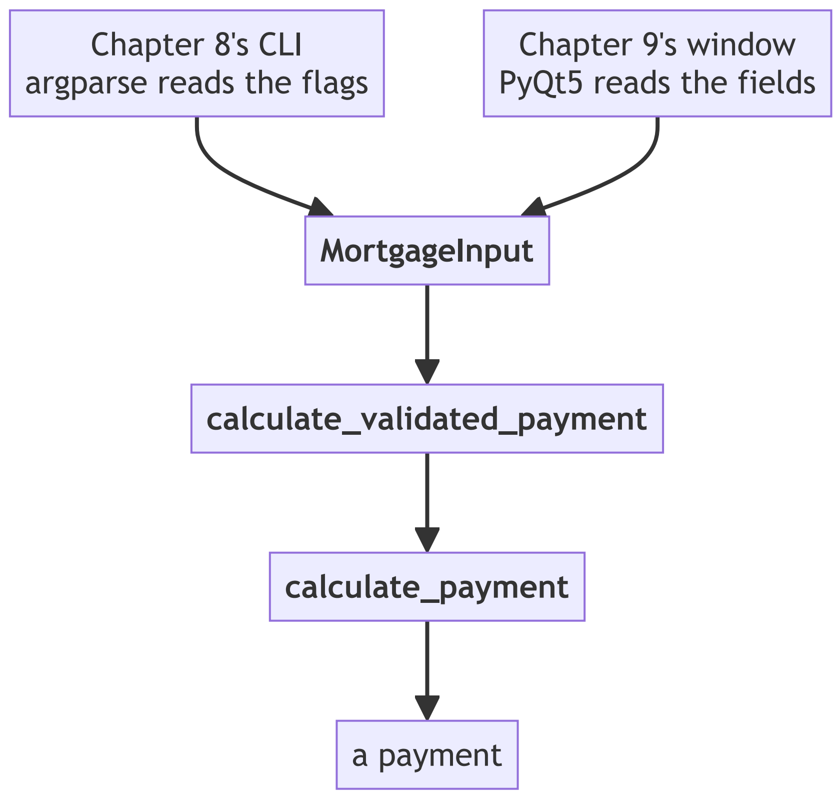
By the end, you'll have a working PyQt5 window that takes the same four inputs as the CLI and produces the same result, with no duplicated calculation logic anywhere.

9.2 Two Front Ends, One Core

9.2.1 The Rule for This Chapter

Chapter 6.2 established that the core has no I/O. Chapter 7.7 established that `MortgageInput` and `calculate_validated_payment` are the one supported path into it. Nothing in this chapter changes either of those. This chapter only adds a new way to *call* that existing path — not a new way to compute anything.

Chapter 7.7.1's diagram, with exactly one thing added:



The new box is the one on the right of the top row. Everything below `MortgageInput` is character-for-character the same code Chapter 8 was already calling — not a parallel implementation, not a shared helper extracted after the fact, just the same functions called from somewhere new. If this chapter ends with anything below that line having changed, something has gone wrong.

9.2.2 What's Allowed to Change, and What Isn't

Allowed: how input is collected, how output is displayed, anything about layout, styling, and interaction. Not allowed: reimplementing any part of the payment calculation, or duplicating validation logic that already exists in `MortgageInput`. If you notice yourself writing an `if` statement that checks whether a number is positive, that's a sign you've drifted into re-doing Chapter 7's job instead of reusing it.

9.3 PyQt5 Basics

9.3.1 Installing

```
uv add pyqt5
```

9.3.2 The Smallest Possible Window

Try this as a real, throwaway script — a GUI genuinely needs to be seen, not just read about. Create it:

```
vi scratch_window.py
```

```
import sys

from PyQt5.QtWidgets import QApplication, QWidget

app = QApplication(sys.argv)
window = QWidget()
window.setWindowTitle("Hello, PyQt5")
window.show()
sys.exit(app.exec_())
```

Run it:

```
uv run python scratch_window.py
```

A small, blank window titled "Hello, PyQt5" should appear. Nothing in it yet — that's expected; an empty `QWidget` is just a blank window, which is exactly what this checks: that PyQt5 is installed correctly and a window can actually open on your machine, before building anything real on top of it. Close the window the normal way for your platform — the script exits on its own once you do, since `app.exec_()` was waiting on exactly that.

`QApplication` manages the application as a whole — there's exactly one per program. `QWidget` is the base building block for anything visible; an empty one, as above, is just a blank window. `.show()` makes it visible, and `app.exec_()` starts the **event loop** — the process that waits for and responds to user interaction (clicks, typing, resizing) until the window is closed.

Delete the scratch file once you've seen it work — not part of the real project:

```
rm scratch_window.py
```

9.3.3 Core Widgets for This Project

- `QLineEdit` — a single-line text input, for the four numeric fields
- `QLabel` — non-editable text, for both field labels and the result display
- `QPushButton` — a clickable button, to trigger the calculation
- `QFormLayout` — arranges label/input pairs in neat rows, exactly the shape this calculator's input form needs

- `QVBoxLayout` — stacks widgets vertically; used here to stack the form, the button, and the result label

9.3.4 Signals and Slots

PyQt5's event handling works through **signals** (something happened — a button was clicked) connected to **slots** (a function that runs in response). `button.clicked.connect(some_function)` means: whenever this button is clicked, call `some_function`. That's the entire mechanism this project needs — PyQt5 has many more signal types, but this one covers everything the calculator does.

9.4 Designing the Calculator Window

9.4.1 Sketching the Layout

Four labeled inputs, matching `SPEC.md`'s fields exactly, a calculate button, and a result area below it:

```
Principal ($):            [____________]
Annual rate (e.g. 0.06): [____________]
Term (years):             [____________]
Payments per year:        [____________]

[ Calculate ]

Fixed periodic payment: $1,199.10
```

9.4.2 What's Worth Planning vs. What Isn't

The field order and labels above are worth deciding deliberately, since they directly mirror the spec. Exact pixel spacing, colors, and fonts are not worth planning in advance — PyQt5's defaults are perfectly usable, and this book treats visual polish as something to iterate on directly in code, not something to wireframe first.

9.5 Wiring the UI to Validation and the Core

9.5.1 The Full Window

In `src/mortgage_calculator_book/ui.py`:

```python
import sys

from PyQt5.QtWidgets import (
    QApplication,
    QFormLayout,
```

```
    QLabel,
    QLineEdit,
    QPushButton,
    QVBoxLayout,
    QWidget,
)
from pydantic import ValidationError

from mortgage_calculator_book.validation import (
    MortgageInput,
    calculate_validated_payment,
)


class MortgageCalculatorWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Mortgage Calculator")

        self.principal_input = QLineEdit()
        self.rate_input = QLineEdit()
        self.term_input = QLineEdit()
        self.frequency_input = QLineEdit("12")

        self.result_label = QLabel("")
        self.calculate_button = QPushButton("Calculate")
        self.calculate_button.clicked.connect(self.on_calculate)

        form = QFormLayout()
        form.addRow("Principal ($):", self.principal_input)
        form.addRow("Annual rate (e.g. 0.06):", self.rate_input)
        form.addRow("Term (years):", self.term_input)
        form.addRow("Payments per year:", self.frequency_input)

        layout = QVBoxLayout()
        layout.addLayout(form)
        layout.addWidget(self.calculate_button)
        layout.addWidget(self.result_label)
        self.setLayout(layout)

    def on_calculate(self) -> None:
        try:
            data = MortgageInput(
                principal=float(self.principal_input.text()),
                annual_rate=float(self.rate_input.text()),
                term_years=int(self.term_input.text()),
```

```
payments_per_year=int(self.frequency_input.text()),
            )
        except (ValueError, ValidationError) as exc:
            self.result_label.setText(f"Error: {exc}")
            return

        payment = calculate_validated_payment(data)
        self.result_label.setText(
            f"Fixed periodic payment: ${payment:,.2f}"
        )


def main() -> None:
    app = QApplication(sys.argv)
    window = MortgageCalculatorWindow()
    window.show()
    sys.exit(app.exec_())


if __name__ == "__main__":
    main()
```

Run it now, before moving on — a GUI is worth confirming the moment it exists, not several sections later:

```
uv run python -m mortgage_calculator_book.ui
```

The window from 9.4.1's sketch should appear, for real this time: four labeled fields, a Calculate button, an empty result area below them. Don't worry about entering real numbers and checking the math yet — that's 9.8's full walkthrough, once 9.5.2's error handling and 9.7's testing have actually been covered. For now, just confirm it launches, looks right, and closes cleanly.

9.5.2 Displaying Errors in the UI

Notice `on_calculate` catches two different kinds of error: a plain `ValueError`, raised if `float(...)` or `int(...)` fails on genuinely non-numeric text (someone typing "abc" into the principal field), and `ValidationError`, raised by `MortgageInput` for numeric-but-invalid input (a negative principal, same as Chapter 8's CLI would reject). Both land in the same result label rather than crashing the application — a user typing garbage into a field should see a message, not watch the window disappear.

9.5.3 Displaying the Result

`f"${payment:,.2f}"` — the same formatting used in Chapter 8's CLI, for the same reason: consistent presentation between front ends, even though the underlying `payment` value is unrounded until this exact point (Chapter 6.8.3).

The window works end to end now — commit it:

```
git add src/mortgage_calculator_book/ui.py
git commit -m "Add calculator window wired to validation and the
core"
git push
```

9.6 Where Agent Assistance Helps, and Where It Doesn't

9.6.1 Good Delegation, and a Trickier Request Bundled With It

The widget-creation and layout code in 9.5.1 is exactly the kind of task worth handing to Pi. Start by updating 9.4.1's sketch — a second pass at a design is normal, the same way SPEC.md gets revised more than once across this book, including right here in 9.6.2 below:

```
Principal ($):            [____________]
Annual rate (e.g. 0.06): [____________]
Term (years):             [____________]
Payments per year:        [____________]

[ Calculate ]  [ Clear ]

Fixed periodic payment: $1,199.10
```

Hand the updated sketch to Pi directly as the spec, and ask for tests in the same breath:

```
pi "Add a 'Clear' button next to Calculate that resets \
    all four input fields and the result label, matching \
    the updated layout in 9.4.1. Also add tests for it in \
    tests/test_ui.py."
```

The GUI half is exactly as mechanical as it looks: run the app, click the new button, confirm all four fields and the result label actually clear. Read the diff first, as always, but there's not much to catch — button wiring like this has an obviously correct answer.

The tests half is where this exercise earns its place. Watch for which of two very different directions Pi actually takes:

- **A widget-level test** — instantiate the window, click Clear, assert the fields are now empty. This doesn't strictly need a new dependency — PyQt5 itself can drive a widget directly in an offscreen `QApplication`, no pytest-qt required — but it does need real setup: an offscreen environment variable, a session-scoped fixture, a widget built just to click one button and check four fields are empty. If the proposal reaches for that machinery, or worse, quietly adds pytest-qt anyway without needing to, that's worth noticing: meaningfully more scaffolding than a Clear button warrants is a bigger, less obvious version of the exact problem 9.6.3 is about to raise with "make this look nicer" — more than was actually asked for, dressed up as thoroughness.

- **No test at all, with a note explaining why** — because unlike parse_form_values in 9.7.1, "clear four fields" has no logic to extract. self.principal_input.clear() four times over isn't a computation with a right answer to assert on; it's direct widget manipulation, indistinguishable in a test from the implementation itself. There's nothing a unit test buys here that clicking the button and looking doesn't already give you, faster.

The second answer is the right one. If Pi proposes the first, this is a case for declining it outright rather than accepting it and cleaning it up afterward: reject the new dependency, keep the manual click-and-check, and don't write a test just because one was asked for. 9.7.2 called this "the judgment of not over-testing" in the abstract; this is what it looks like when an actual proposal tests that judgment directly — agreeing to "add tests for it" doesn't obligate you to accept tests that don't earn their place.

ui.py has real, working changes since its last commit — commit them:

```
git add src/mortgage_calculator_book/ui.py
git commit -m "Add Clear button"
git push
```

9.6.2 Writing the Interface Into SPEC.md, Then Redoing It From There

9.4.1's sketch is a real design artifact, but it lives in this book, not in the project — the same gap Chapter 4.7.3 found in the derivation, and for the same reason: Pi can only read what's actually in the repository. SPEC.md doesn't describe the GUI at all right now, or the CLI either, for that matter — nothing has ever gone back and added them. Fix both, in the spirit of Chapter 2.2.1's rule for what belongs here: *what* the interface must provide, not *how* it's built — no QPushButton, no PyQt5, just the required fields, actions, and behavior.

```
vi SPEC.md
```

```
## Interfaces
- Command-line interface (human-readable and JSON output)
- Desktop GUI:
  - Inputs: principal, annual rate, term (years), payments per
year
```

```
  - Actions: Calculate (computes and displays the payment), Clear
    (resets all fields and the result)
  - Invalid input shows an error message in place, not a crash
```

Add this as its own section, near the top — right after "What it does" reads naturally. Commit it on its own:

```
git add SPEC.md
git commit -m "Document CLI and GUI interfaces in SPEC.md"
git push
```

Now delete `ui.py` and rebuild it from SPEC.md alone, the same shape as Chapters 7.6 and 8.6 — safe, since it's committed:

```
git rm src/mortgage_calculator_book/ui.py
git commit -m "Remove hand-written ui.py to redo from SPEC.md"
```

```
pi "Implement the GUI described in SPEC.md's Interfaces \
    section, in src/mortgage_calculator_book/ui.py, \
    wiring it to MortgageInput and \
    calculate_validated_payment from validation.py."
```

There's no test suite driving this one red-then-green the way 7.6 and 8.6 were — 9.7.1 already established why a GUI like this isn't unit-tested directly. Verify by hand instead, the same way 9.5.1 and 9.6.1 already did: run it, enter the Chapter 4.5 worked example, confirm `$1,199.10`, click Clear, confirm everything resets, try invalid input, confirm it shows an error rather than crashing.

```
uv run python -m mortgage_calculator_book.ui
```

Worth comparing against what you had before, the same way as 7.6.4 and 8.6.4:

```
git log --oneline -- src/mortgage_calculator_book/ui.py
```

Grab the hash for "Add Clear button" — the most recent commit to this file before the delete — and look at it:

```
git show <hash>:src/mortgage_calculator_book/ui.py
```

A reasonable rebuild from this spec should land on something structurally close to what you had by hand — reading the fields, building a `MortgageInput`, catching `ValidationError`, displaying the result — since that's the only obvious way to satisfy what SPEC.md now asks for. If Pi's version differs in some real way rather than just naming or ordering, that's worth understanding before moving on; section 9.7's extraction below assumes a shape like the original, and if what you actually have looks meaningfully different, adapt it rather than forcing a mismatch.

Once you're satisfied:

```
git add src/mortgage_calculator_book/ui.py
git commit -m "Rebuild ui.py from SPEC.md's Interfaces section"
git push
```

9.6.3 Poor Delegation: Layout and Visual Judgment

Asking Pi to "make this look nicer" invites a much harder review problem: "nicer" isn't a test you can run, and accepting a layout change sight-unseen means trusting an aesthetic judgment you haven't actually checked. Reviewing a UI diff correctly here means *running the application and looking at it* — reading the code alone won't tell you whether a change actually improved anything.

> **Process concept: "trust but verify" extended to a domain you check with your eyes.** Chapter 1.6 introduced verification as reading a diff. Most of this book's verification since then has been reading diffs and running tests. This is the first chapter where verification genuinely requires neither — it requires looking at the running application. Recognizing when a task needs that kind of check, rather than defaulting to "the tests still pass, so it's fine," is its own skill worth practicing here.

9.7 Testing UI Logic Where It's Worth It

9.7.1 What's Testable Without a Full GUI Framework

Testing PyQt5 windows directly — simulating clicks, reading rendered widget state — is possible without a new dependency: PyQt5 itself can drive a widget in an offscreen `QApplication`, no `pytest-qt` needed. But doing it properly — a session fixture, an offscreen environment variable, one test per interesting interaction — is real setup for a project this size, the same kind of scope call the Appendix makes for Docker or CI/CD: genuinely achievable, not what this book spends its pages on. Appendix A.7 sketches the technique directly, if you're curious now rather than later. What *is* cheap, regardless of that choice, is the parsing logic hiding inside `on_calculate`. Pull it out as its own function in `ui.py`:

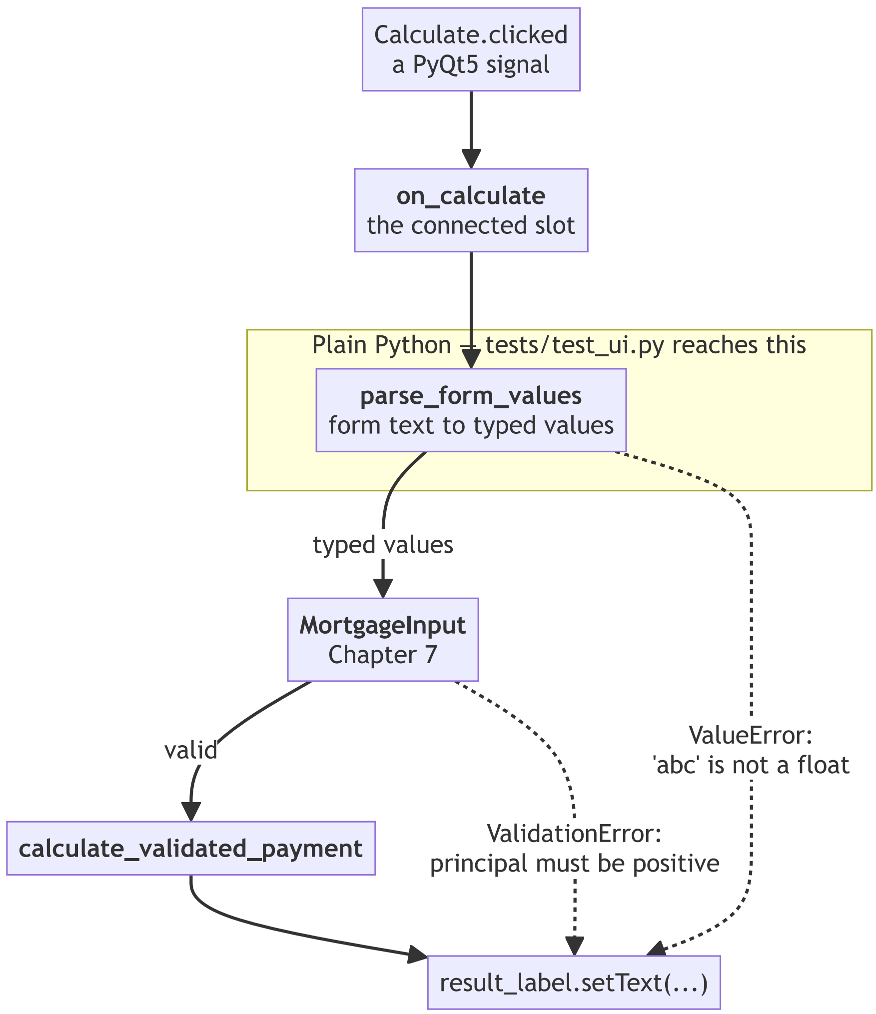
```
def parse_form_values(
    principal_text: str, rate_text: str, term_text: str,
frequency_text: str
) -> dict:
    """Convert raw form text into the types MortgageInput
    expects."""
    return {
        "principal": float(principal_text),
        "annual_rate": float(rate_text),
        "term_years": int(term_text),
        "payments_per_year": int(frequency_text),
    }
```

And update `on_calculate` to call it:

```
    def on_calculate(self) -> None:
```

```
try:
    values = parse_form_values(
        self.principal_input.text(),
        self.rate_input.text(),
        self.term_input.text(),
        self.frequency_input.text(),
    )
    data = MortgageInput(**values)
except (ValueError, ValidationError) as exc:
    self.result_label.setText(f"Error: {exc}")
    return

payment = calculate_validated_payment(data)
self.result_label.setText(
    f"Fixed periodic payment: ${payment:,.2f}"
)
```

That extraction splits the click path into two halves with very different testing stories:



Everything inside the box is ordinary Python with no window attached, which is why it can be tested for the price of an import. Everything outside it is widget behavior — a signal firing, a label's text changing — which 9.7.2 decides to check by clicking the button and looking, rather than by building the off-screen machinery Appendix A.7 describes. The diagram also shows why both error arrows land in the same place: 9.5.2's two exception types come from two different steps, but a user only ever needs one message in one spot.

Now `parse_form_values` is a plain function, testable with no `QApplication` involved at all:

```python
# tests/test_ui.py
import pytest

from mortgage_calculator_book.ui import parse_form_values


def test_parses_valid_form_values():
    result = parse_form_values("200000", "0.06", "30", "12")
    assert result == {
        "principal": 200000.0,
        "annual_rate": 0.06,
        "term_years": 30,
        "payments_per_year": 12,
    }


def test_raises_on_non_numeric_principal():
    with pytest.raises(ValueError):
        parse_form_values("not a number", "0.06", "30", "12")
```

Run it:

```
pytest tests/test_ui.py -v
```

Both pass — this file needed no GUI, no window, no `QApplication`, to genuinely verify. That's the whole point of pulling `parse_form_values` out on its own in the first place.

9.7.2 The Judgment of Not Over-Testing

Notice what's deliberately absent here: no test clicks the actual button, no test checks the actual label text on screen. That's not laziness, and it's not because it can't be done — 9.7.1 already showed it's achievable without a new dependency. It's a decision that the setup cost (an offscreen `QApplication`, a session fixture) isn't worth it for this project, given how cheaply the same thing can be checked by hand (9.6.3). Knowing where to stop testing is as much a skill as knowing where to start.

9.8 Running the App

9.8.1 A Walkthrough

```
uv run python -m mortgage_calculator_book.ui
```

Enter the Chapter 4.5 worked example — `200000`, `0.06`, `30`, `12` — click Calculate, and confirm the window shows `$1,199.10`, matching every other front end this project has produced so far.

9.8.2 Packaging, Briefly

Turning this into a double-clickable application (a `.app` on macOS, an `.exe` on Windows) is a real, separate topic — tools like PyInstaller handle it, but it's genuinely out of scope for this book. Flagged here, and covered properly in Appendix A.8, rather than left as a mystery.

9.9 Refactor and Ruff Pass

```
ruff check . && ruff format .
```

If `ruff check` reports anything, `ruff check --fix .` (3.4.3) is usually the fastest way to clear it before looking at whatever's left by hand.

`parse_form_values` and `tests/test_ui.py` from 9.7 have been sitting uncommitted since they were written — the last loose end from this chapter:

```
git add src/mortgage_calculator_book/ui.py tests/test_ui.py
git commit -m "Extract parse_form_values and test it"
git push
```

9.10 Checkpoint

Before moving to Chapter 10, this should all be true:

- ☐ The PyQt5 window opens and displays four labeled input fields, a Calculate button, and a result area
- ☐ The Clear button (9.6.1) resets all four fields and the result label, verified by clicking it — not by an added GUI-testing dependency
- ☐ `SPEC.md` has an "Interfaces" section documenting both the CLI and the GUI, committed (9.6.2)
- ☐ `ui.py` was rebuilt from that spec via Pi and re-verified by hand, not left as the original hand-written version
- ☐ Entering the Chapter 4.5 worked example produces `$1,199.10`
- ☐ Invalid input (non-numeric text, or values `MortgageInput` rejects) shows an error message instead of crashing
- ☐ `parse_form_values` exists as a standalone, tested function
- ☐ No mortgage math or validation logic is duplicated anywhere in `ui.py` — it all still lives in `core.py` and `validation.py`
- ☐ `ruff check .` and `ruff format .` both pass
- ☐ Everything from this chapter is committed and pushed — `ui.py`, `SPEC.md`, and `test_ui.py` alike, not just the pieces that felt like real commits at the time

What's next: Chapter 10 builds a third front end — not for a human this time, but for a language model, exposing the same validated core as a callable tool.

Chapter 10 — Tool Interface

Contents

10.1 Why This Chapter Exists

Chapters 8 and 9 gave the calculator two front ends for humans. This chapter builds a third — one for a language model. By the end, you'll have a callable, schema-defined tool wrapping the exact same validated core those earlier chapters used, fully tested, and verified by hand with a simulated tool call. Chapter 11 is where an actual model gets to use it; this chapter is entirely about building and proving the contract first.

10.2 What a "Tool" Means in This Context

10.2.1 The Core Idea

A "tool," in the context of language models, is nothing more mysterious than: a function description plus a schema, written in a form a model can read, that tells it what the function does, what arguments it needs, and what it returns. The model doesn't execute anything itself — it decides *when* to call the tool and *what arguments to pass*, and your code does the actual work and hands the result back. It's a very structured, very constrained API, not a new kind of programming.

10.2.2 A Third Front End, Not a New Architecture

Exactly like Chapter 9's opening argument: this chapter follows the same rule Chapters 6 and 7 established. `calculate_payment` stays pure. `MortgageInput` and `calculate_validated_payment` remain the one supported path into it. This chapter adds a way for a *model* to reach that path, the same way Chapters 8 and 9 added ways for a *human* to reach it.

10.2.3 Validation-as-Schema, Revisited

Chapter 7.2.3 flagged this moment directly: a Pydantic model is already most of the way to a tool schema. This chapter is where that foreshadowing pays off.

10.3 Reusing the Pydantic Models

10.3.1 Free JSON Schema

What is a schema, exactly? A description of the *shape* data is expected to have — which fields exist, what type each one is, which are required — written so a program can read it, not just a person. It's not the data itself, any more than a form's blank fields are the answers someone eventually writes into them; it's a specification of what valid data looks like, checkable before anything is actually filled in. **JSON Schema** just means that description is itself written as JSON — which is exactly why it's the format most LLM tool-calling interfaces expect for describing valid arguments: a model needs to know a tool's expected shape before it can call it correctly.

Pydantic models can export their structure as JSON Schema with a single method call. Try it directly, the same REPL technique as 7.4.3:

```
uv run python
```

```python
from mortgage_calculator_book.validation import MortgageInput
import json

print(json.dumps(MortgageInput.model_json_schema(), indent=2))
```

You should see something close to this:

```json
{
  "additionalProperties": false,
  "description": "Validated input for a fixed-rate mortgage
  payment calculation.",
  "properties": {
    "principal": {
      "title": "Principal",
      "type": "number"
    },
    "annual_rate": {
      "title": "Annual Rate",
      "type": "number"
    },
    "term_years": {
      "title": "Term Years",
      "type": "integer"
    },
    "payments_per_year": {
      "default": 12,
      "title": "Payments Per Year",
      "type": "integer"
    }
  },
  "required": [
    "principal",
    "annual_rate",
    "term_years"
  ],
  "title": "MortgageInput",
  "type": "object"
}
```

Exit when you're done: **Ctrl+D**.

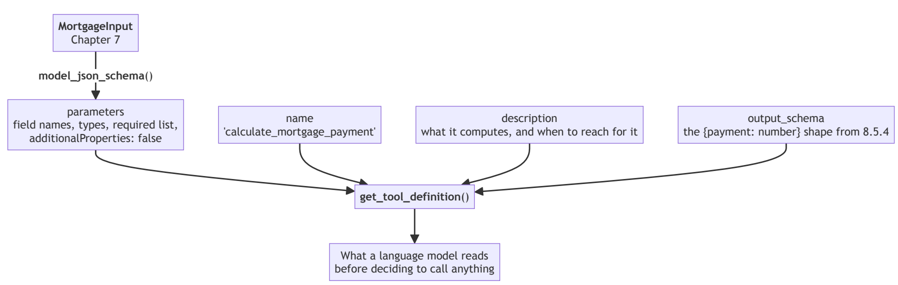
This dictionary describes every field's type, and — because your `field_validators` already enforce them — you got the *shape* of the constraints (types, required fields) for free, with no extra code. Now inspect it for what's just as tellingly *missing*: none of Chapter 7.5's actual business rules show up anywhere. `principal` just says `"type": "number"` — nothing here hints that it rejects zero or negative values, or that `annual_rate` has to stay under 1.0. The specific rules inside your validators aren't expressed in the exported schema itself, but they still run every time `MortgageInput` is constructed — which matters, because it means the tool can't be tricked into skipping validation just because the schema alone doesn't spell out every rule.

One constraint *does* show up on its own, though, and it's worth noticing why: `"additionalProperties": false` appears at the top of the printed schema, correctly advertising that this model rejects fields it doesn't recognize — because Chapter 7.5.1's `model_config = ConfigDict(extra="forbid")` makes that true at runtime, and Pydantic reflects real runtime behavior into the generated schema automatically. Fix a constraint at the model, the one place this project defines what "valid" means, and both the enforcement and the advertisement of it come free from the same line — no separate schema to keep in sync by hand.

10.3.2 What Still Needs to Be Added

A schema alone isn't a complete tool definition — a model also needs a **name** it can refer to, and a **description** written in prose that tells it what the tool is *for* and *when to use it*. Neither of those come from the Pydantic model; both are new content for this chapter.

Four parts go into the finished definition, and it's worth being clear about which of them you already have:



Only the top branch is free. `parameters` falls out of a model you already wrote and already test, which is 7.2.3's foreshadowing paying off exactly as promised. The other three boxes are new writing — and of those, `description` is the only one in this entire project whose quality is judged by a language model rather than by a test, which is why 10.4.1 treats it as prompt engineering rather than documentation.

10.4 Defining the Tool Contract

Everything in this section goes into one new file — `src/mortgage_calculator_book/tool.py`, sitting alongside `cli.py` and `ui.py` as a peer front end, not folded into either. Create it now:

```
vi src/mortgage_calculator_book/tool.py
```

10.4.1 Writing the Description

This is prompt engineering in miniature — worth treating with real care, since a vague or ambiguous description directly affects whether a model calls the tool correctly, or at all:

```
TOOL_NAME = "calculate_mortgage_payment"

TOOL_DESCRIPTION = (
    "Calculate the fixed periodic payment for a fixed-rate
    mortgage, given "
    "a principal amount, an annual interest rate, a loan term in
    years, "
    "and how many payments are made per year. Use this whenever
    the user "
    "asks about a mortgage payment amount for a fixed-rate loan."
)
```

Notice the second sentence does real work: it's not just describing what the function computes, it's telling the model *when* reaching for it is appropriate — language a schema alone can't express.

10.4.2 The Output Schema

Reusing the exact shape from Chapter 8.5.4's forward-note — `{"payment": 1199.10}` — rather than inventing a new one. Same file, appended below what's already there:

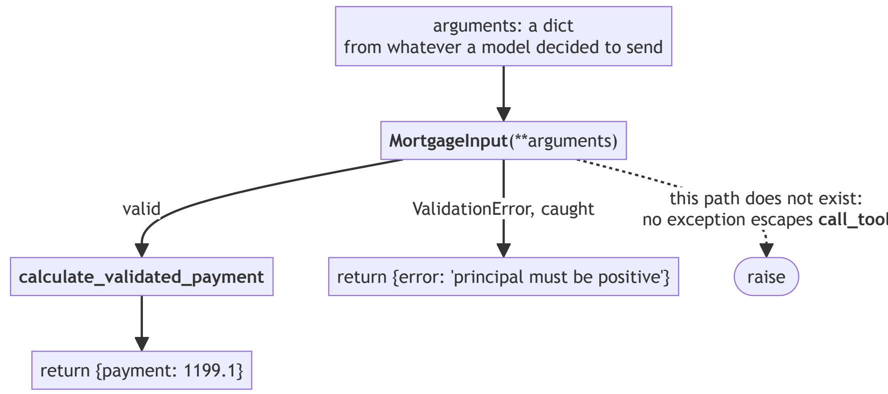
```
OUTPUT_SCHEMA = {
    "type": "object",
    "properties": {
        "payment": {
            "type": "number",
            "description": "The fixed periodic payment, in
            dollars.",
        }
    },
    "required": ["payment"],
}
```

10.5 TDD the Contract Before Wiring Anything Up

10.5.1 Tests First, No Model Involved

```
# tests/test_tool.py
import pytest

from mortgage_calculator_book.tool import call_tool,
get_tool_definition


def test_valid_call_returns_payment():
    result = call_tool(
        {
            "principal": 200000,
            "annual_rate": 0.06,
            "term_years": 30,
            "payments_per_year": 12,
        }
    )
    assert result["payment"] == pytest.approx(1199.10, abs=0.01)


def test_invalid_call_returns_error_dict_not_exception():
    result = call_tool(
        {"principal": -1000, "annual_rate": 0.06, "term_years":
30}
    )
    assert "error" in result


def test_tool_definition_has_name_and_description():
    definition = get_tool_definition()
    assert definition["name"] == "calculate_mortgage_payment"
    assert "mortgage" in definition["description"].lower()
    assert "properties" in definition["parameters"]
    assert "payment" in definition["output_schema"]["properties"]
```

Those three tests describe a function with exactly two exits and no third one:



The dotted path is the whole point of this section. Chapter 6's core and Chapter 7's validation are both allowed to raise, because a human reading a traceback can act on it. Chapter 11's model-calling loop can't — it needs something to hand *back to the model* whichever way the call went, and an exception is not that. Two predictable dictionary shapes, always.

Note the second test's name specifically: `call_tool` must return an **error dictionary**, not raise an exception — a meaningfully different contract than Chapter 6's core or Chapter 7's validation, both of which are allowed to raise. A model-calling layer in Chapter 11 needs to hand *something* back to the model on bad input; an uncaught exception isn't something a model-calling loop can pass along gracefully.

10.5.2 Confirming Errors Are Handleable

This is worth stating as its own requirement, separate from "the function shouldn't crash": the shape of an error response (`{"error": "..."}`) needs to be just as predictable as the shape of a success response, since Chapter 11's wiring will need to check for one or the other every single time.

10.5.3 Agent-Assisted Implementation

```
pi "Implement call_tool and get_tool_definition in \
    src/mortgage_calculator_book/tool.py to make the \
    tests in tests/test_tool.py pass. call_tool should \
    never raise - catch validation errors and return an \
    error dict instead."
```

Review as always — this chapter's implementation:

```
from typing import Any
```

```
from pydantic import ValidationError

from mortgage_calculator_book.validation import (
    MortgageInput,
    calculate_validated_payment,
)


def get_input_schema() -> dict[str, Any]:
    return MortgageInput.model_json_schema()


def get_tool_definition() -> dict[str, Any]:
    return {
        "name": TOOL_NAME,
        "description": TOOL_DESCRIPTION,
        "parameters": get_input_schema(),
        "output_schema": OUTPUT_SCHEMA,
    }


def call_tool(arguments: dict[str, Any]) -> dict[str, Any]:
    """Validate arguments, compute the payment, and return a
    result or an
    error dict. Never raises - a model-calling layer needs
something to
    hand back to the model either way."""
    try:
        data = MortgageInput(**arguments)
    except ValidationError as exc:
        return {"error": "; ".join(e["msg"] for e in
        exc.errors())}

    payment = calculate_validated_payment(data)
    return {"payment": round(payment, 2)}
```

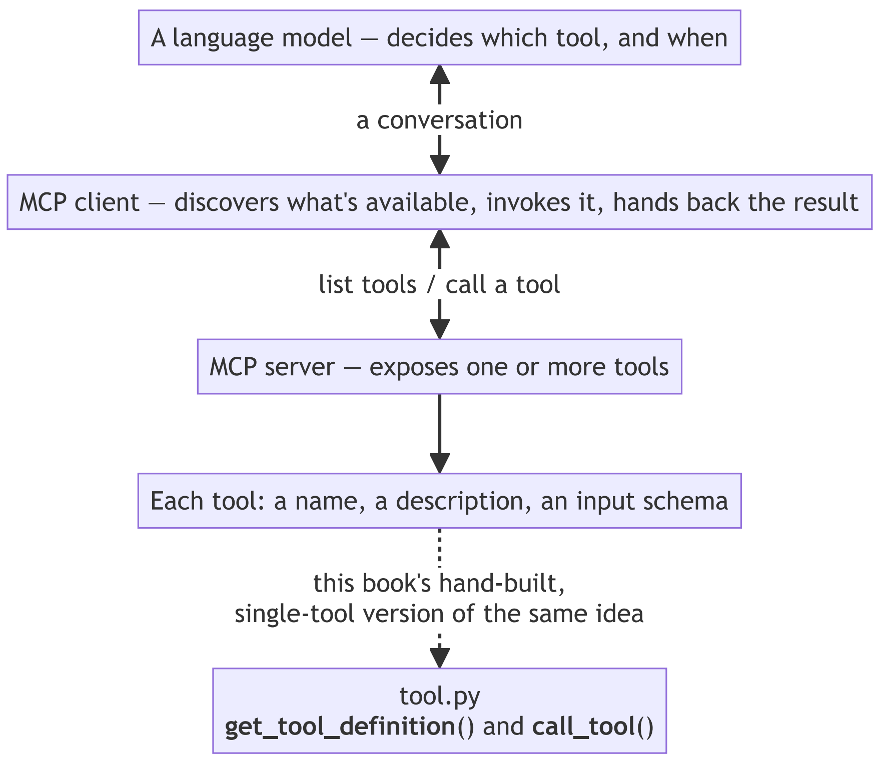
10.6 Sidebar: Model Context Protocol (MCP)

10.6.1 What You Just Built, Named

What `get_tool_definition` and `call_tool` implement by hand — a name, a description, a schema, and a function that executes against validated arguments — is a small, simplified version of a real, standardized protocol called **Model Context Protocol (MCP)**. MCP defines a common way for an application to expose tools like this one to a language model, and for a model-calling client to discover and invoke them, without every project inventing its own bespoke format.

10.6.2 The Shape of MCP, Conceptually

At a high level: an MCP **server** exposes one or more tools, each described the way `TOOL_DESCRIPTION` and `get_input_schema()` describe this one. An MCP **client** — the thing actually talking to a language model — discovers what tools are available and handles calling them when the model asks. This book doesn't implement MCP itself; building `tool.py` by hand, understanding exactly what each piece is for, is meant to make the real protocol legible later rather than an opaque black box the first time you encounter it.



Read the dotted arrow as the whole point of this sidebar. Nothing in this project speaks MCP, and nothing needs to — but the four boxes it points at are the same four you assembled by hand in 10.4, which is why the protocol should read as familiar rather than novel the first time you meet it in a real project.

10.6.3 Why This Matters

Connecting this chapter's exercise to something real, standardized, and widely used means what you've built here isn't just a toy pattern specific to this book

— it's a smaller version of an approach you're likely to run into again.

10.7 Manual Verification: Calling the Tool Like a Model Would

10.7.1 A Simulated Call

Before Chapter 11 puts a real model in the driver's seat, confirm the tool works exactly the way a model-calling layer would use it — JSON in, JSON out:

```
# scratch_verify_tool.py - not part of the test suite, just a
manual check
import json

from mortgage_calculator_book.tool import call_tool

request = json.loads(
    '{"principal": 200000, "annual_rate": 0.06, '
    '"term_years": 30, "payments_per_year": 12}'
)
response = call_tool(request)
print(json.dumps(response))
# {"payment": 1199.1}
```

```
uv run python scratch_verify_tool.py
```

10.7.2 The Last Checkpoint Before a Real Model

This confirms the whole path works end to end — JSON arguments in, a validated calculation, JSON results out — using nothing but the exact mechanism a model will use in Chapter 11. Delete `scratch_verify_tool.py` once you've run it; it was a check, not a permanent part of the project.

10.8 Refactor and Ruff Pass

```
ruff check . && ruff format .
```

10.9 Checkpoint

Before moving to Chapter 11, this should all be true:

- ☐ `get_tool_definition()` returns a name, a description, a parameters schema derived from `MortgageInput`, and the output schema from 10.4.2
- ☐ `call_tool()` returns `{"payment": ...}` on valid input and `{"error": ...}` on invalid input — and never raises
- ☐ All tests in `tests/test_tool.py` pass

- ☐ You've manually verified a JSON-in, JSON-out call against the Chapter 4.5 worked example
- ☐ You can explain, in a sentence, what MCP is and how what you built here relates to it
- ☐ `ruff check .` and `ruff format .` both pass

What's next: Chapter 11 gives a real language model — local first, then hosted — the ability to call this tool, completing the hybrid system end to end for the first time.

Chapter 11 — LLM Interface: Enter A Second Model

Contents

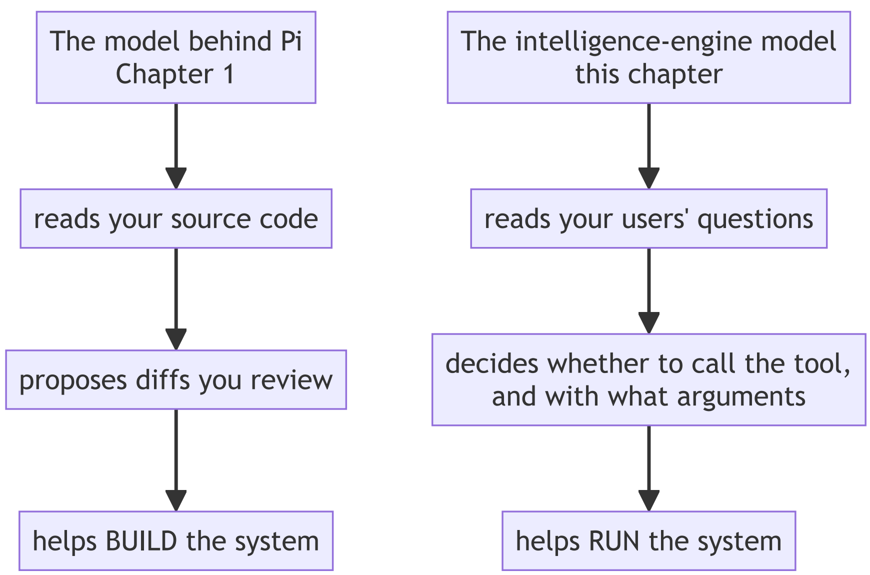
11.1 Why This Chapter Exists

Chapter 10 proved the tool works when called manually, with arguments you typed yourself. This chapter hands that same tool to a language model, and lets the model decide when and how to call it. By the end, you'll be able to ask the calculator a plain-language question — "What would my payment be on a $200,000 loan at 6% over 30 years?" — and get back a real, computed answer, through both a local model and a hosted one. This is the chapter where the hybrid system this book has been building toward finally exists, end to end.

11.2 Two Models, Two Jobs

11.2.1 Explicitly Distinguishing the Roles

Two different local language models will exist in this project by the end of this chapter, and it's worth being precise about why: the model behind Pi (Chapter 1) helped *build* the system — reading code, proposing diffs, drafting tests. The model this chapter adds helps *run* the system — reading a user's plain-language question and deciding whether and how to call the Chapter 10 tool. Same underlying technology, genuinely different jobs.



Two lanes that never touch. They're judged on different things too, which is 11.2.2's point: the top lane is judged on reasoning across several files of code, the bottom on producing well-formed tool arguments reliably. A model that's excellent at one has told you nothing about the other, which is why this chapter pulls a second model rather than reusing the one already installed.

11.2.2 Why the Same Model Isn't Necessarily Right for Both

A model chosen for coding assistance is judged on things like how well it reads and reasons about source code across multiple files. A model chosen for this chapter's job is judged on something narrower and more specific: how reliably it calls a tool with correctly formatted arguments, and how sensibly it responds once it has a result back. A model that's excellent at one of these isn't guaranteed to be excellent at the other — which is exactly why this chapter pulls a second model rather than reusing Pi's.

> **Process concept: naming the distinction once both models exist.** Chapter 1.2.3 flagged that a second model job was coming, without saying much more about it. Now that both models actually exist side by side in this project, the distinction is concrete rather than theoretical: one model's job is understanding *your code*; the other's job is understanding *your users' questions*.

11.3 Choosing a Local Model for the Intelligence Engine

11.3.1 What Matters for This Job

Raw coding ability isn't the priority here — reliable, correctly formatted tool-calling behavior is. A smaller model that reliably produces well-formed tool calls is more useful for this job than a larger, more broadly capable model that occasionally gets the arguments wrong or forgets to call the tool at all.

11.3.2 Pulling a Second Model

```
ollama pull qwen3:8b
```

Sized against the same hardware gut-check table from Chapter 1.4.3 — if your machine comfortably runs the larger model behind Pi, it will run this one without issue; if it doesn't, this is a good chance to try a smaller model specifically for this narrower job.

11.3.3 A Standalone Conversation First

Before wiring anything to the tool, talk to this model the same way you did in Chapter 1.3.3:

```
ollama run qwen3:8b
```

Ask it a mortgage-related question directly, with no tool available — the same $200,000/6%/30-year question from Chapter 4.5. One of two things happens, and it's worth being honest that the better-sounding one isn't actually the reassuring one.

Sometimes the model answers in general terms, or admits it can't compute an exact figure — useful context on its own for appreciating what the tool wiring in section 11.4 adds.

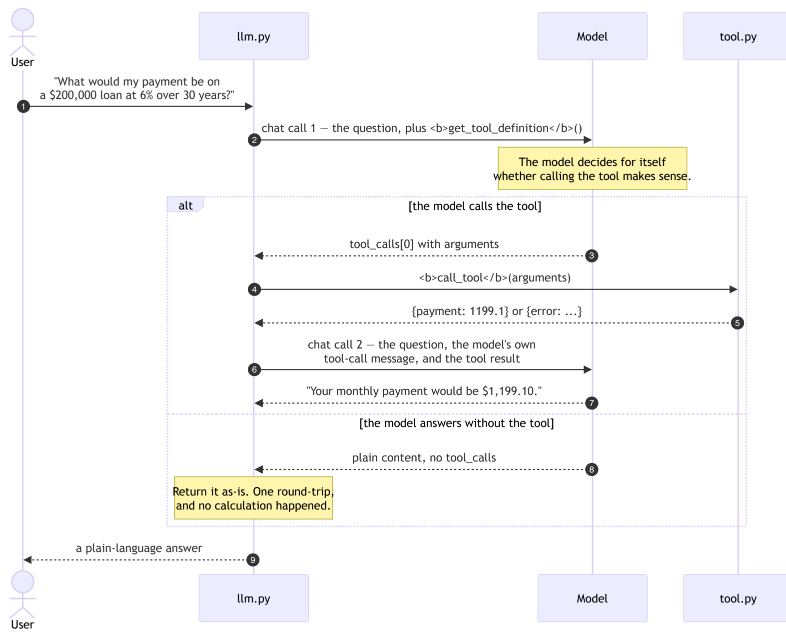
But a reasoning model like this one is just as likely to work through the whole amortization formula inside its own "thinking" output — converting the rate, computing $(1.005)^{360}$, dividing, multiplying — and land on **$1,199.10**, matching Chapter 4.5's answer exactly. If that's what you see, don't read it as evidence the tool is unnecessary. Read it the opposite way. Run it again with `--verbose` and look at what that correct answer actually cost: on ordinary consumer hardware, a real run of this exact question took **61.8 seconds** and **5,107 tokens** of reasoning to reach a number `calculate_payment` returns in about **0.3 microseconds** — not a rounding difference, but roughly 193 million times slower for the identical result. And ask yourself how you'd actually know if one of those 5,107 tokens had gotten the arithmetic slightly wrong along the way — the same question 4.6 and 5.6.4 already asked you to ask about any other model output. Getting it right once, watched closely by someone who already knew the answer, isn't the same claim as getting it right reliably, unwatched, for a question you didn't already know the answer to, in a minute instead of a fraction of a millionth of a second. That gap — not raw capability — is what section 11.4 is actually for.

11.4 Wiring the Local Model to the Chapter 10 Tool

11.4.1 The Request/Response Shape

The pattern this section implements has four steps: a user asks a question; the model, given the Chapter 10 tool definition, decides whether to call it and with what arguments; your code executes the tool and gets a result; that result goes back to the model, which turns it into a plain-language answer.

Those four steps take **two** round-trips to the model, not one, and that is the single detail most worth getting straight before reading the code:



Three things this makes concrete that the prose has to say three separate times. The model never touches `call_tool` — it asks, your code executes, and the arrow from `tool.py` goes back to `llm.py`, not to the model. The second `chat` call replays the *whole* conversation, including the model's own tool-call message, because the model holds no memory between calls; leave that message out and the tool result arrives with nothing to attach itself to. And the `else` branch is not an error — a model choosing not to call the tool is normal behavior, which is why 11.4.3 treats it as something to expect and Chapter 12 makes it a scored outcome rather than a bug.

11.4.2 A Minimal Implementation

```
uv add ollama
```

In `src/mortgage_calculator_book/llm.py`:

```python
import ollama

from mortgage_calculator_book.tool import call_tool,
get_tool_definition

LOCAL_MODEL = "qwen3:8b"
```

```python
def ask_local(question: str) -> str:
    # Ollama expects tools as a list of {"type": "function", ...}
    # entries wrapping the exact name/description/parameters
    Chapter 10's
    # get_tool_definition() already produces.
    tool_def = get_tool_definition()
    tools = [
        {
            "type": "function",
            "function": {
                "name": tool_def["name"],
                "description": tool_def["description"],
                "parameters": tool_def["parameters"],
            },
        }
    ]

    # First call: hand the model the question and the tool it's
    allowed
    # to use. The model decides for itself whether calling it
    makes sense
    # here - nothing forces it to.
    first = ollama.chat(
        model=LOCAL_MODEL,
        messages=[{"role": "user", "content": question}],
        tools=tools,
    )
    message = first["message"]
    tool_calls = message.get("tool_calls")

    if not tool_calls:
        # The model chose to answer without calling the tool at
        all.
        return message["content"]

    # Only one tool exists in this project, so only the first
    call
    # matters - a project with several tools would need to loop
    here.
    call = tool_calls[0]
    result = call_tool(call["function"]["arguments"])

    # Second call: replay the conversation so far (the question,
    then the
```

```
    # model's own tool-call message), and append the tool's
    result as a
    # "tool" message. str(result) turns the {"payment": ...} or
    {"error":
    # ...} dict into text, since that's what this role expects
    here.
    second = ollama.chat(
        model=LOCAL_MODEL,
        messages=[
            {"role": "user", "content": question},
            message,
            {"role": "tool", "content": str(result)},
        ],
    )
    # The model reads the tool's result and turns it into a
    normal,
    # plain-language answer - this is that final answer.
    return second["message"]["content"]
```

(Ollama's exact tool-calling response shape can differ slightly between versions — check `ollama.chat`'s current documentation if a field name here doesn't match what you see. The pattern — two chat calls, with the tool's result inserted between them — stays the same regardless.)

Try it now, the same way as every other piece of this project — a small, throw-away script, not just code on the page:

```
vi scratch_llm_local.py
```

```
from mortgage_calculator_book.llm import ask_local

question = (
    "What would my payment be on a $200,000 loan at 6% over 30
    years?"
)
print(ask_local(question))
```

Run it:

```
uv run python scratch_llm_local.py
```

This calls the real local model, which means it's slower than anything else you've run so far — expect several seconds, not the instant response of a test suite. You're looking for a plain-language answer containing `$1,199.10`, matching Chapter 4.5's worked example. If you get something else — a refusal, a different number, no tool call at all — that's not necessarily wrong; it's 11.4.3's territory, coming up next. Delete the scratch file once you've seen it work:

```
rm scratch_llm_local.py
```

11.4.3 When the Model Gets It Wrong

Two failure modes are worth expecting rather than being surprised by: the model might answer a mortgage question in general terms without calling the tool at all, even when it should have, or it might call the tool with malformed or incomplete arguments. Neither is a bug in your code — it's the model's behavior, and it's exactly why Chapter 10.5.1 insisted `call_tool` return an error dictionary rather than raise: a malformed call still gets *something* sensible back, which the model can then read and potentially recover from on its own.

11.5 Environment Variables in Practice

11.5.1 Where the Chapter 8 Pattern Finally Gets Used

Chapter 8.7 set up `.env`, `.env.example`, and `config.py` well before there was anything to put in them. This is that payoff:

```
# .env (not committed)
OPENROUTER_API_KEY=sk-or-v1-your-real-key-here
```

`config.py`, unchanged since Chapter 8.7.2, already loads this correctly — nothing new to write here, only a real value to add.

11.5.2 Config Alongside the Secret

Worth adding one more constant to `config.py` while you're there, since section 11.6 will need it:

```
HOSTED_MODEL = "qwen/qwen3.8-27b"
```

Keeping the model name in config, rather than hardcoded inline, means switching models later is a one-line change.

11.6 OpenRouter & Paying for Models

11.6.1 What OpenRouter Is, and When Local Isn't Enough

OpenRouter provides API access to many hosted language models — including larger, more capable versions of models you may be running locally — through a single, consistent interface. It's worth reaching for when your local model's context window is too small, when local tool-calling reliability isn't good enough for what you're building, or when you simply don't have hardware capable of running something larger.

11.6.2 Account, Key, and Storage

Create an account at openrouter.ai, generate an API key, and store it exactly the way section 11.5.1 described — never hardcoded, never committed.

11.6.3 Understanding Cost

OpenRouter charges per token, with rates that vary by model — larger, more capable models cost more per request. Set a spending limit on your account before making your first real call, and check the usage dashboard after your first few requests so you have a concrete sense of what a typical calculator query actually costs. For a project this size, expect the cost of testing this chapter's code to be a small fraction of a dollar — but it's worth confirming that for yourself rather than taking it on faith.

11.6.4 Wiring OpenRouter In

OpenRouter exposes an OpenAI-compatible API, so the OpenAI Python library works against it directly:

```
uv add openai
```

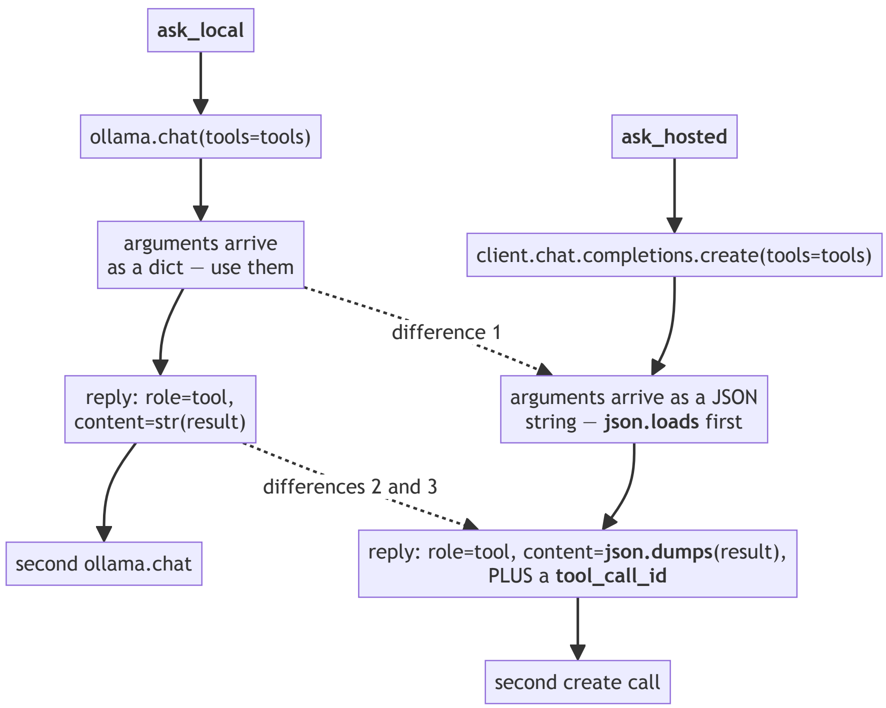
Same file as 11.4.2, `src/mortgage_calculator_book/llm.py` — `ask_hosted` sits alongside `ask_local`, not replacing it:

```python
import json

from openai import OpenAI

from mortgage_calculator_book.config import HOSTED_MODEL,
OPENROUTER_API_KEY
from mortgage_calculator_book.tool import call_tool,
get_tool_definition

# One client, reused across every call - no need to reconnect per
question.
_client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
api_key=OPENROUTER_API_KEY
)


def ask_hosted(question: str) -> str:
    # Same tool shape as ask_local - OpenAI's format and Ollama's
    happen
    # to agree here, which is part of why get_tool_definition()
    didn't
    # need to change to support a second client.
    tool_def = get_tool_definition()
    tools = [
        {
            "type": "function",
            "function": {
```

```
                "name": tool_def["name"],
                "description": tool_def["description"],
                "parameters": tool_def["parameters"],
            },
        }
    ]

    first = _client.chat.completions.create(
        model=HOSTED_MODEL,
        messages=[{"role": "user", "content": question}],
        tools=tools,
    )
    message = first.choices[0].message

    if not message.tool_calls:
        return message.content

    # Only the first tool call matters, same as ask_local - one
    tool, one
    # call. Arguments arrive as a JSON string here, not a dict,
    so they
    # need parsing before call_tool can use them.
    call = message.tool_calls[0]
    arguments = json.loads(call.function.arguments)
    result = call_tool(arguments)

    # Two real differences from ask_local, easy to miss: OpenAI's
    API
    # requires a tool_call_id on the reply, linking this result
    back to
    # the specific call that requested it, and it expects the
    content as
    # an actual JSON string (json.dumps), not Python's str().
    second = _client.chat.completions.create(
        model=HOSTED_MODEL,
        messages=[
            {"role": "user", "content": question},
            message,
            {
                "role": "tool",
                "tool_call_id": call.id,
                "content": json.dumps(result),
            },
        ],
    )
    return second.choices[0].message.content
```

Notice how closely this mirrors `ask_local` from 11.4.2 — same four-step shape, same `call_tool` function reused without modification. That's the tool contract from Chapter 10 doing exactly the job it was built for: one interface, usable from more than one model-calling client.

Closely, but not identically. Three things genuinely differ, and all three are the kind that fail quietly:



Everything not touched by a dotted arrow is the same in both. Those three are worth memorising before 11.8.2 asks you to review Pi's version of `ask_hosted`, because none of them raises an exception when you get it wrong — a forgotten `json.loads` hands `call_tool` a string, a missing `tool_call_id` gets the second call rejected, and `str()` where `json.dumps()` belongs produces almost-JSON the model may or may not read correctly.

Try it the same way as 11.4.2, with a scratch file of its own:

```
vi scratch_llm_hosted.py
```

```
from mortgage_calculator_book.llm import ask_hosted

question = (
    "What would my payment be on a $200,000 loan at 6% over 30
    years?"
```

```
)
print(ask_hosted(question))
```

Run it:

```
uv run python scratch_llm_hosted.py
```

This one costs a small fraction of a cent and should return considerably faster than 11.4.2's local run — worth noticing directly, since 11.7 asks you to compare the two more formally in a moment. You're looking for the same answer as before, `$1,199.10`. Delete the scratch file once you've seen it work:

```
rm scratch_llm_hosted.py
```

`llm.py` has been sitting uncommitted since `ask_local` was first written in 11.4 — both functions are real and working now, worth a checkpoint:

```
git add src/mortgage_calculator_book/llm.py
git commit -m "Add ask_local and ask_hosted"
git push
```

11.7 Comparing Local vs. Hosted, Informally

11.7.1 Running the Same Questions Through Both

One more scratch file, this time running both paths back to back:

```
vi scratch_llm_compare.py
```

```
from mortgage_calculator_book.llm import ask_hosted, ask_local

question = (
    "What would my monthly payment be on a $200,000 loan "
    "at 6% over 30 years?"
)
print("Local: ", ask_local(question))
print("Hosted:", ask_hosted(question))
```

Run it:

```
uv run python scratch_llm_compare.py
```

Expect a noticeable pause before the first line prints — the local call runs first and is the slower of the two, consistent with what 11.4.2's scratch run already showed you — then the hosted line follows more quickly. Both should land on `$1,199.10`, worded differently. Delete the scratch file once you've compared them:

```
rm scratch_llm_compare.py
```

11.7.2 What to Watch For

- **Latency** — the hosted call likely returns faster than a local model running on consumer hardware, though this varies with model size and your own machine.
- **Tool-call reliability** — does each model correctly call the tool every time, with well-formed arguments, or does one need a second attempt more often than the other?
- **Answer quality** — given the same correct tool result, does one model phrase the final answer more clearly, or add unhelpful hedging?

11.7.3 A Practical Decision Framework

For a project at this scale — a single tool, occasional use — local is often entirely sufficient, and free after the initial setup cost. Reach for hosted when reliability genuinely matters more than cost, when your local hardware can't run a model good enough to be reliable, or when you need a context window larger than what fits comfortably on your machine. Chapter 12 turns these impressions into an actual measured comparison — this section is deliberately just a first look, not a conclusion.

11.8 Agent-Assisted Build, Same Habits as Always

11.8.1 Tests First, With Mocking

Live model calls are slow and, for the hosted path, cost real money — neither is acceptable inside a test suite that might run dozens of times a day. Mock the model call instead:

```python
# tests/test_llm.py
from unittest.mock import MagicMock

from mortgage_calculator_book.llm import ask_local


def test_ask_local_calls_tool_and_returns_answer(monkeypatch):
    tool_call = {
        "function": {
            "name": "calculate_mortgage_payment",
            "arguments": {
                "principal": 200000, "annual_rate": 0.06,
                "term_years": 30, "payments_per_year": 12,
            },
        }
    }
```

```
    first_response = {
        "message": {"content": "", "tool_calls": [tool_call]}
    }
    second_response = {"message": {"content": "Payment:
$1,199.10."}}
    mock_chat = MagicMock(side_effect=[first_response,
second_response])
    monkeypatch.setattr(
        "mortgage_calculator_book.llm.ollama.chat", mock_chat
    )

    answer = ask_local(
        "What would my payment be on a $200,000, 6%, 30 year
        loan?"
    )

    assert "1,199.10" in answer
    assert mock_chat.call_count == 2
```

`monkeypatch` — another built-in pytest fixture, alongside `capsys` from Chapter 8.6.2 — temporarily replaces `ollama.chat` with a fake that returns exactly the two scripted responses above, in order. This tests the *wiring* — does `ask_local` correctly extract arguments, call `call_tool`, and pass the result back — without ever touching a real model.

11.8.2 Prompting Pi, Reviewing the Diff

`ask_hosted` already exists, fully written by hand back in 11.6.4 — same issue as Chapters 7.6, 8.6, and 9.6.2, and the same fix: set it aside first, so there's actually something for Pi to build rather than something to redundantly retype. It's committed as of 11.6.4, so removing it costs nothing. Delete just the `ask_hosted` function itself, and the imports only it needs (`json`, `OpenAI`, `HOSTED_MODEL`, `OPENROUTER_API_KEY`, the module-level `_client`) — `ask_local` and everything it depends on stays untouched:

```
git add src/mortgage_calculator_book/llm.py
git commit -m "Remove hand-written ask_hosted to redo via Pi"
```

```
pi "Implement ask_hosted in \
    src/mortgage_calculator_book/llm.py, mirroring the \
    structure of ask_local but using the OpenAI client \
    against OpenRouter."
```

Review this one particularly closely — it's the most consequential integration in the book so far, and a subtle mistake (mismatched argument formats between the two APIs, a missing `tool_call_id`) could fail silently rather than crash outright. `ruff check .` will at least catch any import Pi's version no longer needs, or forgot to add.

Verify it the same way as 11.6.4 did — recreate that scratch file, since it was deleted at the end of that section:

```
vi scratch_llm_hosted.py
```

```python
from mortgage_calculator_book.llm import ask_hosted

question = (
    "What would my payment be on a $200,000 loan at 6% over 30
    years?"
)
print(ask_hosted(question))
```

```
uv run python scratch_llm_hosted.py
```

Same target as before, `$1,199.10`. Delete it again once confirmed:

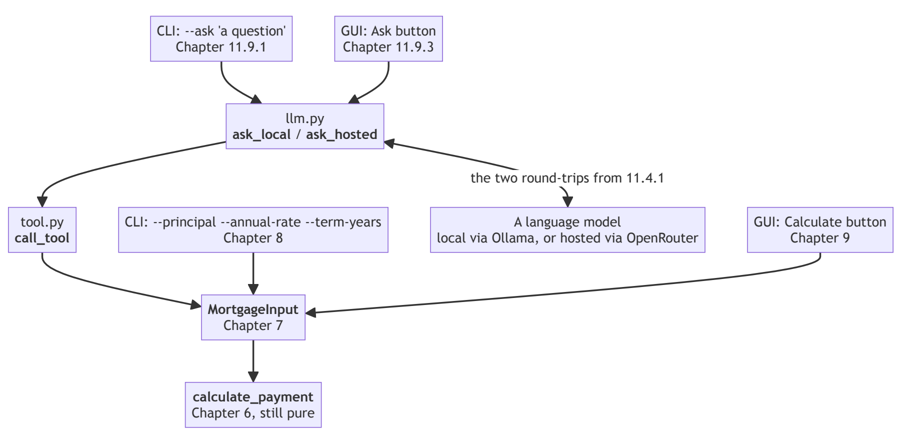
```
rm scratch_llm_hosted.py
```

Once you're satisfied, commit Pi's version:

```
git add src/mortgage_calculator_book/llm.py
git commit -m "Rebuild ask_hosted via Pi"
git push
```

11.9 Connecting This to the CLI and UI

Everything so far has proven `ask_local` and `ask_hosted` work — but only from scratch files nobody else will ever run. The two front ends a real user would actually reach for, Chapter 8's CLI and Chapter 9's UI, still know nothing about either function. Once that gap closes, the system this book has been assembling since Chapter 6 exists in full — every entry point a user has, landing on one core:



Four ways in, one way through. Compare it against Chapter 7.7.1's two-box version and notice what changed: everything from `MortgageInput` rightward is identical, and every addition since has been another arrow arriving on the left. That's not a coincidence to admire in passing — it's the payoff for Chapter 6 refusing to print anything and Chapter 7 insisting on a single entry point, decisions that both looked like unnecessary ceremony at the time.

Close the gap now, the same way as everything else that's touched more than one file in this project: design it, write it down, then build it.

11.9.1 The CLI: A Natural-Language Flag

The shape: a new `--ask` flag that takes a question directly, bypassing `--principal`/`--annual-rate`/`--term-years` entirely, with an optional `--hosted` flag choosing `ask_hosted` over the local default. The three structured flags need to become optional rather than required, since `--ask` is now a second, independent way to use this command — enforced by checking after parsing, not by `argparse` itself.

```
pi "Add an --ask flag to the CLI in \
    src/mortgage_calculator_book/cli.py. When --ask \
    'question text' is given, call ask_local (or \
    ask_hosted if --hosted is also passed) from llm.py \
    and print the returned answer, skipping the normal \
    principal/annual-rate/term-years flow entirely. Make \
    those three flags optional, and enforce after parsing \
    that they're required whenever --ask isn't used."
```

Review the same way as every agent-assisted change since 1.6 — specifically confirm the old, structured usage from Chapter 8 still works unchanged; this is meant to add a mode, not replace one. Verify the new one:

```
uv run mortgage-calculator-book --ask \
    "What would my payment be on a $200,000 loan at 6% over 30
    years?"
```

You're looking for a plain-language answer containing `$1,199.10`.

11.9.2 The UI: Sketching and Documenting the Addition

A new section below the existing form: a single-line question field, an "Ask" button, and an answer area.

```
Principal ($):            [____________]
Annual rate (e.g. 0.06): [____________]
Term (years):             [____________]
Payments per year:        [____________]

[ Calculate ]  [ Clear ]
```

```
Fixed periodic payment: $1,199.10

----------------------------------------

Ask a question:  [________________________________]  [ Ask ]

Answer:
```

Same lesson as Chapter 4's `docs/derivation.md`: a sketch that lives only in this book is invisible to Pi, which reads the project, not the pages you're reading right now. Write this one down for real, in a file that's never actually existed until this sentence:

```
vi docs/ui.md
```

```
# Calculator UI Layout

The current window, as built through Chapter 11.9:

    Principal ($):            [____________]
    Annual rate (e.g. 0.06): [____________]
    Term (years):             [____________]
    Payments per year:        [____________]

    [ Calculate ]  [ Clear ]

    Fixed periodic payment: $1,199.10

    ----------------------------------------

    Ask a question:  [________________________________]  [ Ask ]

    Answer:

## Behavior
- Calculate / Clear: as specified in SPEC.md's Interfaces
section.
- Ask: sends the typed question to ask_local (Chapter 11), and
  displays the returned plain-language answer below the field.
  Uses the same MortgageInput / calculate_validated_payment path
  as everything else in this project, by way of the Chapter 10
  tool interface - never a separate calculation.
```

Commit it, then bring SPEC.md's Interfaces section (9.6.2, extended in 13.6.2) up to date alongside it:

```
git add docs/ui.md
```

```
git commit -m "Add docs/ui.md, documenting the calculator's UI
layout"

vi SPEC.md

## Interfaces
- Command-line interface (human-readable and JSON output; --ask
  for a natural-language question, answered via the tool
interface)
- Desktop GUI (see docs/ui.md for the current layout):
  - Inputs: principal, annual rate, term (years), payments per
year
  - Actions: Calculate, Clear, and Ask (a natural-language
question,
    answered via the tool interface)
  - Invalid input shows an error message in place, not a crash

git add SPEC.md
git commit -m "Document the --ask flag and UI Ask button in
SPEC.md"
git push
```

11.9.3 Building It, via Pi

```
pi "Add an 'Ask a question' section to the calculator \
    window in src/mortgage_calculator_book/ui.py, \
    matching docs/ui.md: a single-line input, an Ask \
    button, and an answer area below it. Clicking Ask \
    should call ask_local from llm.py with the typed \
    text and display the returned answer."
```

Review it the same way as 9.6 — this one's mechanical (a field, a button, a label) plus one real integration point (the call to `ask_local`) worth checking closely: does it handle a slow response without freezing the whole window in a way that looks broken? A multi-second pause with no feedback at all is a rough edge worth noticing, even if fixing it properly (threading, a progress indicator) is more than this project needs.

Run it and try the new field with the same question as 11.9.1's CLI check:

```
uv run python -m mortgage_calculator_book.ui
```

Confirm the answer shows `$1,199.10`, then commit:

```
git add src/mortgage_calculator_book/ui.py
git commit -m "Add natural-language Ask button to the UI"
git push
```

11.10 Refactor and Ruff Pass

```
ruff check . && ruff format .
```

11.11 Checkpoint

Before moving to Chapter 12, this should all be true:

- ☐ `ask_local` correctly calls the Chapter 10 tool and returns a plain-language answer, verified against a real local model at least once
- ☐ `ask_hosted` does the same against OpenRouter, verified against a real API call at least once
- ☐ `.env` holds a real `OPENROUTER_API_KEY` and is confirmed git-ignored
- ☐ Tests for both paths pass using mocked model responses — no live model call or API cost required to run the suite
- ☐ You've run the same question through both `ask_local` and `ask_hosted` and compared the results
- ☐ `mortgage-calculator-book --ask "..."` returns a correct plain-language answer, and the original structured flags still work unchanged
- ☐ The UI's Ask button returns the same worked-example answer as every other front end
- ☐ `docs/ui.md` exists, matches the actual UI, and is committed
- ☐ `SPEC.md`'s Interfaces section mentions the natural-language capability for both the CLI and the UI
- ☐ `ruff check .` and `ruff format .` both pass

What's next: Chapter 12 turns this section's informal local-vs-hosted comparison into a real, repeatable evaluation.

Chapter 12 — Evaluation

Contents

12.1 Why This Chapter Exists

Chapter 11 ended with an informal comparison — run a question through both models, read the two answers, form an impression. That's a reasonable first look, but "it seemed to work when I tried it" isn't evidence, it's an anecdote. This chapter builds a small, repeatable eval set and a way to score both models against it, so the local-vs-hosted comparison from Chapter 11.7 becomes something you can actually point to.

12.2 Evaluation Is Not Testing

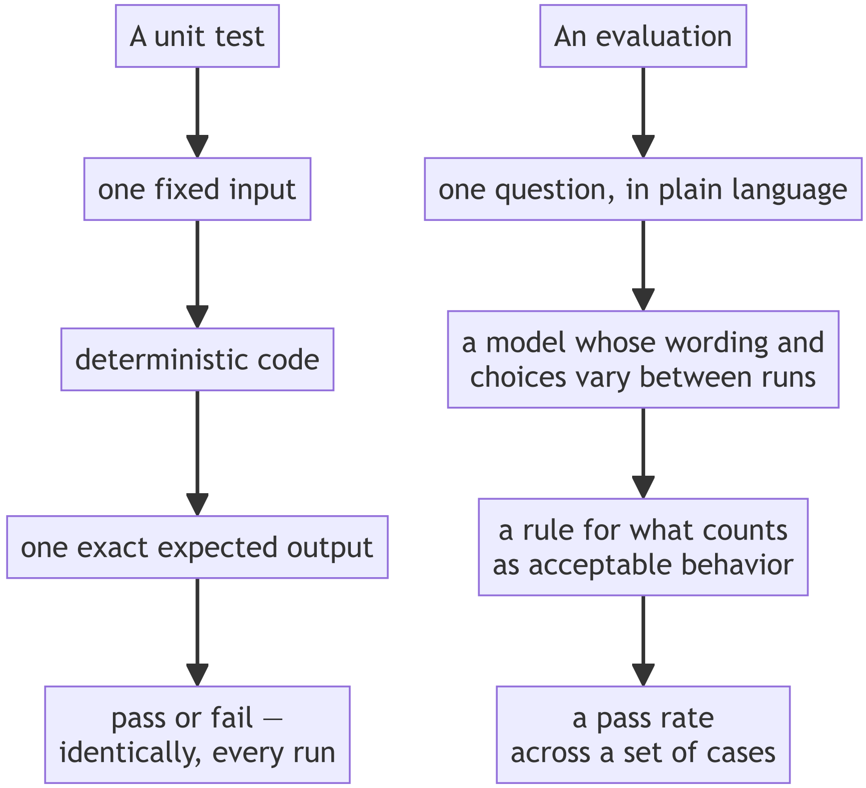
12.2.1 What Unit Tests Check

Every test since Chapter 5 has checked the same kind of thing: given this exact input, does the code produce this exact, deterministic output? `calculate_payment(200_000, 0.06, 30, 12)` either returns `1199.10` or it doesn't — there's no ambiguity in what "pass" means.

12.2.2 Why a Model Needs Something Different

A language model's behavior isn't deterministic in that same sense — ask it the same question twice and you may get differently worded answers, or even a different decision about whether to call the tool at all. "Pass or fail" still applies, but what counts as passing has to be defined more carefully than "matches this exact string."

> **Process concept: evaluation as its own discipline, distinct from unit testing.** This is the chapter's central distinction, worth stating plainly before writing any eval code: a unit test checks that *code* does what it's supposed to, exactly, every time. An evaluation checks that a *model's behavior*, which varies, stays within acceptable bounds often enough to trust. Both matter. They are not the same tool, and conflating them — expecting eval-style tolerance from a unit test, or unit-test-style exactness from an eval — leads to the wrong kind of disappointment in both.



Same shape, three substituted parts — and the substitution in the middle is what forces the other two. Because the thing being checked doesn't repeat itself exactly, the expected value has to become a rule rather than a literal, and a single result has to become a rate. Ask a unit test to tolerate that variation and it stops catching regressions; ask an eval for exactness and every run looks like a failure.

12.3 What to Evaluate

12.3.1 Tool-Call Correctness

Did the model call the tool at all, when it should have? And when it did, were the arguments correct — not just present, but matching what the question actually asked for?

12.3.2 Answer Quality

Given a correct tool result, did the model report it accurately in plain language? A model that calls the tool correctly but then misreads or mangles the result in its final answer has failed just as thoroughly as one that never called the tool. Flagged here, but not scored automatically by this chapter's harness (12.5.2): judging whether a sentence of prose accurately reports a number is a harder problem than comparing two numbers, needing either a human reader or another model as a judge. This project's `score_case` stays intentionally narrower — tool-call correctness and refusal behavior only — and leaves answer quality to a manual read of the transcripts, the same spot-check 11.7 already had you do.

12.3.3 Refusal and Clarification Behavior

Not every question should trigger a tool call. A question outside SPEC.md's scope (Chapter 2.3.4 and 4.7.4's "Out of scope" section: variable-rate mortgages, refinancing, multiple currencies) should be recognized as such, not forced through the calculator anyway. A question that's *in* scope but missing information — "how much would I pay each month?" with no principal, rate, or term given — should prompt for what's missing, not invent plausible-sounding numbers and call the tool with them.

12.4 Building a Small Eval Set

12.4.1 Representative Questions

A full eval set for this project should run 15–25 questions deep; the set below shows a representative slice of that range — enough to demonstrate every category from 12.3, and a real starting point to extend on your own.

12.4.2 Expected Outcomes, Defined Per Case

For each question: should the tool be called, and if so, with which arguments? Store this as data, in `data/eval_set.json`:

```json
[
  {
    "id": "basic-1",
    "question": "What would my monthly payment be on a $200,000
    loan at 6% interest over 30 years?",
    "expected_tool_call": true,
    "expected_arguments": {
      "principal": 200000,
      "annual_rate": 0.06,
      "term_years": 30,
      "payments_per_year": 12
    }
```

```
},
{
  "id": "zero-interest-1",
  "question": "If I borrow $12,000 interest-free and pay it
  back monthly over a year, what's my payment?",
  "expected_tool_call": true,
  "expected_arguments": {
    "principal": 12000,
    "annual_rate": 0.0,
    "term_years": 1,
    "payments_per_year": 12
  }
},
{
  "id": "single-payment-1",
  "question": "I want to pay off a $10,000 loan at 6% in one
  single payment a year from now. How much would that be?",
  "expected_tool_call": true,
  "expected_arguments": {
    "principal": 10000,
    "annual_rate": 0.06,
    "term_years": 1,
    "payments_per_year": 1
  }
},
{
  "id": "frequency-1",
  "question": "Same $200,000 loan at 6% for 30 years, but if I
  paid biweekly instead of monthly, what would each payment
  be?",
  "expected_tool_call": true,
  "expected_arguments": {
    "principal": 200000,
    "annual_rate": 0.06,
    "term_years": 30,
    "payments_per_year": 26
  }
},
{
  "id": "missing-frequency-1",
  "question": "What's the payment on a $150,000 loan at 5% over
  15 years?",
  "expected_tool_call": true,
  "expected_arguments": {
    "principal": 150000,
    "annual_rate": 0.05,
```

```
      "term_years": 15,
      "payments_per_year": 12
    }
  },
  {
    "id": "out-of-scope-1",
    "question": "What's the weather like today?",
    "expected_tool_call": false
  },
  {
    "id": "out-of-scope-2",
    "question": "Can you help me figure out if refinancing my
    mortgage makes sense?",
    "expected_tool_call": false
  },
  {
    "id": "ambiguous-1",
    "question": "How much would I pay each month?",
    "expected_tool_call": false
  }
]
```

Note `missing-frequency-1` and `out-of-scope-2` in particular: the first checks that the model correctly defaults to monthly payments when frequency isn't mentioned (matching `MortgageInput`'s own default from Chapter 7.5.1); the second checks that a question about refinancing — explicitly out of scope since Chapter 2.3.4 — doesn't get forced through the calculator anyway.

12.4.3 Data, Not a Hardcoded Script

Storing this as JSON rather than as Python code inside the eval runner means it can be inspected, extended, and reasoned about independently of the code that runs it — the same argument Chapter 8.5.1 made for JSON output over printed text, applied here to test data instead of program output.

12.5 Running the Eval

12.5.1 A Small Refactor First

Chapter 11's `ask_local` and `ask_hosted` return only a final text answer — not enough for scoring, which needs to know *whether* the tool was called and *with what arguments*. Rather than bolting that onto the existing functions, split each into a detailed version that returns everything, with the original kept as a thin wrapper:

```
# in llm.py, replacing the body of ask_local:
```

```python
def ask_local_detailed(question: str) -> dict:
    # Same tool shape as Chapter 11.4.2 - one entry, wrapping
    # get_tool_definition()'s name/description/parameters.
    tool_def = get_tool_definition()
    tools = [
        {
            "type": "function",
            "function": {
                "name": tool_def["name"],
                "description": tool_def["description"],
                "parameters": tool_def["parameters"],
            },
        }
    ]

    first = ollama.chat(
        model=LOCAL_MODEL,
        messages=[{"role": "user", "content": question}],
        tools=tools,
    )
    message = first["message"]
    tool_calls = message.get("tool_calls")

    if not tool_calls:
        return {
            "answer": message["content"],
            "tool_called": False,
            "arguments": None,
        }

    # Only the first call matters - still one tool, same as
    11.4.2.
    call = tool_calls[0]
    arguments = call["function"]["arguments"]
    result = call_tool(arguments)

    second = ollama.chat(
        model=LOCAL_MODEL,
        messages=[
            {"role": "user", "content": question},
            message,
            {"role": "tool", "content": str(result)},
        ],
    )
    # arguments is returned here too now - the whole reason for
    this
```

```
    # refactor, since scoring needs to see what the model
    actually sent,
    # not just the final answer.
    return {
        "answer": second["message"]["content"],
        "tool_called": True,
        "arguments": arguments,
    }


def ask_local(question: str) -> str:
    return ask_local_detailed(question)["answer"]
```

The same restructuring applies to `ask_hosted`, written out in full since the hosted client's shape differs enough from Ollama's to be worth seeing rather than assuming:

```
# in llm.py, replacing the body of ask_hosted:

def ask_hosted_detailed(question: str) -> dict:
    tool_def = get_tool_definition()
    tools = [
        {
            "type": "function",
            "function": {
                "name": tool_def["name"],
                "description": tool_def["description"],
                "parameters": tool_def["parameters"],
            },
        }
    ]

    first = _client.chat.completions.create(
        model=HOSTED_MODEL,
        messages=[{"role": "user", "content": question}],
        tools=tools,
    )
    message = first.choices[0].message

    if not message.tool_calls:
        return {
            "answer": message.content,
            "tool_called": False,
            "arguments": None,
        }

    # Same two differences from the local path as 11.6.4 noted: a
```

```
    # tool_call_id to thread through, and arguments arriving as a
    JSON
    # string rather than a dict.
    call = message.tool_calls[0]
    arguments = json.loads(call.function.arguments)
    result = call_tool(arguments)

    second = _client.chat.completions.create(
        model=HOSTED_MODEL,
        messages=[
            {"role": "user", "content": question},
            message,
            {
                "role": "tool",
                "tool_call_id": call.id,
                "content": json.dumps(result),
            },
        ],
    )
    return {
        "answer": second.choices[0].message.content,
        "tool_called": True,
        "arguments": arguments,
    }


def ask_hosted(question: str) -> str:
    return ask_hosted_detailed(question)["answer"]
```

This is a small, real example of what Chapter 6.8's "refactor" step looks like several chapters later: existing behavior preserved exactly (both wrappers' signatures and return types are unchanged from Chapter 11), with new capability added underneath.

12.5.2 Scoring

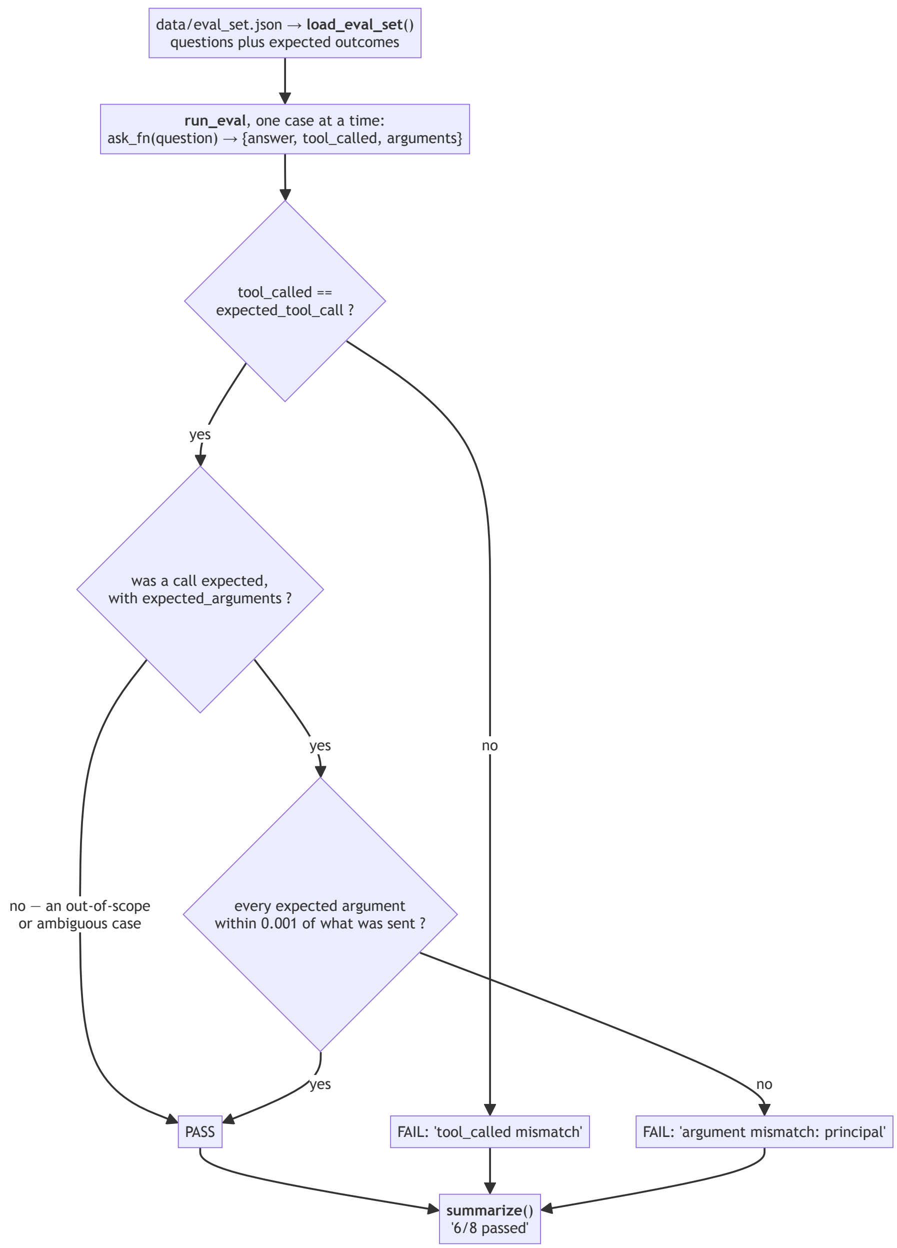
In src/mortgage_calculator_book/eval.py:

```
import json
from pathlib import Path
from typing import Any, Callable

# From eval.py's own location (src/mortgage_calculator_book/),
climb three
# levels to the project root, then into data/ -- the same file
every eval
# question comes from.
```

```
EVAL_SET_PATH = (
    Path(__file__).resolve().parent.parent.parent
    / "data" / "eval_set.json"
)


def load_eval_set() -> list[dict[str, Any]]:
    return json.loads(EVAL_SET_PATH.read_text())


def score_case(
    case: dict[str, Any], ask_fn: Callable[[str], dict]
) -> dict[str, Any]:
    # ask_fn is ask_local_detailed or ask_hosted_detailed
    (12.5.1) --
    # whichever model this run is scoring.
    result = ask_fn(case["question"])

    # First check: did the model call the tool when it should
    have (or
    # correctly not call it, for an out-of-scope question)? Wrong
    either
    # way is an automatic fail -- no need to look at arguments at
    all.
    if result["tool_called"] != case["expected_tool_call"]:
        return {
            "id": case["id"],
            "passed": False,
            "reason": "tool_called mismatch",
        }

    # Second check, only when a call was actually expected: does
    each
    # argument this case cares about match what the model
    actually sent?
    # Comparing as floats with a small tolerance avoids false
    failures
    # from harmless formatting differences like "12" vs "12.0".
    if case["expected_tool_call"] and "expected_arguments" in
    case:
        for key, expected in case["expected_arguments"].items():
            # arguments is None when no call happened; "or {}"
            keeps
            # .get() from raising in that case instead of masking
            it.
            actual = (result["arguments"] or {}).get(key)
```

```
            if (
                actual is None
                or abs(float(actual) - float(expected)) > 0.001
            ):
                return {
                    "id": case["id"],
                    "passed": False,
                    "reason": f"argument mismatch: {key}",
                }

    return {"id": case["id"], "passed": True, "reason": None}


def run_eval(
    ask_fn: Callable[[str], dict], verbose: bool = True
) -> list[dict[str, Any]]:
    # One case at a time, not a list comprehension, so progress
    can be
    # reported as each one finishes -- worth having, given how
    long a
    # real run over real models actually takes (12.6.1).
    cases = load_eval_set()
    results = []
    for i, case in enumerate(cases, start=1):
        result = score_case(case, ask_fn)
        if verbose:
            status = "PASS" if result["passed"] else "FAIL"
            print(f"[{i}/{len(cases)}] {case['id']}: {status}")
        results.append(result)
    return results


def summarize(results: list[dict[str, Any]]) -> str:
    passed = sum(1 for r in results if r["passed"])
    return f"{passed}/{len(results)} passed"
```

`score_case` is the piece worth tracing carefully, because it checks two different things in a fixed order and gives up at the first one that fails:



The early exit at the first check is deliberate, not an optimization: if the model didn't call the tool when it should have, there are no arguments to compare, and a failure reason of "tool_called mismatch" is a more useful thing to read in the output than a cascade of missing-argument complaints. The middle branch is where `out-of-scope-1` and `ambiguous-1` pass — those cases expect *no* call, so correctly declining is the whole test.

Deliberately simple pass/fail scoring — no partial credit, no weighting. At this scale, that's a feature, not a limitation: results stay easy to read and easy to reason about.

Worth noticing that `run_eval` prints, unlike `calculate_payment` back in Chapter 6.2 — not a contradiction of that chapter's purity rule, just a different kind of function. `calculate_payment` is the domain core, meant to be composed into a CLI, a UI, and a tool interface without dragging any of them along; `run_eval` already isn't that pure, since `ask_fn` itself does real network I/O every time it's called. Given that, reporting progress on an operation that can take minutes is a reasonable, honest thing for this specific function to do — set `verbose=False` if you want the old silent behavior back.

12.5.3 Testing the Scoring Logic Itself

The scoring function is plain code, and plain code gets tested the normal way — no model required:

```python
# tests/test_eval.py
from mortgage_calculator_book.eval import score_case


def _fake_ask_correct(question: str) -> dict:
    return {
        "answer": "It would be $1,199.10.",
        "tool_called": True,
        "arguments": {
            "principal": 200000,
            "annual_rate": 0.06,
            "term_years": 30,
            "payments_per_year": 12,
        },
    }


def _fake_ask_no_call(question: str) -> dict:
    return {
        "answer": "I'm not sure.",
        "tool_called": False,
        "arguments": None,
    }
```

```
def test_score_case_passes_on_matching_call():
    case = {
        "id": "basic-1",
        "question": "irrelevant here",
        "expected_tool_call": True,
        "expected_arguments": {
            "principal": 200000, "annual_rate": 0.06,
            "term_years": 30
        },
    }
    assert score_case(case, _fake_ask_correct)["passed"] is True


def test_score_case_fails_when_tool_not_called_but_expected():
    case = {
        "id": "basic-1",
        "question": "irrelevant here",
        "expected_tool_call": True,
    }
    assert score_case(case, _fake_ask_no_call)["passed"] is False
```

12.6 Comparing Local vs. Hosted, Formally

12.6.1 Running the Full Set Against Both

Both pieces of this section belong in the same file, since the second depends on variables the first defines:

```
vi scratch_eval_compare.py
```

```
from mortgage_calculator_book.eval import (
    load_eval_set,
    run_eval,
    summarize,
)
from mortgage_calculator_book.llm import (
    ask_local_detailed,
    ask_hosted_detailed,
)

local_results = run_eval(ask_local_detailed)
hosted_results = run_eval(ask_hosted_detailed)

print("Local: ", summarize(local_results))
print("Hosted:", summarize(hosted_results))
```

Run it:

```
uv run python scratch_eval_compare.py
```

This takes noticeably longer than anything else you've run so far — eight real questions, against two real models, not the mocked responses from 11.8.1's tests. `run_eval`'s progress lines (12.5.2) print as each case finishes, so you'll see `[1/8] basic-1: PASS`-style output scrolling by before the two summary lines at the end. Expect something like `Local: 6/8 passed` and `Hosted: 8/8 passed`; your actual numbers will differ, and that's fine — the point is having real numbers at all, not matching these ones.

12.6.2 Reading the Results Side by Side

Print each case's individual result, not just the summary, and look specifically at *where* the two diverge. Add this to the bottom of the same file:

```
vi scratch_eval_compare.py
```

```python
for local, hosted in zip(local_results, hosted_results):
    if local["passed"] != hosted["passed"]:
        print(
            f"{local['id']}: local={local['passed']} "
            f"hosted={hosted['passed']}"
        )
```

Run the same command again:

```
uv run python scratch_eval_compare.py
```

A case where local fails and hosted passes is informative in a way an aggregate score alone isn't — it might mean local reliably struggles with, say, the `ambiguous-1` case specifically (calling the tool with guessed numbers rather than declining), which tells you something concrete and actionable about that model's behavior, not just "hosted scored higher." Delete the scratch file once you've read through the output:

```
rm scratch_eval_compare.py
```

12.6.3 From Impression to Evidence

This is Chapter 11.7's informal comparison, done properly: instead of "the hosted model seemed to answer better," you now have a specific pass count, on a specific, inspectable set of questions, with specific cases identified where the two diverge. That's a claim you can actually defend, and re-check later.

12.7 Regression Checks

12.7.1 Why Re-Run This Later

Any future change to the tool's description (Chapter 10.4.1), its schema, the model chosen, or even the prompt structure in `llm.py` can shift how reliably the model calls the tool — sometimes for the better, sometimes not. Re-running the eval set after any such change is how you'd actually notice a regression, rather than assuming a change was harmless because it seemed reasonable at the time.

12.7.2 A Lightweight Habit

This doesn't need full continuous-integration infrastructure (Appendix A.3 covers that, if you want it later) — just the habit of running `run_eval` again after a meaningful change and comparing the new summary to the last one you recorded. Simple, and enough for a project this size.

12.8 Checkpoint

Before moving to Chapter 13, this should all be true:

- ☐ `data/eval_set.json` contains at least the eight representative cases above, covering correct calls, edge cases, out-of-scope questions, and an ambiguous question
- ☐ `ask_local_detailed` and `ask_hosted_detailed` exist, with `ask_local` and `ask_hosted` preserved as thin wrappers around them
- ☐ `run_eval` and `score_case` work correctly, verified with mocked `ask_fn` functions in `tests/test_eval.py`
- ☐ You've run the full eval set against both the local and hosted model at least once, and can state both pass counts
- ☐ You've identified at least one case where the two models diverge, and have a plausible explanation why

What's next: Chapter 13 hardens the whole system against the messier reality outside this curated eval set — malformed input, unexpected model behavior, and everything else a real user might do that these eight questions don't cover.

Chapter 13 — Hardening

Contents

13.1 Why This Chapter Exists

Chapter 12 proved the system works against a curated set of questions you wrote yourself. This chapter assumes a less forgiving user — one who fat-fingers a number, whose model produces a malformed tool call, or who asks something the eval set never anticipated. By the end, every seam this project has will handle failure gracefully instead of crashing, structured logging will exist to help you diagnose problems after the fact, and SPEC.md and README.md will each get one last honest check against what actually got built.

13.2 What "Production-ish" Means for a Learning Project

13.2.1 Setting Scope Honestly

This chapter does not turn the mortgage calculator into a production system — that would mean deployment infrastructure, monitoring, authentication, and considerably more than a book chapter can responsibly cover. What it does mean: the system should survive contact with a careless or curious user without crashing, losing data, or producing a silently wrong answer.

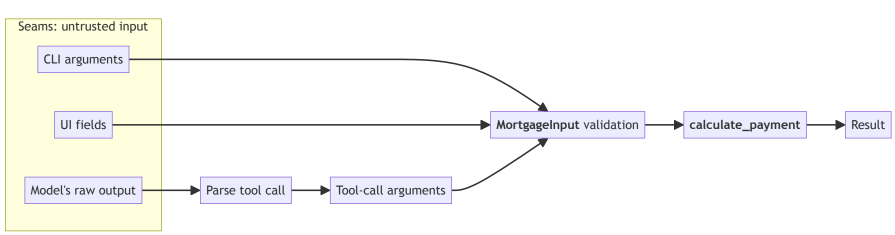
13.2.2 What's Still Out of Scope

Docker, CI/CD, and the rest of Appendix A remain out of scope here too — hardening in this chapter is about the application's own behavior at its boundaries, not about how it's deployed or automated.

13.3 Finding the Seams

13.3.1 Mapping the Boundaries

Four places in this system currently receive input from something outside your own tested code: the CLI (Chapter 8), the UI (Chapter 9), tool-call arguments arriving from a model (Chapter 10–11), and the model's own raw output (Chapter 11). Each is a **seam** — a place where this project's trusted, tested code meets something less predictable.



CLI and UI arguments converge directly on the same validation boundary — `MortgageInput` doesn't care which front end an input came from. The model's path is different: its raw output has to be parsed before tool-call arguments even exist, and that one parsing step has two distinct ways to go wrong — malformed JSON (13.4.2) or no tool call at all (13.4.3) — before whatever survives parsing reaches that same boundary too.

13.3.2 Why Each Needs Different Handling

A bad float typed into the UI (Chapter 9) is already caught by `MortgageInput`'s validation — that seam is largely handled. A model producing malformed JSON where valid tool arguments were expected (Chapter 11) is a genuinely different failure, at a different layer, and nothing built so far specifically guards against it. Treating every seam as if it needed the same fix would miss what's actually still exposed.

13.4 Error Handling at Each Seam

13.4.1 Stress-Testing Existing Validation

Chapter 7's `MortgageInput` already rejects negative or zero principal, out-of-range rates, and non-positive terms. What it doesn't yet catch: a principal so large it's obviously a data-entry mistake rather than a real loan. Add one more constraint:

```
# in validation.py

    @field_validator("principal")
    @classmethod
    def principal_must_be_positive(cls, v: float) -> float:
        if v <= 0:
            raise ValueError("principal must be positive")
        if v > 100_000_000:
            raise ValueError("principal must be less than
            $100,000,000")
        return v
```

And a test:

```
def test_absurdly_large_principal_rejected():
    with pytest.raises(ValidationError):
        MortgageInput(principal=1e12, annual_rate=0.06,
term_years=30)
```

This is a genuinely new constraint, not previously written down anywhere — worth remembering for section 13.6, where SPEC.md gets checked against exactly this kind of drift.

13.4.2 Malformed Tool-Call Arguments

Chapter 10's `call_tool` already handles the case where arguments are present but fail `MortgageInput`'s validation. It does *not* handle a case one layer earlier: in `ask_hosted_detailed`, `call.function.arguments` arrives as a raw JSON string that has to be parsed before it even reaches `call_tool` — and a model can, occasionally, produce malformed JSON there. Harden that specific step:

```
# in llm.py, inside ask_hosted_detailed

    call = message.tool_calls[0]
    try:
        arguments = json.loads(call.function.arguments)
    except json.JSONDecodeError:
        return {
            "answer": (
                "I had trouble understanding those loan "
                "details - could you restate them?"
            ),
            "tool_called": True,
            "arguments": None,
        }

    result = call_tool(arguments)
```

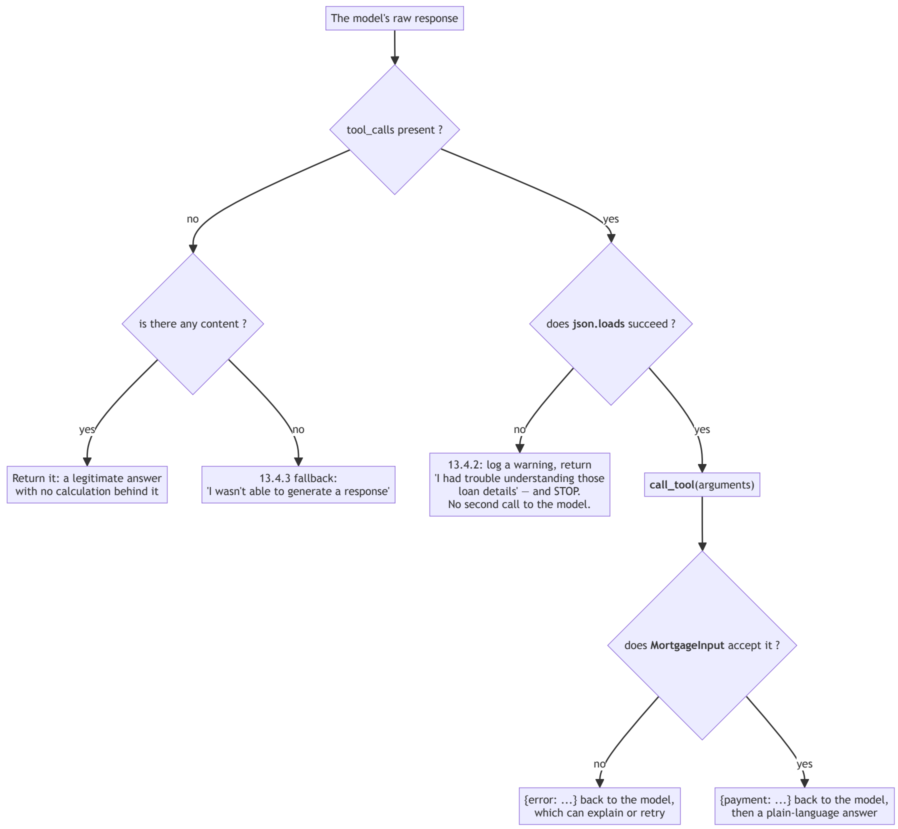
This is a different failure from the one Chapter 10.5.1 already guards against: that chapter handles *valid* JSON with *invalid* values (a negative principal); this handles JSON that isn't valid JSON at all. Both are real, and they needed separate handling because they happen at separate points in the pipeline.

13.4.3 Unexpected Model Output

What if the model returns neither a tool call nor any content — an empty response? `ask_local_detailed` and `ask_hosted_detailed` should return something sensible rather than `None` or an empty string reaching a user unexplained:

```
    if not tool_calls:
        fallback = "I wasn't able to generate a response - try
rephrasing."
        return {
            "answer": message.get("content") or fallback,
            "tool_called": False,
            "arguments": None,
        }
```

Taken together, 13.4.2 and 13.4.3 turn the model's side of the pipeline into a tree with four distinct exits — worth seeing at once, since 13.3.1's diagram showed *where* this seam is without showing what happens inside it:



Only the bottom two exits reach the model a second time. That's not a detail of style — it's the reason 13.4.4's test asserts `call_count == 1` rather than `2`, and reading it off this tree is considerably easier than reconstructing it from the `return` statement's position in 13.4.2's code.

13.4.4 Agent-Assisted, Tests First

For each seam above, the pattern is the same one this book has used throughout: write a failing test describing the bad input and the expected graceful behavior, then let Pi propose the handling, then review. 13.4.1 through 13.4.3 showed you the handling directly; here's the test half, for 13.4.2's malformed-JSON case specifically:

```
pi "Add a test to tests/test_llm.py confirming \
    ask_hosted_detailed handles malformed \
```

```
    tool-call-argument JSON without raising, using a \
    mocked client response."
```

A correct response looks like this:

```python
from unittest.mock import MagicMock

from mortgage_calculator_book.llm import ask_hosted_detailed


def test_ask_hosted_detailed_handles_malformed_json(monkeypatch):
    tool_call = MagicMock()
    tool_call.function.arguments = "{not valid json"
    message = MagicMock(tool_calls=[tool_call])
    first_response = MagicMock()
    first_response.choices = [MagicMock(message=message)]

    mock_create = MagicMock(return_value=first_response)
    monkeypatch.setattr(

        "mortgage_calculator_book.llm._client.chat.completions.create"
        ,
        mock_create,
    )

    result = ask_hosted_detailed("some question")

    assert result["tool_called"] is True
    assert result["arguments"] is None
    assert "trouble understanding" in result["answer"]
    assert mock_create.call_count == 1
```

Review it against two things specifically, not just "does it pass." First: this doesn't look like Chapter 11.8.1's mocked test, and it shouldn't — that one mocked `ollama.chat`, which returns plain dictionaries, so its fixtures were dict literals. This mocks OpenAI's client, which returns objects with attributes (`response.choices[0].message.tool_calls[0].function.arguments`), so the fixture has to match — `MagicMock()` objects with attributes set, not dicts. A proposal that hands `call_tool` a dict here would fail for a reason that has nothing to do with whether the actual error handling works.

Second: `mock_create.call_count == 1`, not 2. Look back at 13.4.2's code — the `except json.JSONDecodeError` branch returns immediately, before the second `_client.chat.completions.create` call that would normally send a result back to the model. A test asserting `call_count == 2` would be testing for behavior the code was never supposed to have; if Pi's version does that, it's wrong even though the rest might look reasonable.

13.5 Structured Logging with loguru

13.5.1 Why Logging, Not Just print

By this point in the book, you've almost certainly reached for a `print()` statement at least once to see what a value was mid-debugging (perhaps back in Chapter 5.7's pdb section, or since). That's fine for a single debugging session — it's a poor substitute for a permanent, structured record of what a running system actually did, especially once something goes wrong somewhere you weren't watching.

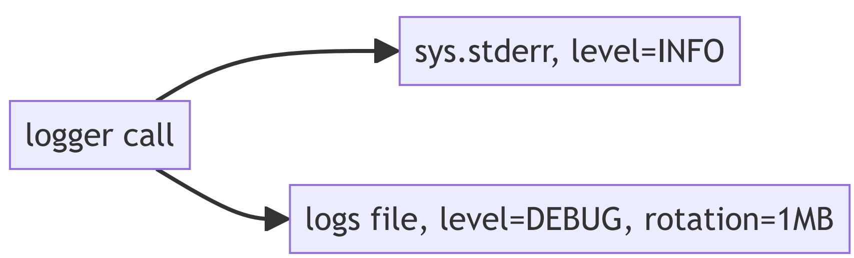
13.5.2 Installing loguru

```
uv add loguru
```

loguru over the standard library's `logging` module specifically because it needs meaningfully less configuration to get something genuinely useful — one call to set up sensible output, rather than the standard library's handler/formatter/logger hierarchy, which is more powerful but considerably more ceremony for a project this size.

In `src/mortgage_calculator_book/logging_config.py`:

```python
import sys

from loguru import logger


def setup_logging() -> None:
    logger.remove()
    logger.add(sys.stderr, level="INFO")
    logger.add(
        "logs/mortgage_calculator_book.log", rotation="1 MB",
        level="DEBUG"
    )
```



One call, two destinations, two different filters: everything INFO and above also goes to your terminal as it happens; everything DEBUG and above, which is everything, goes to the file, capped at 1MB per file before rotating. That's why 13.5.4's log-reading exercise finds a warning in the file that a casual glance at the terminal might have scrolled past.

This is a function, not module-level code that runs on import, deliberately — a bare `import logging_config` for its side effects alone is exactly the kind of thing `ruff check` flags as an unused import (Chapter 3's `F` rules), and fighting your own linter every time you touch this file isn't worth it. Call it explicitly, once, at the top of `main()` in both `cli.py`:

```python
# in cli.py

from mortgage_calculator_book.logging_config import setup_logging


def main(argv: list[str] | None = None) -> int:
    setup_logging()
    parser = build_parser()
    args = parser.parse_args(argv)
    ...
```

and `ui.py`:

```python
# in ui.py

from mortgage_calculator_book.logging_config import setup_logging


def main() -> None:
    setup_logging()
    app = QApplication(sys.argv)
    window = MortgageCalculatorWindow()
    window.show()
    sys.exit(app.exec_())
```

Either front end now configures logging the same way, the moment it starts — one line added near the top of each `main()`, nothing else in either file changes.

13.5.3 Logging at the Seams

Add log calls at each of the seams from 13.4 — starting with the CLI itself, 13.4.1's seam. Right now `main()` only ever prints; nothing in this path calls `logger.*()` at all, which means 13.5.4's exercise below would find an empty log file without this:

```python
# in cli.py

from loguru import logger
```

```
    try:
        data = MortgageInput(
            principal=args.principal,
            annual_rate=args.annual_rate,
            term_years=args.term_years,
            payments_per_year=args.payments_per_year,
        )
    except ValidationError as exc:
        submitted = {
            "principal": args.principal,
            "annual_rate": args.annual_rate,
            "term_years": args.term_years,
            "payments_per_year": args.payments_per_year,
        }
        for error in exc.errors():
            logger.warning(
                "CLI input rejected: {} ({})", submitted,
                error["msg"]
            )
            print(f"Error: {error['msg']}", file=sys.stderr)
        return 1

    payment = round(calculate_validated_payment(data), 2)
    logger.info("CLI computed payment: {}", payment)
    if args.format == "json":
        print(json.dumps({"payment": payment}))
    else:
        print(f"Fixed periodic payment: ${payment:,.2f}")
    return 0
```

The `print()` calls stay exactly as they were — that's still what a person running the CLI actually sees. `logger.*()` is a second, parallel record of the same events, for exactly the situation 13.5.4 describes: reading back what happened after the fact, not watching it happen live.

Next, the seam from 13.4.2, since `loguru` wasn't available yet when that code was first written:

```
# in llm.py, inside ask_hosted_detailed

from loguru import logger

    call = message.tool_calls[0]
    try:
        arguments = json.loads(call.function.arguments)
    except json.JSONDecodeError:
        logger.warning(
            "Malformed tool-call JSON from model: {}",
```

```
            call.function.arguments,
        )
        return {
            "answer": (
                "I had trouble understanding those loan "
                "details - could you restate them?"
            ),
            "tool_called": True,
            "arguments": None,
        }

    result = call_tool(arguments)
```

And the seam from Chapter 10.5.1, in `tool.py`:

```
# in tool.py

from loguru import logger


def call_tool(arguments: dict) -> dict:
    try:
        data = MortgageInput(**arguments)
    except ValidationError as exc:
        logger.warning("Tool call rejected: {} ({})", arguments,
exc)
        return {"error": "; ".join(e["msg"] for e in
        exc.errors())}

    payment = round(calculate_validated_payment(data), 2)
    logger.info("Tool call succeeded: payment={}", payment)
    return {"payment": payment}
```

What not to log: never log `OPENROUTER_API_KEY` or any other secret from `config.py`, even at debug level, even temporarily while troubleshooting — a log file is easy to forget about and easy to accidentally share or commit.

13.5.4 Reading a Log Back

Deliberately trigger a failure, then read what got recorded:

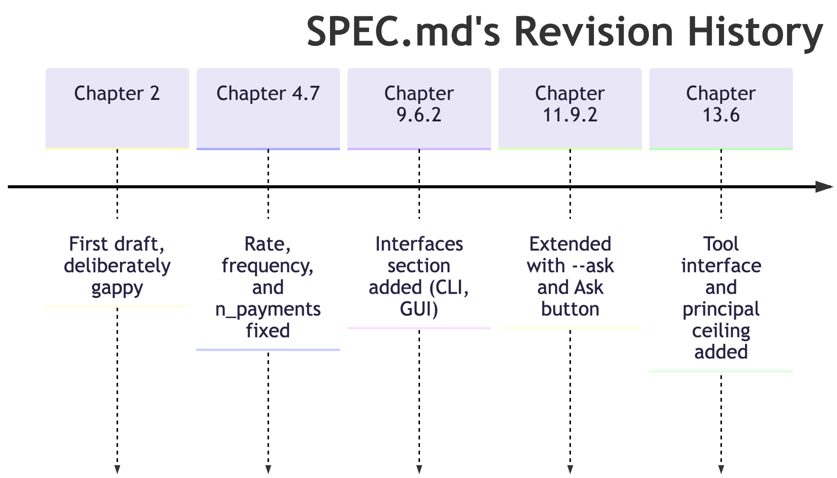
```
mortgage-calculator-book \
    --principal -5000 --annual-rate 0.06 --term-years 30
cat logs/mortgage_calculator_book.log
```

You should see the `WARNING` line from 13.5.3, with the actual rejected arguments and the validation error, timestamped. This is the payoff of this section: a real failure, diagnosed after the fact from a log file alone, the way you'd have to if a user reported a problem you weren't present to watch happen.

13.6 Final Spec and README Check

This section closes the loop on both of this project's living documents — SPEC.md first, since it's been through this before, then README.md in 13.6.3, which hasn't.

This is the fourth time in the book SPEC.md has been opened and found wanting — worth seeing the whole pattern at once before adding to it again:



Not five unrelated edits — one document, checked against reality at five different points as the project grew past what it originally described. This is the last of those checks in this book, not because SPEC.md becomes perfect after it, but because the book does.

13.6.1 Returning to SPEC.md

Open the version last revised in Chapter 11.9.2. Read it fresh, as if you'd never seen the rest of this project, and ask: does this still describe what actually got built?

13.6.2 What's Drifted

Two things stand out. First, section 13.4.1's new principal ceiling is a real constraint that exists in the code now but appears nowhere in the spec. Second: the "Interfaces" section Chapter 9.6.2 started and Chapter 11.9.2 extended lists the CLI, the GUI, and the natural-language additions to both — but not the tool interface those additions actually depend on, from Chapters 10–11. The section describes what a user sees without ever naming the mechanism underneath it.

Reconcile both:

```
## Interfaces
- Command-line interface (human-readable and JSON output; --ask
  for a natural-language question, answered via the tool
interface)
- Desktop GUI (see docs/ui.md for the current layout):
  - Inputs: principal, annual rate, term (years), payments per
year
  - Actions: Calculate, Clear, and Ask (a natural-language
question,
    answered via the tool interface)
  - Invalid input shows an error message in place, not a crash
- Tool interface for language-model use (see tool.py), supporting
both
  local and hosted models

## Inputs
- principal: the loan amount, in dollars (0 < principal <=
100,000,000)
- annual_rate: the annual interest rate, as a decimal (e.g. 0.06
for 6%)
- term_years: the length of the loan, in years
- payments_per_year: number of payments per year (default: 12,
monthly)
```

Only the last bullet under "Interfaces" is actually new here — everything above it should already be sitting in your file from 9.6.2 and 11.9.2. This shows the whole section so you can confirm what's there against what's shown, not because all of it needs retyping.

Commit the reconciliation on its own:

```
git add SPEC.md
git commit -m "Reconcile SPEC.md: add tool interface, \
    document principal ceiling"
git push
```

> **Process concept: closing the loop.** Chapter 2 opened with the idea that a spec is a living document, expected to be wrong in places. Chapter 4 caught and fixed the first round of gaps, once real domain math existed; Chapters 9 and 11 each added more as new interfaces arrived. This is the same check, run one more time, now that the *entire* system exists — and it found exactly the kind of drift this book has been warning about since Chapter 2.2.2: not a mistake, just reality moving faster than the document describing it, caught because someone deliberately went looking.

13.6.3 Reconciling README.md

Chapter 8.8 made the same living-document promise for a different reader — that the README would need another pass once Chapter 9 gave the calculator a second way to run. Chapter 9 added the GUI. Chapter 11 added `--ask` on both the CLI and the GUI, and the tool interface underneath both. None of it ever reached `README.md` — the file still describes the project exactly as it stood at the end of Chapter 8: one command, four structured flags, two output formats.

Not everything from 13.6.2 needs a mirror here. The tool interface itself is written for SPEC.md's collaborator audience, not README's end-user one (8.8's own distinction) — it doesn't need a README section of its own. What does: the two user-visible entry points built on top of it, `--ask` and the GUI's Ask button, since those are exactly the kind of thing a new user opening this file would want to know how to run.

Same tool, same discipline as 8.8.4:

```
pi "Update README.md for this project. It was last written \
    at the end of Chapter 8 and only documents the CLI's \
    structured flags. Read SPEC.md, docs/ui.md, and \
    src/mortgage_calculator_book/cli.py, and add real usage \
    examples for the GUI and for the --ask flag on both the \
    CLI and GUI, based on what you find there rather than \
    guessing at flag names. Don't rewrite sections that are \
    still accurate - only add what's missing."
```

Review it with the same care 8.8.4 asked for, plus one thing specific to this pass: 8.8.4's "run each example yourself" is harder to satisfy here than it was for the structured flags. A `--ask` example only proves out against a real model, local or hosted — the same standard Chapter 11's own checklist held itself to; reading the text and nodding isn't running it. And a GUI example can't be copy-pasted into a terminal at all — confirming it means actually opening the window and following the steps the README claims will work, not just checking that the prose is plausible. Watch too for Pi carrying SPEC.md's "Tool interface" bullet over into README as its own section — per the distinction above, that's collaborator-facing content this file doesn't need.

A reasonable addition looks like this:

```
## Usage

    uv run mortgage-calculator-book \
        --principal 200000 --annual-rate 0.06 --term-years 30
    # Fixed periodic payment: $1,199.10

    uv run mortgage-calculator-book \
```

```
        --principal 200000 --annual-rate 0.06 --term-years 30
--format json
    # {"payment": 1199.1}

    uv run mortgage-calculator-book --ask \
        "What would my payment be on a $200,000 loan at 6% over
30 years?"
    # A plain-language answer containing $1,199.10

## GUI

    uv run python -m mortgage_calculator_book.ui

Fill in the four fields and click Calculate, or type a question
and click Ask - see docs/ui.md for the full layout. Both paths
go through the same validated calculation; neither is a separate
implementation.
```

Commit the reconciliation on its own, same as SPEC.md's:

```
git add README.md
git commit -m "Reconcile README.md: document the GUI and the
--ask flag"
git push
```

Chapter 8.8's promise is finally kept — four chapters late, and only because someone went back and checked.

13.7 The Complete Definition of Done

13.7.1 Assembling Every Checkpoint

Every chapter from 0 through 13 ended with its own checklist. Run through the whole system against all of them at once, not just the most recent chapter's:

- ☐ The project is git-tracked, pushed to GitHub, with a clean commit history (Ch. 0)
- ☐ Pi is configured and every agent-proposed change in this project's history was reviewed before being committed (Ch. 1)
- ☐ `SPEC.md` accurately describes the system as it exists today, including all three interfaces and the principal ceiling (Ch. 2, revised Ch. 4, 9, 11, and 13)
- ☐ `README.md` documents both interfaces (CLI and GUI) with real, working examples, including `--ask` and the Ask button, and is committed (Ch. 8, revised Ch. 13)
- ☐ `ruff check .` and `ruff format .` pass cleanly across the whole project (Ch. 3)

- ☐ The Chapter 4.5 worked example (\$200,000 / 6% / 30yr → \$1,199.10) is verified correct through every front end: CLI, UI, and the tool interface (Ch. 4, 6, 8, 9, 10)
- ☐ The full test suite passes: core, validation, CLI, UI logic, tool, LLM wiring (mocked), and eval scoring (Ch. 5–12)
- ☐ `calculate_payment` remains pure — no I/O, no printing, no external state (Ch. 6)
- ☐ `MortgageInput` is the only path from raw input into the core, for every front end (Ch. 7)
- ☐ `.env` holds real secrets, is git-ignored, and `.env.example` documents what's needed (Ch. 8)
- ☐ The eval set (Ch. 12) has been run against both local and hosted models, with results recorded
- ☐ Every identified seam (13.3) fails gracefully rather than crashing, and is logged (13.4–13.5)

13.7.2 What to Do With What Doesn't Pass

This list is meant to surface a few gaps, not to be a formality you skim and check off. If something's missing — an old test that's been silently skipped, a `.env.example` that's drifted out of date, a README that still only documents the CLI, a front end that was never actually re-tested against the Chapter 4.5 example after some later change — fix it now, with the whole system in front of you, rather than leaving it for a reader (including future-you) to discover.

13.8 Checkpoint

Before moving to the Closing chapter, this should all be true:

- ☐ Every seam identified in 13.3 has explicit, tested error handling
- ☐ loguru is configured and logging real events at each seam, with no secrets ever logged
- ☐ You've deliberately triggered a failure and diagnosed it from the log file alone
- ☐ `SPEC.md` has been reconciled with the system as it actually exists, including interfaces and the principal ceiling
- ☐ `README.md` has been reconciled with the system as it actually exists, including the GUI and the `--ask` flag
- ☐ The full checklist in 13.7.1 has been run against the whole project, not just this chapter's changes

What's next: the Closing chapter looks back across all fourteen chapters — what was hand-written, what was agent-assisted, and why that boundary sat where it did throughout.

Closing

Contents

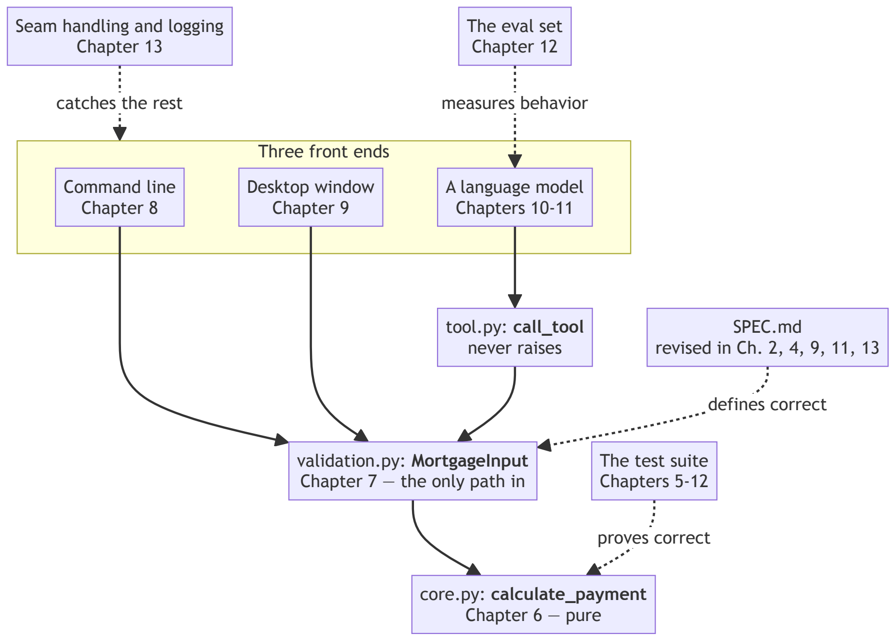
C.1 Why This Chapter Exists

Fourteen chapters is a lot of ground covered one small step at a time. This chapter is a deliberate pause — not a new build step, but a chance to look back at what all those steps actually added up to, before you close the book.

C.2 What Got Built

Underneath everything — the terminal commands, the Pydantic models, the agent-reviewed diffs — one system exists now: a pure mathematical core (Chapter 6), wrapped in a validation layer that enforces what SPEC.md says is valid (Chapter 7), reachable through three genuinely different front ends that all share that same core without duplicating a line of its logic — a command line (Chapter 8), a graphical window (Chapter 9), and a language model that can operate it on a user's behalf (Chapters 10–11). Around all of that: an evaluation discipline that checks the model-facing layer honestly rather than anecdotally (Chapter 12), and a hardening pass that assumes the world outside your code is messier than your tests ever were (Chapter 13). One core, three front ends, held together by a spec that got checked against reality four times and corrected every time.

That paragraph names eight chapters' worth of components in one breath. Here they are with the wiring visible:



This is the same diagram Chapter 11.9 drew, with Chapters 12 and 13's contributions added around the outside. Nothing in the solid path changed after Chapter 11 — the last two chapters didn't extend the system, they made it honest about how well it works and how it fails. The dotted arrows are the ones the next section is about: every one of them starts at something you wrote.

C.3 What Was Hand-Written vs. AI-Assisted, Chapter by Chapter

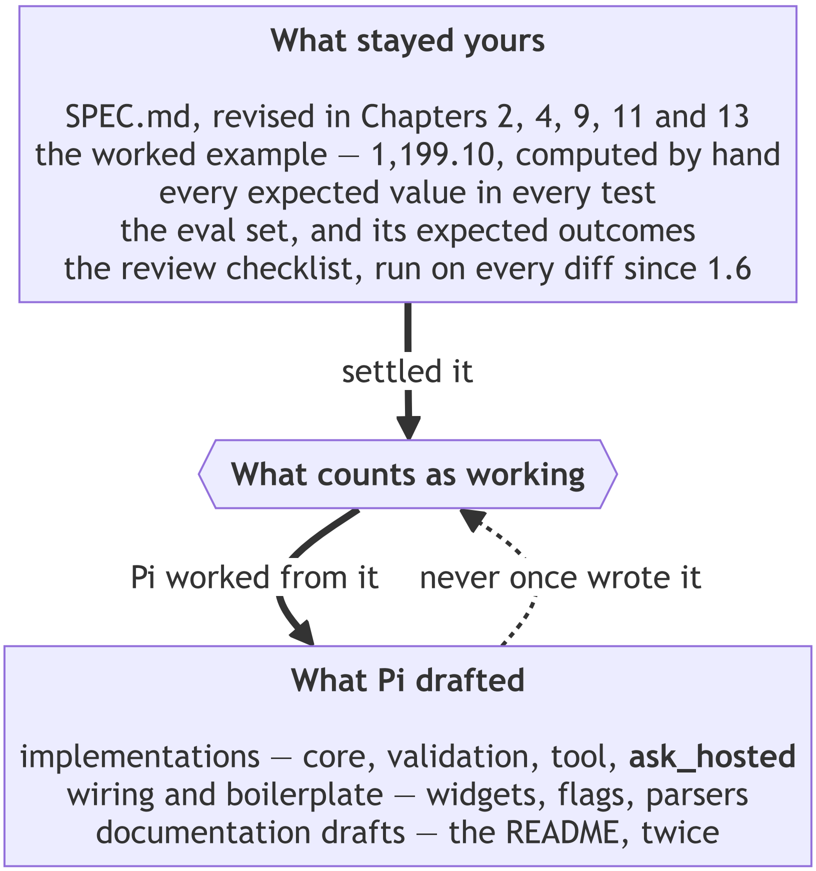
Chapter	Pi's role	What stayed yours
0 — Orientation	None yet	Everything — pure setup
1 — Meet Your Coding Agent	First delegated task (a docstring)	Configuration, review, the habit of reading every diff
2 — Writing SPEC.md	Drafted initial structure	Catching the gaps it papered over (rate vs. periodic rate, unstated frequency)

Chapter	Pi's role	What stayed yours
3 — Holding the Agent to a Standard	Fixed flagged lint issues	Deciding what standard to hold it to
4 — Fixed Mortgages	Optional arithmetic double-check	The entire derivation, the worked example, the spec revision
5 — Tests & TDD	Drafted additional test skeletons	The worked-example test and both edge-case tests, and every expected value
6 — Mathematical Core Library	Implementation draft	The formula translation, the review checklist (rate conversion, off-by-one, float safety)
7 — Validation	Implementation draft	Every boundary condition, sourced directly from SPEC.md
8 — CLI	Wiring implementation	The command shape, the stderr/stdout distinction, the `.env` pattern
9 — Calculator UI	Boilerplate widget wiring	Layout judgment, the decision not to test the GUI directly
10 — Tool Interface	Implementation draft	`TOOL_DESCRIPTION`'s wording — the one piece of this project that's really prompt engineering
11 — LLM Interface	Drafted `ask_hosted` from `ask_local`'s pattern	`ask_local` itself, the two-model distinction, the closest review in the book
12 — Evaluation	None — the eval set is entirely hand-authored	Every eval question and its expected outcome — the actual judgment the whole chapter rests on
13 — Hardening	Proposed handling per seam, drafted the README update	Identifying the seams themselves, the logging policy, the final spec and README reconciliation

C.4 Why the Boundary Sat Where It Did

Look at the right-hand column of that table as a whole, rather than row by row, and a pattern emerges: Pi drafted implementations, wiring, and boilerplate — real, useful work — but it never once decided what "correct" meant. That decision stayed encoded in things you wrote yourself: SPEC.md's revisions, the worked example's exact numbers, the eval set's expected outcomes, the review checklists you ran every proposed change against. Pi worked *from* those artifacts. It never got to write them.

That's the front matter's diagram again, with fourteen chapters of real artifacts filled in where the abstractions were:



Every arrow still points the same way it did before Chapter 0. The top box got longer and far more specific over fourteen chapters; the direction of the heavy arrows never changed once. That's the whole claim — not that the bottom box is small, it plainly isn't, but that nothing in it ever moved up.

That's the actual thesis of this book, stated plainly now that there's a whole system to point at rather than an abstract claim in the front matter: a hybrid system isn't defined by how much of the code an agent wrote. It's defined by who — or what — gets to decide what counts as working. Keep that decision yours, and an agent drafting most of your implementation code is a force multiplier. Let that decision drift to the agent, even a little at a time, across enough chapters, and "hybrid" quietly becomes "the agent did it," which was never the point.

That risk doesn't expire when the book does. Every project you build after this one will offer the same quiet trade — accept a plausible-looking output instead of actually understanding it — and the tools available for making that trade easier will only get better, not worse. The habits this book tried to build — reading every diff since Chapter 1.6, writing the test before the implementation since Chapter 5, catching a spec against reality since Chapter 4.7 — are worth keeping specifically because the temptation they're guarding against isn't going away.

C.5 Where to Apply This Pattern Next

The shape generalizes cleanly: pure core, validation layer, multiple front ends — including one for a model — evaluation, hardening. Nothing about that shape is specific to mortgages. Pick a domain you already understand well enough to write a Chapter 4-style primer for — something you could derive a formula or a rule for by hand, and verify a worked example against, the same way this book verified $1,199.10 before a single line of `core.py` existed. That primer is the real starting point for your next project, more than any of the tooling chapters were.

The Appendix is still there, with Docker, CI/CD, type checking, and the rest, named and briefly described, for whenever a future project actually needs one of them.

C.6 A Closing Note

This wasn't a book about how impressive an AI coding agent can be — plenty of other places will make that case for you, and reasonable people are still working out how much of it holds up. It was a book about a narrower, more durable claim: the discipline this book asked you to practice — a written spec, tests before implementation, a habit of reading every diff, an honest eval instead of an anecdote — is what made the agent genuinely useful here, not the other way around. Keep the discipline. The tools will keep changing.

Appendix — Where to Go Next

Contents

A.1 Why This Appendix Exists

Fourteen chapters back, the front matter promised a note on what this book deliberately left out, and why leaving it out was a choice rather than an oversight. This is that note, expanded. Each entry below gets a paragraph: what the tool or practice solves, why it stayed out of scope here, and where it would slot into a project like this one if you decided to add it later. Nothing here is a tutorial — think of it as a map for your next project, not homework for this one.

A.2 Docker

Docker packages an application together with everything it needs to run — system libraries, Python version, dependencies — into a single, portable image that behaves identically on any machine that runs it. What it solves that this book's `uv`-based setup (Chapter 0.7) doesn't: reproducibility *across* machines, not just within one. `uv.lock` guarantees that everyone working on this project gets the same Python packages; it doesn't guarantee the same operating system, the same system libraries, or the same version of Ollama sitting alongside it. Docker was left out because that gap genuinely doesn't matter for a single-developer desktop application — it starts mattering once you're deploying to a server you don't control by hand, or handing the project to someone whose machine looks nothing like yours. If you wanted to add it, the natural place is packaging the CLI (Chapter 8) or the tool interface (Chapter 10) as a standalone service, not the PyQt5 UI, which doesn't containerize as cleanly.

A.3 CI/CD (GitHub Actions)

Continuous integration means running your checks — Ruff (Chapter 3), pytest (Chapters 5 onward), the eval set (Chapter 12) — automatically, on every push, rather than by habit. It exists to catch the day you forget to run `ruff check .` before committing, which this book has been asking you to remember by hand since Chapter 3.6.1. GitHub Actions, specifically, connects directly to the GitHub account and repository set up in Chapter 0.4 — of everything in this appendix, it's the most natural next step, since the infrastructure it needs already exists. A minimal workflow for this project would run, in order: Ruff, then pytest, then (optionally, since it costs real API calls) the eval set — failing the whole check if any step fails. Left out here specifically so the *reason* you run these checks stayed the lesson, rather than the checks becoming something that happens invisibly before you've felt why they matter.

A.4 mypy / Type Checking

Python's type hints — used informally throughout this project, in `MortgageInput`'s field types (Chapter 7) and every function signature since Chapter 6 — are optional and unenforced by Python itself. mypy is a separate tool that reads those hints and checks them *before* the code runs, catching a category of mistake (passing a string where a function expects a number, say) that would otherwise only surface at runtime, or not at all if the buggy path never gets exercised by a test. It was left out because Pydantic already provides real, *runtime* enforcement for the inputs that matter most in this project (Chapter 7) — mypy would add a second, static layer of type-checking on top of that, genuinely useful but genuinely additional cognitive load for a first read of this book. It would matter most applied to `core.py` (Chapter 6) and `tool.py` (Chapter 10), where a type mismatch is easiest to introduce by accident and hardest to notice without either a very specific test or a type checker looking over your shoulder.

A.5 pre-commit Hooks

pre-commit automates exactly the habit Chapters 3.6.1 and 6.8.2 asked you to build manually: running Ruff (and, if you've added it, mypy) automatically every time you run `git commit`, rather than trusting yourself to remember. It's genuinely a small, low-cost addition once the habit already exists — which is precisely why this book didn't start with it. Automating a check away before you've felt the cost of skipping it teaches the tool, not the judgment; this book chose to teach the judgment first; you already know when and why you'd want this.

A.6 Typer

Chapter 8.3.4 flagged this in passing: Typer builds a CLI from Python type hints directly, producing less boilerplate and friendlier help output than the `argparse` this book actually used. What would change if you swapped it in: mostly ergonomics — nicer `--help` text, less manual parser configuration — not architecture. Because Chapter 8's CLI already sits cleanly on top of `MortgageInput` and `calculate_validated_payment` (Chapter 7), replacing `argparse` with Typer touches `cli.py` alone; nothing about validation or the core would need to change. A reasonable exercise once you've finished this book, if you want to feel the difference directly.

A.7 Direct PyQt5 Widget Testing

Chapter 9.7 pulled `parse_form_values` out of the UI specifically so it could be tested without a running window, and stopped there rather than testing the widget itself. Worth being precise about why: not because it's impossible, and not because it needs a new dependency. PyQt5 can drive a widget directly — instantiate it, call `.click()` on a button, emit a signal like `returnPressed`, and check what changed — inside a `QApplication` running in offscreen mode

(`QT_QPA_PLATFORM=offscreen`, set before the application is created), with nothing beyond `pytest` and `PyQt5` already in this project. A session-scoped fixture holding one shared `QApplication` is the only new piece, and it's a handful of lines, not a new tool.

What holds it out of the main chapters is concept load, not setup cost: an offscreen platform plugin, a singleton application shared across tests, and the idea of driving a widget without a real display are three new things at once, on top of everything else Chapter 9 was already teaching. Worth trying directly if you want to feel how close it actually is — write one test that builds the calculator window, sets its fields, calls `.click()` on Calculate, and asserts on the result label. If it passes on the first real try, that's the whole technique.

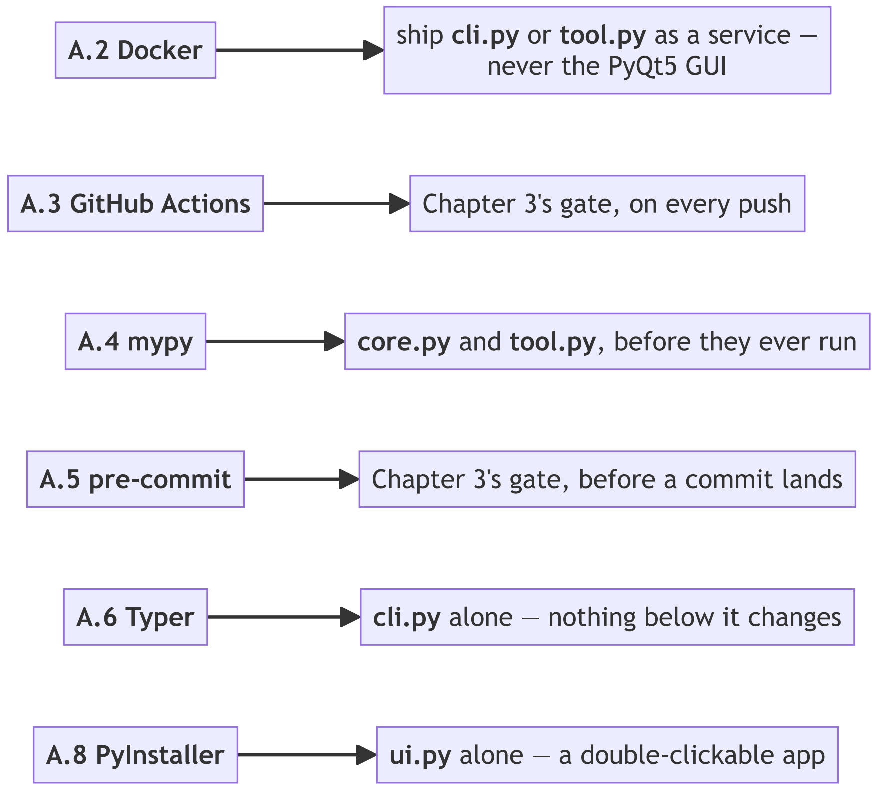
A.8 Packaging with PyInstaller

PyInstaller bundles a Python application — the interpreter, every dependency, your own code — into a single distributable file: a `.app` on macOS, an `.exe` on Windows, a standalone binary on Linux. What it solves that `uv run python -m mortgage_calculator_book.ui` doesn't: handing the calculator to someone who has never heard of Python, never installed `uv`, and shouldn't have to — someone who just wants to double-click something and see a window open. It was left out because every front end in this book assumes its reader is already following along in a terminal, with the project's dependencies already set up the way Chapter 0 left them; packaging solves a distribution problem this book's reader doesn't have yet, not a technical one standing in the way of finishing the project. If you wanted to add it, the natural place is the GUI specifically (Chapter 9) — the CLI and tool interface are already comfortable living inside a terminal, and Docker (A.2) is the better fit if either of those ever needs to run as a standalone service; a desktop window double-clicked into existence is the one front end here a packaged binary actually suits.

A.9 Beyond This Book

Strip away the mortgage-specific details and what's left is a pattern with nothing to do with mortgages: a pure, human-verified core; a validation layer wrapping it; multiple front ends on top — some for humans, one for a model; an evaluation discipline that checks the model-facing layer honestly, not anecdotally; and a hardening pass that assumes the world outside your code is messier than your tests — a pattern reusable well beyond a fixed-rate payment calculation.

Every entry above named the place it would attach to. Collected onto the finished system, they stop being six separate essays and become one map of a next project:



Two of them attach to the same place, which is the appendix's own hint about ordering: pre-commit and GitHub Actions both automate Chapter 3's quality gate, one before a commit lands and one after it's pushed. Adding either is a smaller step than it sounds, and A.3 is the most natural first one, since Chapter 0.4 already built the GitHub account and repository it needs.

Worth sitting with before you close this book: what domain do you understand well enough to write a Chapter 4-style primer for — one you could hand-derive a formula from, verify an example against, and build the same fourteen-chapter shape around? That's the transferable skill this book was really about, more than any formula or tool.

License